



Fernuniversität Hagen
Fakultät für Mathematik und Informatik

in Kooperation mit dem
Bundesamt für Sicherheit in der Informationstechnik

Abschlussarbeit

**Entwicklung eines Konzeptes für ein Framework zur
teilautomatisierten Erstellung von prozessbasierten IDS-Regeln in
ICS**

von
Brigitte Fischer
Studiengang Bachelor of Science Informatik

Hinweis:

Die Arbeit wurde auf Wunsch des BSI nach der Abgabe um weitere Informationen ergänzt.
Personenbezogene Daten, die nicht forschungsrelevant sind, wurden vor der Veröffentlichung entfernt.

Kurzfassung/Abstract

Die vorliegende Bachelorarbeit hatte das Ziel, ein Konzept für ein Framework zur teilautomatisierten Erstellung von prozessbasierten IDS-Regeln in ICS zu erstellen. Zuerst wurden Grundlagen über den generellen Aufbau von ICS, die Unterschiede von einem ICS zur herkömmlichen IT und über die S7-Kommunikation vermittelt. Darauf aufbauend wurden die Anforderungen an das Framework abgeleitet. Im Weiteren wurden die kritischen Befehle und Operationen identifiziert und klassifiziert. Auf diesen Erkenntnissen basierend wurde eine Vorgehensweise für die Regelerstellung beschrieben. Das Konzept wurde für das S7comm-Protokoll angewendet. Anhand der Analyse von PCAPS wurden Regeln für das S7comm-Protokoll erstellt. In einer Testanlage wurden unter der Aufzeichnung von Netzwerkmitschnitten kritische Operation durchgeführt. Die erstellten Regeln wurden in eine virtuelle Suricata-Maschine implementiert und getestet. Mit dem entwickelten Konzept ist es möglich, allgemeine Regeln zu erstellen, die für viele Operationen zutreffend sind.

The aim of this bachelor thesis was to develop a concept for a framework for the semi-automated creation of process-based IDS rules in ICS. First, the basics about the general structure of ICS, the differences between an ICS and conventional IT and about S7 communication were taught. Based on this, the requirements for the framework were derived. Furthermore, the critical commands and operations were identified and classified. Based on these findings, a procedure for rule creation was described. The concept was used for the S7comm protocol. Based on the analysis of the PCAPS, rules for the S7comm protocol were created. In a testbed, critical operations were performed by recording network recordings. The rules were implemented and tested in a virtual machine running Suricata. With the developed concept it is possible to create general rules that apply to many operations.

Inhaltsverzeichnis

Abbildungsverzeichnis	V
Tabellenverzeichnis	VI
Listings	VII
Abkürzungsverzeichnis	VIII
1 Einleitung	1
1.1 Motivation	1
1.2 Zielsetzung	2
1.3 Gliederung	2
2 Grundlagen	3
2.1 Informationstechnologie (IT)-Sicherheit	3
2.1.1 Schutzziele	3
2.1.2 Gefährdung	4
2.1.3 Schwachstellen	4
2.1.4 Bedrohungen	4
2.1.5 IT-Sicherheitsmaßnahmen	5
2.2 Aufbau und Struktur eines Industrial Control Systems (ICS)	5
2.3 IT- und Operative Technik (OT)-Unterschiede	8
2.3.1 Überblick	8
2.3.2 Operative Unterschiede	8
2.3.3 Technologische Aspekte	9
2.3.4 Verwaltungstechnische Unterschiede	10
2.4 Die Siemens S7-Kommunikation	12
2.4.1 Die S7 Protocol Data Unit	12
2.4.2 Authentifizierung und Schutz	13
2.4.3 Lese- und Schreib-Variable	14
2.5 Einführung in Intrusion Detection Systeme	14
2.5.1 Hostbasiertes Intrusion Detection System	14
2.5.2 Netzwerkbasiertes Intrusion Detection System	15
2.5.3 Netzwerk-basierte Intrusion Detection Systeme in ICS	15
2.6 Stand der Forschung	17

3	Konzept des Frameworks	20
3.1	Kritische Befehle	20
3.2	Anforderungen an das Framework	28
3.3	Regelerstellung für Intrusion Detection System (IDS)-Gruppen	29
3.3.1	Einfache Regeln	29
3.3.2	Protokollregeln	30
3.3.3	Threshold Regeln	30
3.3.4	Exploit Regeln	30
3.3.5	Adressbereich Regeln	31
3.3.6	Allgemeine Regeln	31
4	Anwendung des Framework-Konzeptes	32
4.1	Prozesse der Testanlage	32
4.2	Analyse der PCAPS	34
4.2.1	S7comm Kommunikation	35
4.2.2	S7comm Protokollstruktur	36
4.3	Regelerstellung S7comm	40
4.4	Anwendung der erstellten Regeln	43
4.4.1	Einfache Regeln	43
4.4.2	Protokollregeln	45
4.4.3	Thresholdregeln	49
4.4.4	Exploitregeln	50
4.4.5	Adressbereich Regeln	51
4.4.6	Allgemeine Regeln	51
5	Diskussion	52
6	Fazit und Ausblick	54
	Ausblick	55
7	Literaturverzeichnis	56
	Anhänge	60
A	Liste der PCAPS	60
A.1	Liste der PCAPS für den Entwurf des Frameworks (2017QUT_S7Comm dataset)	60
A.2	Liste der erstellten PCAPS für die Anwendung der Regeln	60
B	Regeln für das Protokoll s7comm	64

Abbildungsverzeichnis

2.1	SCADA Konfiguration	6
2.2	PERA Modell	7
2.3	Herausforderungen OT gegenüber IT	8
2.4	Operative Anforderungen an OT vs. IT	9
2.5	Position im ISO-OSI-Referenz-Modell.	12
2.6	S7comm - Protocol Encapsulation	13
2.7	Aufbau S7comm Header	13
3.1	Generierte Regel-SID S7comm	29
4.1	Prozessübersichtsanzeige einer Branium-Raffinerie	33
4.2	Netzplan der Testanlage	34
4.3	Kommunikation HMI und Master-PLC	36
4.4	Kommunikation Angreifer und Master-PLC	36
4.5	S7comm-Paket mit Header, Parameter und Daten	37
4.6	Adressierung im Speicherbereich Merker	39
4.7	Adressierung im Speicherbereich DB	40
4.8	DB-Type Adressierung eines FB	40
4.9	Verbindungsaufbau Host und Master-PLC	43
4.10	Setup Communication Request	44
4.11	Setup Communication Ergebnis	44
4.12	Ausgabe eines Verbindungsaufbau in Suricata	45
4.13	Befehlsabfolge eines Downloads	46
4.14	Suricata-Ergebnis zum Download	47
4.15	Vorlage Eventfilter	49
4.16	Diagramm Schreib- und Leseoperationen	51

Tabellenverzeichnis

2.1	Netzwerktafel IT und OT	10
2.2	OT-Beschränkungen vs. Informationssicherheitskontrollen	11
3.1	Kritische Befehle und Funktionen	21
3.2	Gruppe 01_Download	22
3.3	Gruppe 02_Upload	22
3.4	Gruppe 03_Update	23
3.5	Gruppe 04_Upgrade	23
3.6	Gruppe 05_Date-Time	24
3.7	Gruppe 06_Variable	24
3.8	Gruppe 07_Value	25
3.9	Gruppe 08_Address	25
3.10	Gruppe 09_Authentication	26
3.11	Gruppe 10_Mode	26
3.12	Gruppe 11_Command	27
3.13	Gruppe 12_Communication	27
3.14	Gruppe 13_Functions	27
3.15	Gruppe 14_Memory	28
3.16	Gruppe 15_Reset	28
4.1	IP-Adresszuordnung zu den Hosts	35
4.2	Parameter Function S7comm	38
4.3	Zulässige Adressen Waschtank	39
4.4	Kritische Befehle S7comm	41
4.5	Userdata Functions S7comm	49

Listings

4.1	Regel-Muster	41
4.2	S7comm-Regel zum Kommunikationsaufbau in Suricata	42
4.3	Regeln zum 'Request download' in S7comm	46
4.4	Eventfilter	50
4.5	Threshold Regel	50
B.1	Variablen mit IP-Zuordnung	64
B.2	Security Function Rules	65
B.3	Setup Communication Rules	65
B.4	Download Rules	66
B.5	Upload Rules	68
B.6	Time Function Rules	70
B.7	Value Rules	71
B.8	Mode Rules	71
B.9	Function Rules	73
B.10	Memory Rules	75
B.11	Reset Rules	77
B.12	Threshold Rules	78
B.13	Generic Rules	78
B.14	Eventfilter	78

Abkürzungsverzeichnis

ALDI	Additional Layer of Detection for ICS
CRC	Cyclic Redundancy Check
COTP	Connection Oriented Transport Protocol
DB	Datenblock
DCS	Distributed Control System
DFA	Deterministic Finite Automaton
DMZ	Demilitarized Zone
DoS	Denial of Service
EWS	Engineering Workstation
FB	Functionblock
FC	Function
FTP	File Transfer Protocol
HIDS	Host Intrusion Detection System
HMI	Human Machine Interface
ICS	Industrial Control Systems
ID	Identifier
IDS	Intrusion Detection System
IP	Internet Protocol
IPS	Intrusion Prevention System
ISO	International Organization for Standardization
IT	Informationstechnologie

LDAP	Lightweight Directory Access Protocol
NIDS	Network Intrusion Detection System
NSM	Network Security Monitoring
OB	Organsiationbaustein
OISF	Open Information Security Foundation
OSI	Open Systems Interconnection
OT	Operative Technik
PCAP	Packet Capture
PERA	Purdue Enterprise Reference Architecture
PDU	Protocol Data Unit
PIN	Persönliche Identifikationsnummer
PLC	Programmable Logic Controller
RFC	Request for Comments
RFU	Reserved For Future Use
RTU	Remote Terminal Unit
SCADA	Supervisory Control and Data Acquisition
SDB	Systemdatablock
SFB	Systemfunktionsblock
SFC	Systemfunction
SID	Security Identifier
SIEM	Security Information and Event Management
SPS	speicherprogrammierbare Steuerung
TAN	Transaktionsnummer
TCP	Transmission Control Protocol
TCP-PKT	Transmission Control Protocol Paket
TPKT	Transport Packet

UDP	User Datagram Protocol
VFD	Variable Frequency Drives
VLAN	Virtual Local Area Network
VM	Virtuelle Maschine

1 Einleitung

1.1 Motivation

In den vergangenen zwei Jahren wurden sieben von zehn Industrieunternehmen Opfer von Sabotage, Datendiebstahl oder Spionage. Der daraus resultierende wirtschaftliche Schaden war hoch. Jedes fünfte Unternehmen (19%) berichtete von digitaler Sabotage von Informations- und Produktionssystemen oder Betriebsabläufen. [1]

Gerade die erfolgreichen Angriffe auf industrielle Anlagen, wie Stuxnet, Industroyer und Triton [2, 3, 4], haben die Gefahren für die Bevölkerung demonstriert, welche von diesen Angriffen ausgehen. Ziele waren stets kritische Infrastrukturen, die eine besondere Bedeutung für das staatliche Gemeinwesen haben. Ein Ausfall kann nachhaltig zu Versorgungsengpässen führen und erhebliche Störungen der öffentlichen Sicherheit verursachen. [5, 6, 7]

Zum Messen, Steuern und Regeln von Abläufen, beispielsweise zur Automation von Prozessen und zur Überwachung von großen Systemen, kommen in vielen Bereichen der Industrie Industrial Control Systems (ICS) zum Einsatz. Diese finden häufig Verwendung in der produzierenden Industrie und in Branchen, die zu den kritischen Infrastrukturen gezählt werden [8]. Aus historischen Gründen waren hier Verfügbarkeit gefolgt von Integrität von besonderer Bedeutung. Vertraulichkeit war oft eine untergeordnetes Schutzziel. Dies führte dazu, dass in der Vergangenheit Cybersicherheit bei Design- und Betriebsüberlegungen für ICS kaum eine Rolle spielte. Die verwendeten obligatorischen Kommunikationsprotokolle stammen zu einem großen Teil aus einer Zeit, in der ICS autark betrieben wurden und keine Netzwerkkonnektivität zur Unternehmens-Informationstechnologie (Unternehmens-IT) oder in das Internet hatten. Die vielfältigen ICS-Protokolle schützten nicht vor unautorisiertem Zugriff auf Protokollnachrichten, weil keine Schutzfunktionen implementiert waren [5].

Die in klassischen IT-Systemen bewährten Vulnerability Scanner liefern handfeste Informationen über Server, Dienste, Netzwerkkomponenten und potentielle Verwundbarkeiten. Deren Einsatz hat jedoch den Nachteil, dass die aktiven Scanner selbst Netzwerkverkehr erzeugen. Passives Monitoring erzeugt keinen eigenen Netzwerkverkehr. Es lassen sich damit Informationen über Unternehmen, Personen und teilweise über Infrastrukturen sammeln. Ein Nachteil passiven Monitorings ist, dass nur aktive Geräte erkannt werden können. Teilweise treten hohe Falsch-Positiv-Ergebnisse in Bezug auf die Verhaltensanalyse eines Systems auf. [9].

Intrusion Detection Systeme (IDS) sind Monitoring-Systeme, die Bedrohungen des Netzwerks

erkennen. Sie werden zum Schutz von Computersystemen eingesetzt, weil sie bekannte Signaturen bössartigen Codes oder das Verhalten eines Systems analysieren und Anomaliedektoren identifizieren. Hier stellt sich die Frage, inwieweit etablierte Schutzmaßnahmen aus herkömmlichen IT-Systemen auf ICS übertragbar sind.

Für verschiedene Betriebssysteme in Computersystemen sind Open-Source- und kommerzielle Lösungen entwickelt worden. Die proprietären ICS-Protokolle unterscheiden sich von herkömmlichen IT-Protokollen. Grundsätzliche und allgemein gehaltene IDS-Regeln sind für die Protokolle meistens vorhanden. Sie sind aber nicht trivial nutzbar oder bieten keinen zusätzlichen Schutz. Der Aufwand für eine Erstellung oder die Anpassung von Regeln für einzelne Anlagen oder Prozesse ist hoch. [5]

1.2 Zielsetzung

Ziel dieser Bachelorarbeit ist es, ein Konzept für ein grundsätzliches Framework für teilautomatisierte Regelerstellungen mit Benutzerverifikation zu erarbeiten. Diese Arbeit konzentriert sich auf die Grundlagen zur Erstellung von IDS-Regeln. Damit kann der Anwender seine Regeln performant erzeugen. Möglicherweise lässt sich auf diese Art eine höhere Akzeptanz für die Anwendung von IDS in ICS-Netzwerken erzielen.

Das Konzept basiert auf der Idee, protokollunabhängig kritische Befehlsgruppen zu identifizieren. Jedes Protokoll wird geprüft, ob solche kritischen Befehle vorhanden sind. Anhand des Siemens-Protokoll *S7comm* wird dieses Vorgehen getestet, um für spätere Arbeiten eine automatisierte Lösung implementieren zu können. Es wird ein Regelwerk erarbeitet, das ins Open Source IDS Suricata in einer virtuellen Maschine Virtuelle Maschine (VM) implementiert wird. Für verschiedene kritische Szenarien wird der Netzwerk-Traffic einer Testanlage, die eine Raffinerie simuliert, mitgeschnitten. Die daraus entstehenden PCAPS werden mit einer Suricata-VM analysiert.

Die Ergebnisse dieser Arbeit bieten eine Grundlage für weitere Forschungsarbeiten für Entwicklung und Implementierung eines allgemeinen Frameworks zur Erstellung von prozessbasierten IDS-Regeln in ICS.

1.3 Gliederung

Die Arbeit ist wie folgend gegliedert: Kapitel 2 behandelt Grundlagen zu IDS in ICS und verwandte Arbeiten zur Thematik. Das Konzept des Frameworks mit den kritischen Befehlen wird im 3. Abschnitt erarbeitet. Das darauf folgende Kapitel stellt die Regelerstellung zum Protokoll *S7comm* und die Testdurchführung des Konzepts vor. Ein Fazit und Ausblick beenden die Arbeit.

2 Grundlagen

Seit dem Bekanntwerden von Stuxnet im Jahr 2010 sind verschiedene Forschungsprojekte entstanden, welche eine Anomalieerkennung anhand von Netzwerkverkehr in ICS durchführen. Diese Arbeit soll die bisherigen Forschungsergebnisse ergänzen.

Um nachvollziehen zu können, warum für ICS spezielle IDS entwickelt werden sollten, ist es erforderlich, die Unterschiede von der klassischen IT zu ICS zu kennen.

Zur Abgrenzung von klassischen IT-Netzwerken werden in diesem Abschnitt zunächst Grundlagen zur IT-Sicherheit und die wichtigsten Unterschiede zwischen IT und ICS aufgezeigt. Anschließend wird die generelle Funktionsweise des *S7comm*-Protokolls erklärt. Zuletzt werden in diesem Abschnitt die bisherigen Forschungsergebnisse betrachtet. Im Anschluss wird der Ergänzungswert dieser Arbeit zum derzeitigen Forschungsstand dargestellt.

2.1 IT-Sicherheit

Ein Umstand, der es den Angreifern zunehmend leichter macht, in ein Netz mit Operativer Technik (OT) einzudringen, ist die steigende Vernetzung mit IT-Systemen aus Büroumgebungen und Unternehmensdomänen. In neueren Produktionssystemen kann eine Vernetzung mit Internet- und Cloud-Lösungen realisiert sein [8]. ICS-Protokolle basieren zunehmend auf Internet Protocol (IP) und verwenden Netzwerke wie Ethernet. OT-Geräten wird dadurch ermöglicht, viele in der IT üblichen Netzwerkdienste zu nutzen [5].

Dies hat zur Folge, dass ICS ähnlichen Gefährdungen wie klassischen IT-Systemen aus Unternehmensumgebungen ausgesetzt sind. Finden potentielle Angreifer unberechtigten Zugang in die IT-Systeme von Unternehmen, kann das Netzwerk ausgekundschaftet und weitere Systeme infiziert werden. Ist das eigentliche Zielsystem erreicht, können folgenschwere Manipulationen vorgenommen werden [8]. Hier besteht ein Bedarf an wirksamen IT-Sicherheitsmaßnahmen, um Systeme und Komponenten vor unbefugten Operationen zu bewahren. [10].

2.1.1 Schutzziele

Innerhalb von IT-Systemen haben Ingenieure und Betreiber das Ziel, elektronisch gespeicherte Informationen und deren Verarbeitung und Verwaltung zu schützen. Die Sicherheit konzentriert sich auf die Aufrechterhaltung von Integrität, Vertraulichkeit und Verfügbarkeit der Daten. Risi-

ken betreffen die nachhaltige Störung von Geschäftsprozessen. Der Fokus liegt auf den Schutzzielen Integrität und Vertraulichkeit.

2.1.2 Gefährdung

Eine Gefährdung eines IT-Systems setzt voraus, dass eine Bedrohung und eine Schwachstelle zugrunde liegen. Eine Bedrohung allein wäre für ein perfekt abgesichertes System nicht gefährlich. Erst dadurch, dass das System eine Schwachstelle aufweist, die durch eine Bedrohung ausgenutzt werden kann, entsteht eine Gefährdung. [11]

2.1.3 Schwachstellen

Schwachstellen in IT-Systemen sind allgegenwärtig. Täglich werden neue Schwachstellen bekannt. Fast ebenso oft gibt es für irgendein Produkt ein neues Sicherheits-Update, um gefundene Schwachstellen zu beseitigen.

Schwachstellen können an vielen Stellen auftreten: In der Software und Hardware, in der Netztopologie, in der Infrastruktur des Systems und im Verhalten des Anwenders. Die Ursachen für Schwachstellen können in der Konzeption, technischen Realisierung, Konfiguration oder in Verhaltensfehlern eines Systems und den umgebenden Komponenten wie z. B. Personen, Räumlichkeiten oder Regelungen liegen.

Ein Beispiel für eine konzeptionelle Schwäche ist die Übertragung von Passwörtern im Klartext im Übertragungsprotokoll File Transfer Protocol (FTP). Fehler in rechtlichen und organisatorischen Regelungen fallen auch unter die konzeptionellen Schwächen. Dies ist z. B. der Fall, wenn kein Vertreter für den einzigen Administrator einer Firma vorgesehen ist.

Fehler entstehen bei der Implementierung oder der technischen Realisierung der Konzepte. Ein Implementierungsfehler wäre, dass in einer Konzeption zwei physikalisch getrennte Netze geplant sind, eines mit Internetzugang und ein internes, von außen nicht zugängliches Netz. In der Umsetzung werden sie aber nur logisch über Virtual Local Area Network (VLAN)s, die über Switches verbunden sind, getrennt. Manchmal werden Standardkonfigurationen eines neu installierten Systems beibehalten und nicht an die eigenen Gegebenheiten angepasst. [11]

2.1.4 Bedrohungen

Bedrohungen in IT-Systemen gehen in der Regel vom menschlichen Handeln aus. Unterschieden wird zwischen Fahrlässigkeit und Vorsatz. Eine fahrlässige Bedrohung für Daten und Systeme besteht, wenn IT-Anwender oder Mitarbeiter im IT-Bereich Fehler bei der Arbeit machen. Bei Vorsatz geht es um eine absichtliche Bedrohung der Daten und Systeme. Sie kann von Außen- oder Innentätern ausgehen. Die Bedrohung kann auf ein bestimmtes Ziel gerichtet sein oder allgemein sein.

Ein Angreifer kann versuchen, in Systeme einzudringen oder Systeme zu übernehmen. Das erfolgt in der Regel, um anschließend weitere Angriffe auszuüben.

Das Ausspähen oder Entwenden von Daten stellt einen Angriff auf die Vertraulichkeit dar. Dies kann durch Mitschneiden von Netzverkehr, Ausleiten von Daten über Netzwerke oder Kopieren

von Daten auf externe Datenträger geschehen.

Die Verfügbarkeit eines Systems oder der Daten kann beeinträchtigt werden, indem die Server einer Firma mit Anfragen überflutet werden, bis die Server weitere Anfragen verweigern. Hierbei handelt es sich um einen Denial of Service (DoS).

Das Verändern von Daten oder das Täuschen über die eigene Identität stellen eine Bedrohung für die Integrität bzw. Authentizität dar. Hierunter fällt insbesondere das Vortäuschen falscher Identitätsmerkmale, das auch Maskerade oder Spoofing genannt wird. Beispielsweise kann ein Angreifer die Seite einer Bank auf den eigenen Server kopieren um Persönliche Identifikationsnummer (PIN) und Transaktionsnummer (TAN) von Benutzern abzufangen. Ein klassisches Verändern von Daten ist z. B. das Einbringen von Viren oder von Ransomware, die Daten des angegriffenen Systems verschlüsselt. [11]

2.1.5 IT-Sicherheitsmaßnahmen

Viele Angreifer, deren Ziel OT-Systeme sind, dringen über Schwachstellen der klassischen IT in Netzwerke ein. Es ist erforderlich, die speziellen Schwachstellen der Unternehmens-IT zu identifizieren und durch entsprechende Sicherheitsmaßnahmen gegen potentielle Angreifer zu schützen. Wenn es schwieriger ist, in die Unternehmens-IT einzudringen, sind die OT-Systeme stärker geschützt. Trotzdem müssen OT-Systeme separat abgesichert werden. Die für IT-Sicherheit gängigen Sicherheitsmaßnahmen dürfen dabei den laufenden Betrieb nicht stören. Sicherheitspatches und Antiviren-Scanner sind im OT-Bereich nur eingeschränkt oder überhaupt nicht nutzbar. Den Schutz von OT müssen andere Maßnahmen sicherstellen. IDS eignen sich gut für diese Aufgabe. Sie greifen nicht in den Netzwerkverkehr ein und beeinflussen die Produktionsabläufe nicht.

2.2 Aufbau und Struktur eines ICS

OT bezieht sich auf Computersysteme, die zur Verwaltung von Industriebetrieben eingesetzt werden. ICS umfassen Systeme, die zur Überwachung und Steuerung industrieller Prozesse eingesetzt werden. Sie sind typischerweise geschäftskritische Anwendungen mit einer hohen Verfügbarkeitsanforderung. ICS werden oft über Supervisory Control and Data Acquisition (SCADA) verwaltet. SCADA zeigen dem Bediener den kontrollierten Prozess an und ermöglichen den Zugriff auf Steuerungsfunktionen. Abbildung 2.1 zeigt verschiedene Komponenten, die zu einer Anlage gehören. Die SCADA-Anzeigeeinheit zeigt über eine graphische Oberfläche den zu verwaltenden Prozess an. Eine Steuereinheit schließt die Fernbedienungseinheiten an das System an. Remote Terminal Unit (RTU) dienen zum Anschluss eines oder mehrerer Geräte wie Monitore oder Stellglieder an die Steuereinheit. Kommunikationsverbindungen können Ethernet für ein Produktionssystem, eine WAN-Verbindung über das Internet oder privater Funk eines verteilten Betriebes für Geräte in einem entfernten Gebiet sein. [12].

Mit der Purdue Enterprise Reference Architecture (PERA) existiert ein Referenzmodell für ICS, welches in den 1990er Jahren entwickelt wurde. Das Modell liegt beispielsweise der IEC 62443 zugrunde. [13]

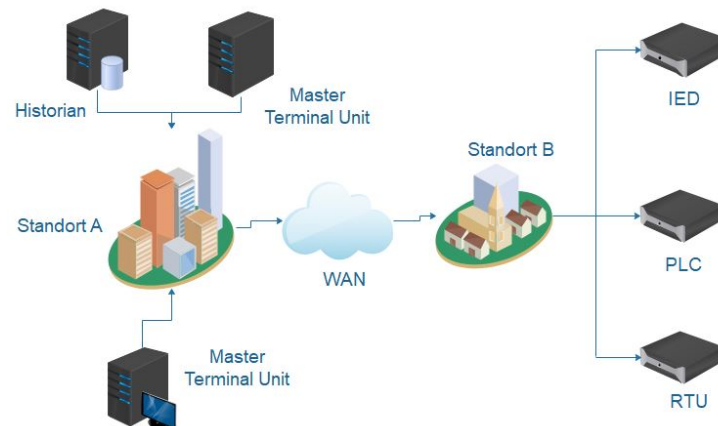


Abb. 2.1 Darstellung einer SCADA Konfiguration mit verschiedenen Komponenten

Abbildung 2.2 zeigt das Modell. PERA unterteilt ein ICS in die folgenden Ebenen:

Level 5 – Enterprise Zone: In der Unternehmenszone befinden sich ERP-Systeme, wie beispielsweise SAP und JD Edwards. Diese Zone erstreckt sich ggf. über mehrere Standorte, an denen ein Unternehmen tätig bzw. ansässig ist. [13]

Level 4 – Site Business Planning and Logistics: Die Ebene 4 repräsentiert die IT-Systeme, die an jedem Standort, jeder Anlage oder Einrichtung zur Steuerung des Betriebs der lokalen Anlage eingesetzt werden. Diese Ebene nimmt Aufträge der Ebene 5 entgegen und überwacht die Leistung auf niedrigeren Ebenen. [13]

ICS – Demilitarized Zone (DMZ): Die ICS-DMZ ist die Ebene für den Informationsaustausch zwischen IT und OT. In diesem Bereich befinden sich häufig Systeme, die für 'Patch'-Management oder 'Configuration'-/ 'Change'-Management zuständig sind. Häufig sind im ICS-DMZ Replikationsserver, Patch-Management-Server, Engineering-Arbeitsplätze und Configurations-/ Change-Managementsysteme vorhanden. Der Zweck der DMZ ist es, einen sicheren Austausch von IT-Informationen zu gewährleisten, ohne kritische Komponenten in unteren Schichten direkten Angriffen auszusetzen. [13]

Level 3 - Site Manufacturing and Operations Control: Die Ebenen 3 und darunter umfassen die Systeme auf der OT des Netzwerks. Ebene 3 enthält typischerweise SCADAs Überwachungsaspekt, Distributed Control System (DCS)-Ansicht mit kontrolliertem Zugriff oder die Kontrollräume mit Sicht- und Überwachungsfunktionen für den Rest des OT-Netzwerks. [13]

Level 2 – Area Supervisory Control: Auf der Ebene 2 befinden sich ICS-Komponenten wie z. B. speicherprogrammierbare Steuerung (SPS) und Variable Frequency Drives (VFD). Zu den wichtigsten Systemen auf dieser Ebene gehören jedoch die Human Machine Interfaces

(HMI). Innerhalb dieser Ebene ist eine lokale Sicht auf Live-Prozess-Events und automatisierte Steuerungen eines Prozesses zu sehen. [13]

Level 1 – Basic Control: Innerhalb der Ebene 1 befinden sich vor allem SPS, welche den In- und Output von Sensoren und Aktoren empfangen bzw. senden, sowie RTU wieder [14]. Hier wird der eigentliche Prozess gesteuert sowie die entsprechenden Daten für die Human Machine Interface (HMI)-Stationen zur Verfügung gestellt. Das Bedienpersonal kann mit Hilfe der HMI-Stationen Einfluss auf den Prozess nehmen. Darüber hinaus werden Alarme und Ereignisse an die höheren Level versendet, um die Planung und die Überwachung des industriellen Prozesses zu verwalten. [13]

Level 0 – Process: Diese Ebene beinhaltet Sensoren und Geräte zur Instrumentierung, welche direkt mit dem industriellen Prozess verbunden sind. Die Geräte werden von den Systemen im Level 1 gesteuert bzw. verwendet. [14]

Safety Layer: Hierbei handelt es sich um eine spezielle Zone. Sie beinhaltet Systeme, welche den industriellen Prozess auf Abweichungen untersuchen bzw. erkennen können und stellt fest, wenn ein Prozess außerhalb vorgegebener Spezifikationen läuft [14]. In dieser Zone sollen vor allem Zustände erkannt werden, welche einen Zusammenbruch des Gesamtsystems herbeiführen bzw. zu gefährlichen Situationen führen können. [13]

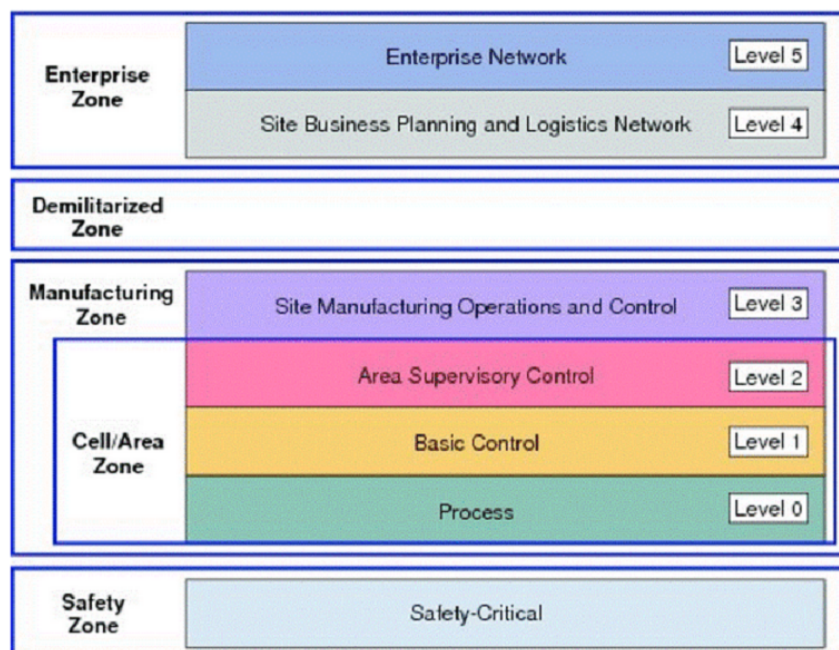


Abb. 2.2 Ebenen eines PERA Modells [15]

2.3 IT- und OT-Unterschiede

2.3.1 Überblick

Der Hauptunterschied zu IT besteht darin, dass sich OT auf die Überwachung und Kontrolle des industriellen Prozesses konzentriert. Das wirkt sich auf die Art und Weise aus, wie OT-Systeme im Gegensatz zu IT-Systemen betrieben und verwaltet werden. OT hat oft zusätzliche administrative, betriebliche oder technologische Einschränkungen, die ein anspruchsvolleres Sicherheitsumfeld erfordern. Abbildung 2.3 stellt die IT- und OT-Unterschiede in den operativen, technischen und verwaltungstechnischen Bereichen der Systeme dar.

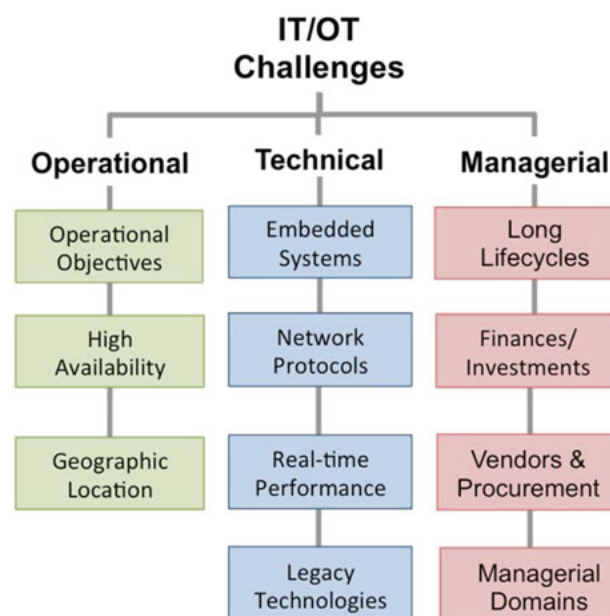


Abb. 2.3 OT stellt gegenüber IT unterschiedliche Herausforderungen [5]

2.3.2 Operative Unterschiede

Wichtige Ziele von OT sind die Minimierung der Sicherheits- und Umweltauswirkungen. Eine Fehlfunktion im ICS kann zur Bedrohung für die Betriebssicherheit, die Mitarbeiter und das lokale Umfeld führen. Anders als in IT-Strukturen haben Störungen das Potential, physikalische Systemkomponenten wie z. B. Kessel, Motoren, Transformatoren, Speicherbehälter und Generatoren zu beschädigen. Diese Komponenten fordern extrem hohe Investitionen und sind schwer

zu reparieren oder zu ersetzen. Das führt zu langen Systemausfall-Zeiten und hohen Kosten.

ICS müssen oft mit einer Verfügbarkeit arbeiten, die mindestens 99,99% der Jahreszeit beträgt, d. h. sie können maximal 50 Minuten eines Jahres außer Betrieb sein. Die Ausfallzeit enthält sowohl unvorhergesehene Ausfälle als auch die Ausfälle durch Systemwartungsfunktionen [5]. IT-Systeme müssen auch stets verfügbar sein, um eine uneingeschränkte Produktion zu ermöglichen. Kurzfristige Ausfälle lassen sich jedoch in der Regel kompensieren. Für IT-Systeme werden häufig Sicherheitsupdates veröffentlicht, die die Betriebssysteme und die Anwendungen auf dem neuesten Stand halten. Die Updates werden häufig automatisiert heruntergeladen, und sind nach dem nächsten Neustart verfügbar.

Die Anforderungen der hohen Verfügbarkeit für ICS haben negative Auswirkungen auf die erforderlichen Sicherheitsmechanismen für die Systeme. Ein Neustart nach einem Sicherheitsupdate kann die Verfügbarkeit beeinträchtigen.

Abbildung 2.4 zeigt eine Übersicht der operativen Anforderungen an IT und OT im direkten Vergleich. Bei OT steht die funktionale Sicherheit im Vordergrund, in klassischen IT-Umgebungen liegt der Augenmerk auf der IT-Sicherheit.

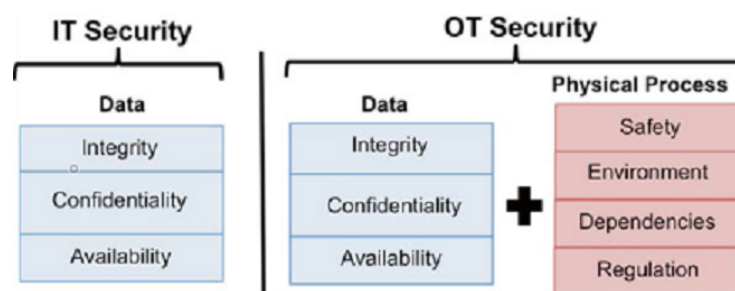


Abb. 2.4 Die operativen Anforderungen an OT sind gegenüber IT umfassender. [5]

Während IT-Umgebungen meistens lokal eingrenzbar sind, müssen ICS häufig an geografisch verteilten Standorten eingesetzt werden. Stromnetze, Ölpipelines und Transportsysteme erstrecken sich über sehr weite Distanzen. Diese geografische Verteilung führt zu Problemen bei der Implementierung physischer Schutzmaßnahmen.

2.3.3 Technologische Aspekte

Zusätzlich zu den operativen Unterschieden haben ICS besondere technische Anforderungen an Software- und Kommunikationsplattformen, die zur Unterstützung des Betriebes verwendet werden:

In ICS finden neben den traditionellen Netzwerkprotokollen aus der IT speziell für die Unterstützung von OT-Anforderungen entwickelte Protokolle Verwendung. Sowohl IT- als auch OT-Protokolle wurden oft ohne Sicherheitsfunktionen entwickelt. Viele IT-Protokolle bieten in ihren aktuellen Versionen zusätzliche Sicherheitsfunktionen. OT-Protokolle erhöhen in ihrer Konzeption die Zuverlässigkeit bei Kommunikationsfehlern durch den Einsatz von Cyclic Redundancy Checks (CRC). Diese Technik bietet aber keine zusätzliche Sicherheit bei Cyberangriffen. OT-Protokolle haben begonnen, Sicherheitsmerkmale zu implementieren. Tabelle 2.1 zeigt eine Übersicht von Netzwerkprotokollen und implizierten Sicherheitsfunktionen für IT und OT.

ICS-Systeme müssen zum Steuern eines physikalischen Prozesses oft in Echtzeit arbeiten. Ein-

Tab. 2.1 Netzwerktafel für IT und OT [5]

	IT	OT
Protokolle	HTTP, DNS, SSH, SMTP, SNMP, NTP	DNP3, Modbus, IEC 61850, IEC 608705, EtherCat, BACnet
Daten	großer Payload	analog, binäre Werte
Operationen	Stochastik	Deterministik
Sicherheit	in letzter Zeit entwickelt	noch im Entstehen

schränkungen wie Kommunikationslatenz und Jitter stellen Implementierungen von Sicherheitsmechanismen vor, die Herausforderungen und rechenintensive Operationen wie z. B. Public-Key-Kryptographie in verfügbarer Zeit durchzuführen.

Tabelle 2.2 beschreibt gebräuchliche Sicherheitsmechanismen, die in der OT beschränkt sind. Informationen über Sicherheitsmechanismen, die zum Schutz eines ICS erforderlich sind, gibt NIST 800-82 [16].

2.3.4 Verwaltungstechnische Unterschiede

Das Management der OT-Systeme unterscheidet sich von den IT-Systemen im Wesentlichen in den hohen Kapitalinvestitionen und der langen Lebensdauer. Ein ICS-System muss über einen langen Zeitraum in der Produktion bleiben, um die Kapitalkosten zu decken. Durch sich entwickelnde Cyber-Bedrohungen stellen diese langen Lebenszyklen viele Herausforderungen an die Sicherheit der Systeme.

Viele gängige kryptografische Mechanismen und Protokolle wie DES, MD5 oder SSLv2 bieten keine ausreichende Sicherheit mehr. Windows XP ist in ICS noch ein gängiges Betriebssystem, Microsoft stellt für die meisten Editionen keine Sicherheitspatches mehr zur Verfügung.

Berichtete Schwachstellen werden nicht immer gepatcht. Verfügbare Patches werden häufig nicht implementiert, weil befürchtet wird, dass die Systemverfügbarkeit beeinträchtigt wird [17].

Tab. 2.2 OT-Beschränkungen, die gemeinsame Informationssicherheitskontrollen behindern [5]

Kontrollkategorie	Allgemeine OT-Beschränkungen
Zugriffskontrolle	Viele OT-Plattformen bieten keine Möglichkeiten, Zugriffskontrollen für Benutzer durchzusetzen, und können daher keine detaillierte Kontrolle über den Zugriff auf Informationen oder Systemfunktionen bieten.
Überprüfung der Sicherheitsverletzungen	OT-Systeme sind oft nicht in der Lage, sicherheitsrelevante Ereignisse zu sammeln und zu speichern, um die Integrität des Systems zu überprüfen und mögliche IT-Sicherheitsverletzungen zu erkennen.
Konfigurations-Management	OT-Systeme bieten dem Eigentümer möglicherweise keine ausreichende Kontrolle über die Konfigurationen des Systems. Beispiele könnten sein, dass das Deaktivieren von nicht verwendeten Netzwerkservern oder fest programmierten Systemanmelde-Informationen nicht erlaubt ist.
Identität und Authentifizierung	OT-Systeme unterstützen möglicherweise keine starken Techniken zur Identifizierung und Authentifizierung von Benutzern. Sie können schwache Identifikatoren unterstützen, wie z. B. kurze Passwörter anstelle von Multifaktor-Authentifizierung. Darüber hinaus können sie nicht so konfiguriert werden, dass sie Authentifizierungsserver (z. B.. LDAP) oder Authentifizierungsprotokolle (z. B. RADIUS) verwenden.
System- und Kommunikationsschutz	OT-Systeme verfügen oft über begrenzte Mechanismen zum Schutz von Daten während der Kommunikation. Beispiele für häufige Einschränkungen sind das Fehlen starker Kryptographie-Algorithmen und -Protokolle und eine unzureichende Fähigkeit, Denial-of-Service-Angriffen standzuhalten.
System- und Informationsintegrität	OT-Systeme können die Integrität eines Systems oft nicht ausreichend durchsetzen oder überprüfen. Beispiele für unzureichende Fähigkeiten sind (i) Mechanismen zur Unterstützung von Systempatches, (ii) Unterstützung von Funktionen zur Erkennung von Malware und (iii) Mechanismen zur Überprüfung der Integrität des Systems und der Informationen.

2.4 Die Siemens S7-Kommunikation

Alle SIMATIC S7-Steuerungen haben integrierte S7-Kommunikationsdienste, mit denen ein Anwenderprogramm Daten lesen oder schreiben kann. Das S7-Protokoll *S7comm* wird unabhängig vom Busmedium wie z. B. Profibus oder Industrial Ethernet hauptsächlich für die Verbindung von SPS mit PC-Stationen verwendet [18]. Abbildung 2.5 dokumentiert die Anbindung des *S7comm* auf der Anwendungsebene im ISO-OSI-Referenzmodell.

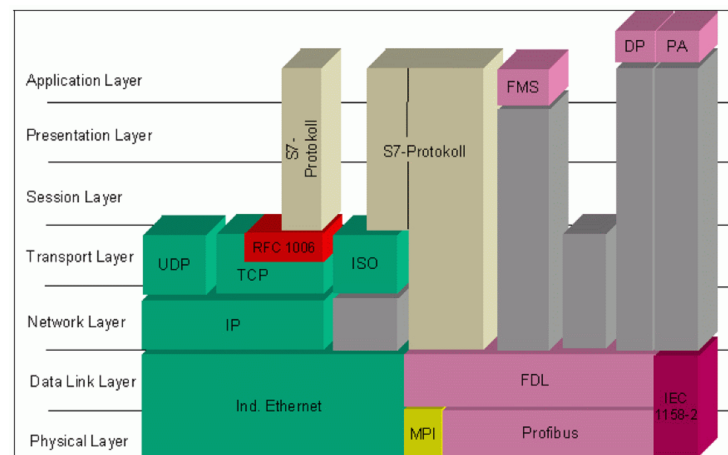


Abb. 2.5 Position des S7comm im ISO-OSI-Referenz-Modell. [18]

Meistens folgt die S7-Kommunikation dem Client/Server-Modell. Das S7-Protokoll ist funktions- und kommandoorientiert, d. h. eine Übertragung besteht aus einer Anfrage und einer entsprechenden Antwort. In den Anfragen können vom jeweiligen Client Daten vom Server 'Programmable Logic Controller (PLC)' abgefragt oder an ihn gesendet werden. Außerdem lassen sich bestimmte Befehle ausführen. Die PLC antwortet bzw. bestätigt. Gegebenenfalls sendet sie die angeforderten Daten. Die Anzahl der parallelen Übertragungen und die maximale Länge einer Protocol Data Unit (PDU) werden bei Verbindungsaufnahme ausgehandelt.

Dieser unidirektionale Dienst wird PUT-/ GET-Dienst genannt. Er dient zum Übertragen kleiner Datenmengen. Es gibt auch bidirektionale Dienste zum Übertragen mittlerer oder großer Datenmengen zwischen zwei Stationen nach dem Client/ Client-Modell (USEND/ URCV und BSEND/ BRCV). Dieser Modus wird auch als Partnermodus bezeichnet. [18]

2.4.1 Die S7 Protocol Data Unit

Die Implementierung des *S7comm* basiert auf dem blockorientierten ISO-Transportdienst. Das S7-Protokoll ist in die Protokolle TPKT und ISO-COTP eingebettet, wodurch die PDU über Transmission Control Protocol (TCP) übertragen werden kann. Abbildung 2.6 bildet die Einbettung des *S7comm* ab. Die ISO over TCP Kommunikation ist in RFC 1006 definiert, die

ISO-COTP in RFC 2126, das auf dem ISO 8073 Protokoll (RFC 905) basiert.

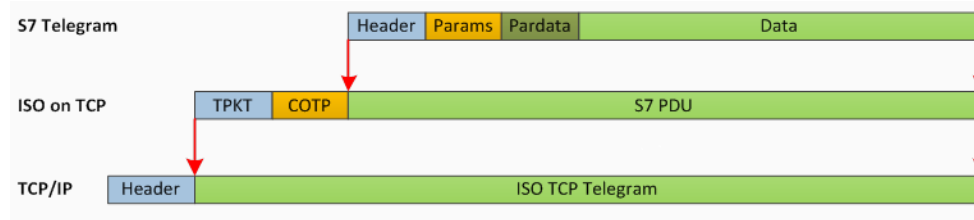


Abb. 2.6 S7comm - Protocol Encapsulation [19]

Die S7 PDU besteht aus drei Hauptteilen: Header und Parameter sind obligatorisch, der Datenteil ist optional.

Gemäß Abbildung 2.7 enthält der Header u. a. den Protokoll-Identifizierer (ID), den Nachrichtentyp, ein Feld 'Reserved', die PDU-Referenz, Parameterlänge und Datenlänge. Der weitere Aufbau der Nachricht ist vom Nachrichtentyp und dem Funktionscode abhängig [20].

Protokoll-ID	MSG-Type	Reserved	PDU Referenz	Parameter Length	Data Length	Error Class	Error Code
1 Bit	1 Bit	2 Bits	2 Bits	2 Bits	2 Bits	1 Bit	1 Bit

Abb. 2.7 Aufbau S7comm Header

2.4.2 Authentifizierung und Schutz

Es gibt drei Schutzmodi, die bei der Konfiguration der CPU eingestellt werden können: Kein Schutz, Schreibschutz und Lese-/Schreibschutz. Der Schreibschutz erfordert für bestimmte Schreibvorgänge und Konfigurationsänderungen eine Authentifizierung. Der Lese-/Schreibschutz erfordert zusätzlich für bestimmte Lesevorgänge eine Authentifizierung.

Auch bei aktiviertem Lese-/Schreibschutz sind bestimmte Operationen erlaubt, z. B. das Lesen von SZL-Listen oder das Lesen und Schreiben in den Marker-Bereich. Andere Operationen wie das Lesen oder Schreiben von Objekt/Funktion/Datenblöcken geben einen Berechtigungsfehler zurück.

Der CPU sind zwei Schutzstufenansätze zugeordnet, die Clients abfragen, wenn sie Lese- oder Schreibrechte benötigen: die zugewiesene Schutzstufe und die tatsächliche Schutzstufe. Die zugewiesene Schutzstufe ist diejenige, die bei der Konfiguration festgelegt wurde, während die tatsächliche die aktuelle Schutzstufe ist, die für die Kommunikationssitzung gilt. Ist eine Authentifizierung erforderlich, wird das Passwort in einer Benutzerdatennachricht an das Gerät gesendet.

2.4.3 Lese- und Schreib-Variable

Datenlese-Operationen werden mit dem Parameter-Function-Code 0x04 gekennzeichnet, Schreib-Operationen mit dem Functions-Code 0x05. Für die Durchführung der Operationen ist die Angabe des Speicherbereichs der Variablen, der Adresse und der Größe oder des Typs erforderlich. Die Implementierung der Funktionen 'Lesen' und 'Schreiben' unterstützt die Abfrage mehrerer variabler Lese- und Schreibvorgänge in einer einzigen Nachricht mit unterschiedlichen Adressierungsmodi. Die häufigsten sind der Standardadressiermodus und der Datenblock (DB)-Typ-Modus zum Ansprechen von DB-Bereichsvariablen.

Der Aufbau des Parameter-Header ist für jeden Adressiermodus gleich. Er besteht aus einem Funktionscode, dem Item Count und dem Anforderungselement.

Der Datenteil der S7-PDU variiert je nach Typ und Richtung der Nachricht. In Leseoperationen ist in der Anfrage kein Datenteil enthalten, in der Antwort wird in der Regel das Lesen mit Angabe des Datentyps und der Länge des gelesenen Wertes bestätigt. Bei einer Schreiboperation wird in der Anfrage der zu schreibende Wert mit Angabe des Datentyps und seiner Länge im Datenteil übermittelt. Die Antwort enthält in der Regel die Bestätigung, dass der Wert geschrieben wurde.

2.5 Einführung in Intrusion Detection Systeme

Eine Intrusion ist ein unerwünschter oder nicht autorisierter Vorgang. Unter dem Begriff Intrusion Detection versteht man die Überwachung von Systemen und/oder Netzwerken mit dem Ziel, unerwünschte und/oder nicht autorisierte Vorgänge zu erkennen, zu protokollieren und zu melden. Ein IDS überwacht ein System und ein Netzwerk und analysiert, ob das beobachtete Verhalten des Systems/Netzwerkes vom erwarteten Verhalten abweicht [21].

Es existieren prinzipiell zwei verschiedene Arten von IDS, die sich hinsichtlich ihrer Anforderungen und Arbeitsweise unterscheiden: Host Intrusion Detection System (HIDS), deren Aufgabe in der Überwachung eines einzelnen Rechners besteht und Network Intrusion Detection System (NIDS), deren Aufgabe in der Überwachung des Netzwerkverkehrs besteht. Hybride IDS sind eine Mischform der beiden IDS-Arten, sie enthalten Netzwerk- und Hostbasierte Komponenten. [21]

2.5.1 Hostbasiertes Intrusion Detection System

Ein HIDS wird eingesetzt, um Einbrüche lokal auf dem Host erkennen zu können. Typische Operationen sind Prozess-Analyse, Protokolldatei-Analyse und die Überwachung der Netzwerkkommunikation des Hosts. HIDS sind selten, wenn überhaupt, in der Lage, auf SPSen oder Feldgeräten zu laufen und werden nur der Vollständigkeit halber erwähnt. [5]

2.5.2 Netzwerkbasiertes Intrusion Detection System

Netzwerküberwachung schließt Technologien und Methoden ein, die einen Angriff zwar nicht verhindern, aber zu dessen Entdeckung beitragen können. An die Entdeckung eines Angriffs schließen sich reaktive Maßnahmen an. Sie haben die Ziele, entstandenen Schaden zu minimieren und beheben, künftige Angriffe ähnlicher Art zu verhindern und gegebenenfalls einen Täter zu ermitteln.

Einer möglichst frühzeitigen Erkennung von Angriffen kommt eine entscheidende Bedeutung zu: Je früher ein Angriff erkannt wird, desto schneller kann der Angriff analysiert und die ausgenutzte Schwachstelle geschlossen werden. Mit eingeleiteten Gegenmaßnahmen wird die Fortführung eines Angriffs unterbunden. [21]

NIDS arbeiten typischerweise auf drei grundlegende Arten: durch signaturbasierte Analyse, Protokollerkennung und/oder Verhaltensanalyse. Signaturbasierte Analyse vergleicht Header, Paketinhalte und Paketströme mit Datenbanken bekannter, bösartiger Bytefolgen, um verdächtigen Datenverkehr zu identifizieren. Mit Hilfe dieser Datenbanken kann auf bestimmten TCP-Ports und IP-Adressen nach verdächtigen Byte-Sequenzen im Netzwerk gefiltert werden [22].

Protokollbasierte Analyse: Normaler Datenverkehr entspricht typischerweise den RFC-Spezifikationen. Das bedeutet, dass der Aufbau eines Ethernet-Frames oder eines TCP-Paketes im Normalfall immer gleich ist. Protokollfähige NIDS reassemblieren Fragmente (Layer 3) und Streams (Layer 4) und rekonstruieren sogar ganze Protokolle (Layer 7). So kann beispielsweise geprüft werden, ob ein Frame die richtige Länge hat oder alle Prüfsummen korrekt sind [22].

In der Verhaltensanalyse wird eine 'Baseline' des normalen Netzwerkbetriebs aufgebaut, mit der zukünftiges Verhalten des Netzwerkes verglichen werden kann. Das System lernt, was normal ist, um später Abweichungen des Netzwerkverkehrs zu erkennen. Warnmeldungen können z. B. ausgelöst werden, wenn das Datenvolumen pro Transaktion, das von dem Webserver ausgeht, zwei Standardabweichungen größer ist als der erwartete Mittelwert. Selbst wenn es keine Hinweise auf Protokollverletzungen gibt oder der Inhalt der Web-Transaktionen mit keinem bekannten schlechten Verhalten übereinstimmt, könnte dies dennoch ein sehr wichtiges Ereignis sein [22].

2.5.3 Netzwerk-basierte Intrusion Detection Systeme in ICS

ICS haben unterschiedliche Arten von Netzwerken: primäre und sekundäre Busse. Der primäre Bus verbindet ein Standard-IT-Gerätenetz mit speicherprogrammierbaren Steuerungen SPS und ist IP-basiert vernetzt oder seriell. Die Workstations werden in der Regel unter Windows- oder Linux-Systemen betrieben. Hinter den speicherprogrammierbaren Steuerungen befinden sich Sekundärbusse, die beliebig vielfältige Feldgeräte steuern. Die Sekundärbusse eines komplexen ICS können eine Vielzahl von verschiedenen herstellerspezifischen Kommunikationsmethoden verwenden. Die sekundären Buskomponenten sind möglicherweise nicht global adressierbar, wie es IP-Netzwerkkomponenten sind. Insofern ist der Primärbus das Netzwerk, auf dem der Einsatz eines NIDS sinnvoll ist. [23]

Signaturbasierte Intrusion Detection-Methoden in ICS

Signaturbasierte Intrusion Detection-Methoden zielen in der Regel darauf ab, ein bekanntes Bitmuster im Netzwerkverkehr für dokumentierten Schadcode zu finden. Signaturbasierte Verfahren zum Schutz des IDS-Protokollverkehrs sind allein in der Regel nicht ausreichend. Aufgrund des inhärenten Mangels an Authentifizierung und Verschlüsselung in den ICS-Protokollen ist ein unbefugter Netzwerkzugriff auf Geräte ohne Verwendung von Malware möglich. Methoden, die ausschließlich auf Signatur-Erkennung basieren, können durch Täuschung über die eigene Identität oder durch Verwendung unbekannter Malware umgangen werden.

ICS-Sicherheitsforscher favorisieren bei der Entwicklung effizienter IDSs für ICSs Kombinationen aus signaturbasierten Intrusion Detection-Methoden und nicht signaturbasierten Intrusion Detection-Methoden. [23]

Nicht signaturbasierte Intrusion Detection-Methoden in ICSs

Ein Anomalie-basiertes IDS überwacht Parameter, die den Zustand eines Netzwerkes und den Datenverkehr als normal oder nicht normal klassifizieren. Parameter können die Anzahl der Pakete pro Zeiteinheit zwischen den einzelnen Teilnehmern sein, die Länge der Pakete oder die Geschwindigkeit und die Richtung, in der sie aufeinander folgen. Komplexere Modelle berechnen Wahrscheinlichkeiten, mit denen ein Netz von einem Zustand in einen anderen übergehen wird und erkennen Abweichungen anhand dieser Wahrscheinlichkeiten.

Ein Anomalie-basiertes IDS muss den normalen Zustand inklusive der zulässigen Parameter und Übergangswahrscheinlichkeiten kennen. Dann kann es prinzipiell jede Abweichung von der Norm erkennen und Angriffe, die bisher völlig unbekannt sind, identifizieren. Eine allein auf Anomalien basierende Intrusion Detection hat ebenfalls Mängel. Die False-Positiv-Rate kann hoch sein. Darüber hinaus kann ein entsprechend vorbereiteter Angriff diese IDS täuschen [23].

Prozessorientierte Einbruchserkennung

In einem ICS werden geeignete Prozesswerte zur Erzeugung einer gewünschten Leistung festgelegt. Der Zweck von ICS-Steuerungsmeldungen besteht darin, die Synchronisation der SPS-Register zu unterstützen und eine lokale, HMI-seitige Kopie dieser Register bereitzustellen, um die Steuerung der Anlagenprozesse zu ermöglichen. Ein ICS-Netzwerk kann Registerwerte ändern und lesen. Registerwerte wirken sich direkt auf die Prozessparameter und damit auf den Prozess aus. Da die ICS-Sicherheit dem Schutz der Prozessvariablen und nicht des Netzwerkverkehrs selbst dient, sind prozessorientierte Designs wie die Definition kritischer Prozessvariablen Grenzen für Intrusion Detection interessant. [23]

Das Konzept des Frameworks in dieser Arbeit basiert auf einer Kombination aus signaturbasierten und nicht signaturbasierten IDS-Methoden. Die Anomalie-Erkennung basiert auf Analyse der Protokolle und auf Basis statistischer Daten. Weicht das verwendete Protokoll vom definierten Protokollstandard ab, kann eine Anomalie erkannt werden. Bei der Erkennung auf Basis statistischer Daten werden Kennwerte festgelegt. Im vorgestellten Konzept ist ein definierter Kennwert,

dass einige Operationen nur von bestimmten Clients durchgeführt werden dürfen. Beispielsweise darf nur eine Engineering Workstation (EWS) einen Firmware-Update durchführen. Anhand der statistischen Werte kann festgestellt werden, ob eine signifikante Verhaltensänderung zum Normalbetrieb vorhanden ist.

Suricata

Suricata¹ ist ein NIDS. Es wird durch Open Information Security Foundation (OISF)² entwickelt und betreut. Die Software steht unter einer freien GPLv2³ Lizenz. Die Suricata-Engine ist in der Lage, Intrusion Detection (IDS), Inline-Intrusion Prevention (IPS), Network Security Monitoring (NSM) und Offline-PCAP-Verarbeitung in Echtzeit durchzuführen. Suricata überprüft den Netzwerkverkehr mit einer umfangreichen Regel- und Signatursprache. Mit Lua-Skripting⁴ ist es möglich, Analysen durchzuführen, die innerhalb der Regelsyntax von Suricata nicht möglich sind.

Für die Analysen wurde Suricata auf einem Linux-System installiert. In der Konfigurationsdatei wurde der Pfad zu den erstellten Regeln eingetragen. Suricata wurde im PCAP Offline-Modus verwendet. Im Offline-Modus liest das IDS aus dem angegebenen Pfad die PCAPS und wendet die erstellten Regeln an. Die Ausgabe enthält die Angabe, welche Regeln wie oft Treffer erzielt haben. Detaillierte Informationen können in der Dokumentation zu Suricata nachgelesen werden [24].

2.6 Stand der Forschung

In vergangenen Forschungen wurden Systeme für den Einsatz von IDS entwickelt, die auf verschiedenen Ansätzen basieren. Einige konzentrierten sich auf die Art der Erkennung, andere Arbeiten zielten sich auf den Austausch zwischen den verschiedenen Komponenten in ICS ab. Es wurde in den Arbeiten häufig mit unterschiedlichen Angriffsvektoren gearbeitet. Trotz der unterschiedlichen Ansätze lieferten die Ergebnisse oft gute Erkennungsraten.

Yüksel et al. untersuchten den Einsatz von IDS zur Schwachstellenanalyse in ICS [9]. Die auf Missbrauch basierende Erkennung, die bekannte Angriffe sucht, wies niedrige Falsch-Positiv-Raten auf. Die Anomalie-basierte Erkennung, die ein Modell des normalen Verhaltens eines Systems erstellt, zeigte beim Einsatz in ICS hingegen eine hohe Falsch-Positiv-Rate. Dabei erstellte Warnungen gaben in der Regel keine Einblicke in zugrunde liegende Ursachen, weil sie wenig Informationen lieferten. Entwickelt wurde ein Whitebox-Framework für die drei Protokolle *Modbus-RTU*, *Modbus-TCP* und *S7comm*. Implementiert wurden elementare und zusammengesetzte Charaktereigenschaften wie z. B. Art des Befehls, Adresse, Rückgabe- oder

¹<https://suricata-ids.org/>

²<https://oisf.net/>

³<http://www.gnu.org/licenses/old-licenses/gpl-2.0.html>

⁴<https://www.lua.org/>

Änderungswerte. Im Ergebnis ermöglichte das konfigurierte Whitebox-Framework Einstellungen zur Reduzierung der Fehllarme und eine Erhöhung der Erkennungsraten.

Zu einem ähnlichen Ergebnis wie Yüksel et al. kamen Genge et al. bereits im Jahr 2014 [25]. Ihre Untersuchung bestand aus einem systematischen Ansatz SPEAR bestehend aus einer Tool-Suite zur Modellierung der Topologie von ICS und dem IDS Snort. SPEAR stützt sich auf das Vorhersageverhalten von Verbindungen zwischen verschiedenen ICS-Hosts zur Erkennung eines abnormen Paketaustausches.

Ghaeini et al. entwickelte ein Framework für ein hierarchisches Intrusion Detection System (HAMIDS) [26]. Das HAMIDS-Framework erkennt Anomalien in den Ebenen 0 und 1 in industriellen Steuergeräten. Ebene 1 enthält Prozesskontrollgeräte, die Sensorwerte auslesen, gewünschte Informationen berechnen und die Daten an ein Ziel senden. Ebene 0 enthält Sensoren und Instrumentierungselemente, die von Ebene-1-Geräten gesteuert werden. Darüber hinaus sammelt das Framework cyberphysikalische Prozessdaten, die weiteren Analysen im Intrusion Detection dienen. Das HAMIDS-Framework wurde bei der S3-Veranstaltung *SWaT Security Showdown* im Jahr 2016 an der Singapore University of Technology and Design in einem realen industriellen Kontrollsystem getestet. Es übertraf andere vorgeschlagene akademische IDS in Bezug auf das Erkennen von ICS-Bedrohungen.

Jardine et al. untersuchten den Einsatz einer Kombination bestehend aus einem passiven Überwachungsansatz und einem selektiven aktiven Monitoring auf der Grundlage von ICS-spezifischen Angriffsvektoren (SENAMI) [27]. Die Angriffsvektoren bestanden aus Aufklärungsscans, DoS-Angriffen, Hochladen von Logikcode und Manipulieren von Werten. Implementiert wurde das IDS für den Einsatz von S7-Geräten der Firma Siemens. SENAMI erreichte eine Erkennungsrate von 99% beim Einsatz von passiven Monitoring gegen passive Bedrohungen. Das aktive Monitoring konnte aktive Bedrohungen systematisch erkennen. Es zeigte gelegentlich Falsch-Positiv-Meldungen, war aber effektiver als passives Monitoring gegen aktive Bedrohungen. Erreicht wurden in der Untersuchung Erkennungsraten von 81 – 93%.

Markman et al. entwickelten zur SCADA-Anomalieerkennung einen zyklischen deterministischen endlichen Automaten (engl. Deterministic Finite Automaton (DFA)) zu einem Burst-DFA-Modell weiter [28]. Festgestellt wurde, dass die Kommunikation in ICS-Netzwerken auf allen Kanälen in Paketbursts erfolgte. Das Modell behandelte den Verkehr auf jedem Kanal als eine Reihe von Bursts und ordnete jeden Burst dem DFA zu. Analysiert wurde mit dem Modell *Modbus*-Verkehr aus einer Wasseraufbereitungsanlage. Das Burst-DFA-Modell ordnete zwischen 95% und 99% der Pakete im Datenkorpus zu.

Im darauffolgenden Jahr entwickelten Markman et al. ein k-Phasenmodell [29]. Das Modell berücksichtige verschiedene Verkehrsphasen, indem für jede erkannte Verkehrsphase ein eigener DFA erstellt wurde. Das neue Modell erreichte eine niedrigere Falschalarmrate. Mit einer burstbasierten Abtastung wurden eine Genauigkeit von 98,9%–99,99% erkannter Pakete erreicht.

Kleinmann et al. entwickelten ein modellbasiertes IDS für S7-Netzwerke [30]. Der Ansatz

basierte auf der zentralen Beobachtung, dass der S7-Verkehr zu und von einer bestimmten SPS in der Regel periodisch ist. Jeder HMI-SPS-Kanal kann mit seinem eigenen, einzigartigen DFA modelliert werden. Das resultierende DFA-basierte IDS kann Anomalien wie z. B. eine Nachricht, die in der normalen Reihenfolge aus ihrer Position erscheint, oder eine Nachricht, die sich auf ein einzelnes unerwartetes Bit bezieht, stark beeinflussen. Trotz der hohen Empfindlichkeit wies das System eine sehr geringe False-Positive-Rate auf. Über 99,82% des Datenverkehrs wurden als normal identifiziert.

Bestehende Arbeiten konzentrieren sich auf die Anwendung von IDS und der Erkennung von Angriffen anhand der Angriffsvektoren auf der Basis verschiedener Modelle.

Diese Arbeit konzentriert sich auf die Darstellung von Grundlagen zur Erstellung von IDS-Regeln. Der Schwerpunkt dieser Arbeit liegt auf der Auswahl allgemeingültiger Merkmale und Zusammensetzungen für viele Protokolle. Das Konzept für das Framework soll als Basis für die Implementierung verschiedener Protokolle dienen und die Erstellung von Regeln für IDS erleichtern. Damit kann der Anwender seine Regeln selbst schnell erzeugen, was zu einer erhöhten Akzeptanz für die Anwendung von IDS im Netzwerk sorgen könnte.

Berücksichtigt werden im Konzept der Paketaustausch zwischen Komponenten, allgemeine protokollbasierte Merkmale und verhaltensbasierte Merkmale.

3 Konzept des Frameworks

Wissenschaftler haben die Entwicklung von IDS als eine effektive Methode zur Erkennung von Angriffen auf ICS [31] identifiziert. Dies liegt an der Art des IDS, welche das Systemverhalten analysiert und minimale Auswirkungen auf den Systembetrieb hat. Für einzelne Cyber-Angriffe sind IDS-Regeln erstellt worden. Nach dem Hatman-Angriff wurden z. B. die *RAPSN SETS* für Schneider Electric's TriStation entwickelt [32].

Um den Aufwand für die Regelerstellung für einzelne Anlagen oder Prozesse zu reduzieren, ist in dieser Forschungsarbeit ein Konzept für ein modulares Framework entwickelt worden, dass performant eine prozessspezifische Regelerstellung für verschiedene ICS-Protokolle in verschiedenen IDS-Systemen ermöglicht.

Basis der Arbeit ist das Identifizieren von kritischen Befehlen und Funktionen. Innerhalb dieser Befehle und Funktionen müssen die Segmente und Elemente erkannt werden, die für Störungen in ICS verantwortlich sein können. Die Entwicklung eines Konzeptes für ein generelles Framework erfordert eine Reihe von Anforderungen an diese Erkennung. Das vorgestellte Konzept und die Anforderungen beschränken sich nicht nur auf Ethernet-basierte Netzwerke, sondern sind auch modular auf serielle und drahtlose Netzwerke aufgebaut. Das Ziel-ICS-Protokoll im Framework-Konzept ist austauschbar und ermöglicht auf diese Weise die Regel-Generierung zu standardisierten oder proprietären ICS-Protokollen.

Die Entwicklung des Framework-Konzeptes ist wie folgend aufgebaut: Abschnitt 3.1 beschreibt einen Überblick zu kritischen Befehlen. In Abschnitt 3.2 werden die Anforderungen an ein Framework zur Generierung von IDS-Regeln vorgestellt. Abschnitt 3.3 zeigt die Unterteilung in Regelgruppen.

3.1 Kritische Befehle

Welche Operationen Störungen auslösen können, ist ein wichtiges Kriterium für die Entwicklung des Konzeptes. Zur Identifizierung der kritischen Befehle wurden die Befehlssätze aus Publikationen zu verbreiteten ICS-Protokolle wie *S7comm* [33], *Modbus* [34], *Ethercat* [35] und *Profibus* [36] sowie der umfassende Befehlssatz aus dem Triton-Code [37] selektiert. Diese Auswahl wurde mit grundsätzlichen Funktionen aus der IT dahingehend abgeglichen, ob sie Merkmale aufweisen, die Einfluss auf das Programm oder die laufenden Prozesse nehmen können. Diese verbleibenden Befehle wurden als kritisch eingestuft, wenn der Einfluss geeignet war, Störungen auslösen zu können. Um die kritischen Befehle klassifizieren zu können, erfolgte

eine Subsumption in Gruppen mit übergeordneten Begriffen.

Als kritisch wurden alle Operationen betrachtet, die schreibenden Charakter haben und auf ein System zugreifen. Schreibende Zugriffe erzeugen Eingaben von Informationen in einen Speicher, die temporäre oder dauerhafte Veränderungen an einem System hervorrufen. Die relevanten Informationseinheiten sind nicht notwendigerweise einzelne Bits, es können auch Bitfolgen wie Worte oder Bytes sein. Tabelle 3.1 zeigt eine Übersicht der identifizierten kritischen Befehle und Funktionen, die im Folgendem näher beschrieben werden. Weder die Aufzählung der Regel-Gruppen noch die einzelnen Befehle in den Gruppen erheben einen Anspruch auf Vollständigkeit. Sie können jederzeit erweitert werden.

Tab. 3.1 Kritische Befehle und Funktionen aus verschiedenen Protokollen wurden in Gruppen zusammengefasst:

Nr.	Kritische Gruppe	Kurzbeschreibung
0	00_Generic	Generische Regeln und Reserved For Future Use (RFU)
1	01_Download	Daten, die auf einer anderen Netzkomponente heruntergeladen werden
2	02_Upload	Daten, die von einer anderen Netzkomponente hochgeladen werden
3	03_Update	Aktualisierung von Software
4	04_Upgrade	Aktualisierung von Funktionen
5	05_Date-Time	Synchronisieren oder Anlegen von Zeitstempeln
6	06_Variable	Anlegen von Variablen
7	07_Value	Schreiben von Werten
8	08_Address	Festlegen und Ändern von Speicheradressen
9	09_Authentication	Authentifizierung durch Zugangsdaten
10	10_Mode	Zustandsbezeichnungen und -änderungen
11	11_Command	Erteilen von Kommandos
12	12_Communication	Kommunikationsaufbau und -abbau
13	13_Functions	Übertragung von Datenblöcken
14	14_Memory	Speicherzugriffe
15	15_Reset	Zurücksetzen von Daten

Die Gruppe **00_Generic** ist reserviert für generische Regeln und weitere spätere Verwendung (engl. RFU). Die Generischen Regeln filtern jeden Netzverkehr, der über TCP von bzw. an die PLC oder den Port 102 gesendet wird.

01_Download erfasst alle Operationen, die beim Herunterladen von Daten auftreten können (Tabelle 3.2). Es kann sich dabei um eine oder mehrere Dateien handeln, die von einer bereitstel-

lenden Einheit zum Download angeboten bzw. bereitgestellt werden. Inbegriffen sind das aktive Anfordern, der Empfang und die dauerhafte oder temporäre Speicherung dieser Dateien auf dem eigenen Gerät.

Tab. 3.2 Die Gruppe 01_Download erfasst alle Operationen, die beim Herunterladen von Daten auftreten können.

Gruppe	Befehle
01_Download	start download all stop download all cancel download all start download change stop download change cancel download change

In der Gruppe **02_Upload** sind alle Operationen enthalten, die beim Hochladen von Daten auftreten können (Tabelle 3.3). Das kann den Upload von Programmen, Funktionen, oder Konfigurationen betreffen. Upload bezeichnet den Vorgang des Schreibens von Programmlogik von der PLC zum Client. Es handelt sich um schreibende Zugriffe, die Veränderungen hervorrufen können.

Tab. 3.3 Die Gruppe 02_Upload enthält Befehle, die beim Hochladen von Daten auftreten können.

Gruppe	Befehle
02_Upload	start program upload stop program upload cancel program upload start function upload stop function upload cancel function upload start configuration upload stop configuration upload cancel configuration upload

Die Gruppe **03_Update** beinhaltet Befehle zur Aktualisierung von Programmen und Treibern (Tabelle 3.4). Ein Update sorgt in der Regel für kleinere Verbesserungen zur Vorgängerversion oder beseitigt Fehler innerhalb eines bestimmten Softwarestands. Dies wird in der IT als Service Release, Patch oder Hotfix bezeichnet. Updates können Probleme verursachen, wenn die aktualisierte Version nicht mit der Hardware oder anderer Software auf den Geräten kompatibel ist. Möglicherweise handelt es sich um eine manipulierte Update-Version, die vorsätzlich Störungen

verursacht.

Tab. 3.4 Die Gruppe 03_Update umfasst Operationen, die im Zusammenhang mit Softwareaktualisierung auftreten.

Gruppe	Befehle
03_Update	start firmware update stop firmware update cancel firmware update start program update stop program update cancel program update start function update stop function update cancel function update start configuration update stop configuration update cancel configuration update

Die Gruppe **04_Upgrade** enthält Befehle zum Upgraden von Systemen (Tabelle 3.5). In Abgrenzung zum Update erweitert ein Upgrade eine Software um neue Funktionen. Ein Software-Update steht für eine neue Version einer Software und wird in der Regel durch eine Änderung der Versionsnummer gekennzeichnet. Ein Software-Upgrade kann als eine neue Variante bezeichnet werden, die auf der ursprünglichen Fassung basiert und eine technische Neuerung beinhaltet. In der klassischen IT kann ein Upgrade mit anderen Befehlen durchgeführt werden als ein Update. Beispielsweise enthält Linux ein Programm Update und ein Programm Upgrade, während Windows zwischen Sicherheits-Updates und Funktions-Updates unterscheidet. Für OT ist die Verfahrensweise protokollabhängig. Ein Upgrade verändert das System genauso wie ein Update und kann auch die gleichen Probleme hervorrufen.

Tab. 3.5 Die Gruppe 04_Upgrade erfasst Operationen, die zur Funktionserweiterung von Software gehören.

Gruppe	Befehle
04_Upgrade	start firmware upgrade stop firmware upgrade cancel firmware upgrade start program upgrade stop program upgrade cancel program upgrade

Die Gruppe **05_Date-Time** beinhaltet sowohl Operationen zum Synchronisieren von Datum und Zeit als auch das Erstellen und Anpassen von Zeitstempel und Datumsangaben, sofern sie Bestandteil von Feldbussen sind (Tabelle 3.6). Eine Vielzahl an Prozessen, aber auch administrative Tätigkeiten, beruhen in ICS-Umgebungen auf einer genauen und abgestimmten Zeit. Darunter fällt z. B. die Nachvollziehbarkeit verteilter Protokolldaten oder die Beigabe von Zusatzstoffen in der Produktion zum richtigen Zeitpunkt. Daher kann die Modifizierung der Zeit die Qualität des Produkts oder den Produktionsprozess beeinträchtigen [8].

Tab. 3.6 Die Gruppe 05_Date-Time umfasst Veränderungen von Zeit- und Datumsangaben.

Gruppe	Befehle
05_Date-Time	synchronize time (Feldbusse) set time adjust time modify time set date modify date

Eine Variable ist eine belegbare Leerstelle für unbekannte, unbestimmte oder veränderliche Formulierungen. Sie muss noch nicht mit Werten gefüllt werden, wenn sie angelegt wird. Zu den kritischen Operationen in der Gruppe **06_Variable** zählen das Erstellen von neuen Variablen, das Verändern und das Löschen bestehender Variablen (Tabelle 3.7). Die genannten Operationen können in Produktionsprozessen zu ungewollten Veränderungen führen, wenn bestehende Variablen wie z. B. Speicheradressen verändert werden.

Tab. 3.7 Die Gruppe 06_Variable listet Befehle zum Setzen, Verändern und Löschen von Variablen.

Gruppe	Befehle
06_Variable	set variable modify variable delete variable

Ein Wert kann eine Zahl, ein Buchstabe, eine Zeichenkette oder die Zuweisung zu einer Variablen sein. Die Gruppe **07_Value** beinhaltet das Setzen von gültigen und ungültigen Werten, das Verändern und Löschen von gültigen Werten (Tabelle 3.8). Diese Operationen beinhalten auch *force*- und *write*-Befehle. Hierbei kann es sich um einzelne Bits oder Worte und Bytes handeln. Durch das Verändern und Löschen bestehender Werte oder Hinzufügen von Werten können eventuell Produktionsabläufe beeinträchtigt werden. Wenn z. B. das Wasser-Chlor-Verhältnis in einem öffentlichen Schwimmbad durch veränderte Werte in ein Missverhältnis gerät, können Folgen für

die Gesundheit der Besucher auftreten.

Tab. 3.8 Die Gruppe 07_Value beinhaltet Befehle zum Setzen, Ändern und Löschen von Werten.

Gruppe	Befehle
07_Value	set valid value modify valid value delete valid value set invalid value set multiple point values modify multiple point values delete multiple point values force value

In der Kategorie **08_Address** werden das Schreiben, Verändern und Löschen von einzelnen Speicheradressen und begrenzten Speicheradressen-Bereichen subsumiert (Tabelle 3.9). Speicheradressen dienen der eindeutigen Bezeichnung von Speicherzellen. Sie werden beim Speicherzugriff verwendet, um den genauen Ort zu benennen, auf den ein Zugriff erfolgt. Werden die Speicheradressen umbenannt, z. B. in Adressen anderer Speicherbereiche, wird möglicherweise auf falsche Werte zugegriffen. Das kann zu verändertem Verhalten laufender Prozesse oder Programme führen.

Tab. 3.9 Die Gruppe 08_Address enthält Befehle für Operationen zu Speicheradressen.

Gruppe	Befehle
08_Address	set I/O-Adress modify I/O-Adress delete I/O-Adress set I/O-Adress limit modify I/O-Adress limit delete I/O-Adress limit

Die Gruppe **09_Authentication** umfasst versuchte, erfolgreiche und fehlgeschlagene Authentifizierungen (Tabelle 3.10). Authentifizierung wird als Bezeugung der Echtheit interpretiert. Fehlgeschlagene Versuche könnten einen Passwort-Angriff indizieren. Die Anmeldung von einem nicht autorisiertem Host mit korrektem Passwort kann auf einen Passwortdiebstahl hinweisen.

Die Kategorie **10_Mode** enthält Operationen, welche den Zustand einer Anlage bezeichnen (Tabelle 3.11). Starten, Stoppen, Pausieren und Beenden sind typische Befehle, um in einen neuen Zustand zu gelangen. Betreffen können die Operationen dieser Gruppe Prozesse, Programme

Tab. 3.10 Die Gruppe 09_Authentication enthält Befehle zur Authentifizierung.

Gruppe	Befehle
09_Authentication	attempt authentication success authentication failed authentication

oder einzelne Komponenten wie die PLC. Werden die PLC oder einzelne Prozesse unerwartet gestoppt, nimmt das Einfluss auf den Produktionsablauf. In hochverfügbaren Produktionsumgebungen kann dies zu hohen wirtschaftlichen Schäden führen.

Tab. 3.11 Die Gruppe 10_Mode enthält Befehle, die den Zustand einer Anlage bezeichnen.

Gruppe	Befehle
10_Mode	start process stop process pause process start program stop program pause program cancel program start PLC stop PLC pause PLC

Gruppe **11_Command** umfasst Handlungen zu fertigen Befehlen (Tabelle 3.12). Das Ausführen von unbekannten Befehlen oder das Modifizieren, Löschen und Zurückweisen von bekannten Befehlen veranlassen Maschinen zu einem ungewöhnlichen Verhalten. Aber auch das Wiederholen regulärer Befehle in ungewöhnlicher Anzahl oder Zeitpunkten kann Störungen in Produktionsabläufen verursachen.

12_Communication: Kommunikationsaufbau und -abbau zu und von der PLC liefert Erkenntnisse zu autorisierten und nicht-autorisierten Clients und zu den Übertragungsmodalitäten (Tabelle 3.13). Für die Nachvollziehbarkeit von Protokolldateien oder die forensische Analyse sind solche Erkenntnisse von Bedeutung.

Die Gruppe **13_Functions** umfasst Handlungen zum Ausführen von Funktionen und Scripten, die benutzerdefiniert oder protokollspezifisch sein können (Tabelle 3.14). In Abgrenzung zur Gruppe 11_Command werden in dieser Gruppe kleine Unterprogramme, die durchaus aus mehreren Befehlen bestehen können, subsumiert. Solche Funktionen sind in der Regel individuell für das

Tab. 3.12 Gruppe 11_Command enthält Operationen zu ganzen Befehlen.

Gruppe	Befehle
11_Command	command reject set command modify command delete command unknown command

Tab. 3.13 Die Gruppe 12_Communication enthält Befehle zum Kommunikationsaufbau zwischen PLC und Client.

Gruppe	Befehle
12_Communication	start communication end communication

jeweilige Protokoll erstellt. Scripte sind in einer Scriptsprache geschrieben und könne Anwendung auf verschiedene Protokolle finden. Bei der Nutzung ausführbarer Funktionen und Scripte besteht immer die Gefahr einer unautorisierten Manipulation.

Tab. 3.14 Die Gruppe 13_Functions enthält Befehle zu ganzen Funktionen.

Gruppe	Befehle
13_Functions	Block Function (OB, DB, SDB, FC, SFC, FB, SFB) CPU Function (SZL) Programmer Command benutzerdefinierte Funktionen benutzerdefinierte Scripte

Die Gruppe **14_Memory** enthält Befehle zum Kopieren, Abbilden und Zurücksetzen von Speichern Tabelle 3.15. Ordnungsgemäß genutzt werden die Operationen zu Analyse-Zwecken und für das Anlegen von Backup-Dateien. Der Hauptspeicher kann von einem Angreifer kopiert und zurückgesetzt oder kopiert werden, um die Informationen aus dem Speicher für sein weiteres Vorgehen zu verwenden.

In der Gruppe **15_Reset** sind Operationen enthalten, mit denen Daten zurückgesetzt werden können (Tabelle 3.16). Möglicherweise ist eine Konfiguration fehlerhaft und es lässt sich aus der bestehenden Konfiguration heraus der Fehler nicht reparieren. Dann kann nach einem

Tab. 3.15 Gruppe 14_Memory enthält Befehle zum Kopieren, Abbilden und Zurücksetzen von Speichern.

Gruppe	Befehle
14_Memory	memory reset mam to mom (copy memory) memory dump

Factory Reset das System neu konfiguriert werden. Ein Angreifer könnte einen Auslieferungszustand herstellen wollen, um dann eine eigene Konfiguration zu starten.

Tab. 3.16 Gruppe 15_Reset enthält Operationen zum Zurücksetzen von Konfigurationen.

Gruppe	Befehle
15_Reset	reset factory settings

3.2 Anforderungen an das Framework

In ICS erfolgt die Kommunikation zwischen den zugehörigen Netzkomponenten. Ein Client fragt an, die PLC antwortet, indem sie die Anfrage bestätigt oder ausführt. In einem regulären Netzwerkverkehr führen autorisierte Clients Requests mit regulären Operationen aus. Um nicht regulären Netzwerkverkehr identifizieren zu können, muss das Framework verschiedenen Anforderungen gerecht werden. Es muss sichergestellt werden, dass Anfragen von nicht autorisierten Clients und Antworten von PLCs an nicht autorisierte Clients erkannt werden und entsprechend einen Alarm auslösen. Gewährleistet werden muss auch, dass Clients nur korrekte Anfragen stellen und PLCs nur legitime Anfragen ausführen. Anfragen sind zutreffend, wenn sie dem jeweiligen Protokoll entsprechend zulässige Parameter und korrekte Werte und Wertebereiche verwenden. Aus diesen Vorgaben lassen sich folgende Anforderungen ableiten:

- A1: Kritische Operationen von autorisierten Hosts werden protokolliert.
- A2: Unkorrekte Operationen lösen einen Alarm aus.
- A3: Autorisierte Clients werden von nicht autorisierten Clients unterschieden.
- A4: Nicht autorisierte Clients lösen einen Alarm aus, wenn sie mit der PLC kommunizieren.
- A5: Gültige Anfragen werden erkannt.
- A6: Gültige Antworten werden erkannt.

A7: Pakete mit korrekten Parametern und Werten werden von Paketen mit unkorrekten Parametern und Werten unterschieden.

A8: Exploits werden erkannt und lösen einen Alarm aus.

3.3 Regelerstellung für IDS-Gruppen

Für das Erkennen von kritischen Operationen müssen für die jeweiligen Protokolle Regelsätze erstellt werden. Die Regeln dienen der Überprüfung des Netzwerkverkehrs in ICS, insbesondere zwischen der PLC und HMI bzw. EWS. Diese Regeln werden in Gruppen unterteilt: einfache Regeln, Protokollregeln, Threshold-Regeln, Exploit-Regeln, Adressbereich-Regeln und Allgemeine Regeln. Jede einzelne Regel lässt sich anhand ihres Security Identifier (SID) einer Regelgruppe zuordnen und identifizieren.

Die SID besteht aus 10 Ziffern. Die ersten 4 Ziffern stellen einen allgemeinen Identifier dar, der für dieses Projekt *Additional Layer of Detection for ICS (ALDI)*¹ festgelegt wurde. Die nächsten beiden Ziffern (RR) weisen das Protokoll aus. Zwei weitere Ziffern (GG) ordnen die Regel einer Gruppe aus den kritischen Befehlen, die in Abschnitt 3.1 vorgestellt wurden, zu. Die letzten beiden Ziffern (NN) nummerieren die Regeln innerhalb der Gruppe.

Die Regeltypen der einfachen-, Protokoll- und Threshold-Regeln bleiben in der SID unberücksichtigt, da für die Auswertung die kritischen Gruppen GG von Interesse sind. Die Regeltypen hingegen sind für die Strukturierung des Framework-Konzeptes von Bedeutung.

Abbildung 3.1 zeigt ein Beispiel für eine SID: Für das Protokoll *S7comm* wird eine SID für die erste Regel der Gruppe 07_Value erstellt.

Ziffern	10, 9, 8, 7	6, 5	4, 3	2, 1
generisch	2534	RR	GG	NN
Beschreibung	ALDI- Identifier	Protokoll- Identifier	Gruppen- Identifier	Regel-Nummer in der Gruppe
Beispiel (S7comm)	2534	32	07	01

Abb. 3.1 Erstellung eines Security Identifier (SID) für die erste Regel der Gruppe 07_Value

3.3.1 Einfache Regeln

In den einfachen Regeln wird geprüft, ob gültige Pakete zum Kommunikationsaufbau und zur Identifizierung von bzw. zu einem nicht autorisierten Client gesendet werden. Handelt es sich um gültige Pakete zwischen autorisierten beteiligten Komponenten, werden die Pakete lediglich

¹Das Projekt wurde erstmals beim Industrial Control System Security Symposium (28. und 29. Mai 2019, Charlotetown, PEI, Kanada) öffentlich vorgestellt [38]. Die Folien des Vortrags stehen unter [39] zum Abruf bereit.

protokolliert, andernfalls wird ein Alarm ausgelöst. Alarmer sollten zeitnah von einem Security Information and Event Management (SIEM) überprüft und bearbeitet werden [32].

3.3.2 Protokollregeln

Die Protokollregeln überprüfen, ob gültige Pakete von oder zu einem autorisierten Host mit möglichen Auswirkungen auf wichtige Funktionen gesendet werden. Bei diesen Paketen handelt es sich um Telegramme, die kritische Befehle übertragen. Das Normalverhalten ist dabei durch Protokollspezifikationen definiert. Es wird geprüft, ob der Kommunikationsverkehr den zugrundeliegenden Protokollspezifikationen genügt. Netzverkehr zwischen dem gültigen Host und der PLC werden protokolliert. Pakete von bzw. zu einem für die kritischen Befehle nicht autorisierten Host lösen einen Alarm aus. Alarmer sollten hier auch an ein SIEM zur Überprüfung übersendet werden [32].

3.3.3 Threshold Regeln

Gültige Pakete, die in ungewöhnlich hoher Anzahl oder Frequenz gesendet werden, lösen einen Alarm aus. Die entsprechenden Grenzwerte hierfür müssen an die reguläre Nutzung der Funktionen angepasst werden, da sie in der Regel betriebs- oder anlagenspezifisch sind. Ein ausgelöster Alarm nach den Threshold-Regeln spricht nicht zwingend für einen Angriff. Der Vorfall sollte jedoch untersucht werden. Wenn die Alarmer vermehrt auftreten und die Analyse jeweils zu dem Ergebnis kommt, dass es sich um eine Falsch-Positiv-Meldung handelt, sind möglicherweise die festgelegten Grenzwerte nicht korrekt auf den normalen Netzwerkverkehr abgestimmt. Dann sollte eine Änderung der Grenzwerte sukzessive vorgenommen werden.

3.3.4 Exploit Regeln

Befehle, die zur Steuerung bekannter oder erkannter Angriffe ausgeführt werden oder bekannte Schwachstellen ausnutzen, lösen Alarm aus.

Exploits nutzen Schwachstellen aus, die bei der Entwicklung eines Programmes entstanden sind. In der Regel wollen Angreifer in Systeme eindringen, um sich Zugang zu Ressourcen zu verschaffen oder die Systeme zu beeinträchtigen. Beispielsweise unterscheiden viele Systeme nicht zwischen Programmcode und Nutzdaten. Bei einem Pufferüberlauf wird der Code in einen nicht dafür vorgesehenen Speicherbereich geschrieben, wodurch die Ausführung einer Anwendung manipuliert oder eigener Code ausgeführt werden kann.

Die Netzwerkpakete sollten auf jeden Fall zur späteren Analyse oder Beweissicherung mitgeschnitten werden. Regeln dieser Gruppe werden anhand bereits bekannt gewordener Angriffe erstellt. In diese Gruppe können auch bereits veröffentlichte IDS Regeln zu bekannten Angriffen implementiert werden.

3.3.5 Adressbereich Regeln

Gültige Pakete mit falschem Datentyp oder Werten außerhalb des zugewiesenen Adressbereichs lösen einen Alarm aus. Bestimmte Datentypen dürfen ihre Werte nur in einem bestimmten Adressbereich schreiben. Vorfälle, die einen Alarm ausgelöst haben, sollten analysiert und untersucht werden.

3.3.6 Allgemeine Regeln

Diese Regeln warnen bei jedem Paket, das innerhalb des Netzwerks über Transportprotokolle wie TCP oder User Datagram Protocol (UDP) von bzw. an eine PLC oder den protokollspezifischen Port gesendet wird. Diese Regeln dienen Testzwecken von z. B. IDS-Konfigurationen und sind nicht für den Einsatz in einer Produktiv-Umgebung vorgesehen.

4 Anwendung des Framework-Konzeptes

Die Funktionalität des Framework-Konzeptes wurde gezeigt, indem es für das Protokoll *S7comm* angewendet wurde. Dafür wurden in einer Testanlage erzeugte Testdaten¹ aus dem Jahr 2016 analysiert. Die Ergebnisse der Analyse waren die Grundlage für die Regelerstellung nach dem entwickelten Konzept.

Regeln wurden für verschiedene *S7comm*-spezifische Funktionscodes, die nach den in Kapitel 3 beschriebenen Kriterien als kritisch betrachtet werden, entwickelt. Die Regeln wurden für Anfrage-Nachrichten vom Client zum Server und für Antwort-Nachrichten vom Server zum Client erstellt.

Nach Erstellung der Regeln wurden die verschiedenen kritischen Szenarien in einer Testanlage durchgeführt. Verschiedene PCAPS² präsentieren sowohl reguläre Tätigkeiten im ICS als auch fehlerhaftes Verhalten. Als reguläre Tätigkeiten wurden Operationen eingestuft, die im Rahmen von Wartungs- oder Aktualisierung-Vorgängen auftreten können, wie Block Functions oder Updates. Fehlerhaftes Verhalten war z. B. ein außerplanmäßiges Starten und Stoppen der PLC.

Die erstellten Regeln wurden in eine virtuelle Suricata-Maschine implementiert und über das Auswerte-Tool 'Wireshark'³ an den erzeugten PCAPS² zu den kritischen Szenarien getestet, die Ergebnisse wurden ausgewertet.

Zunächst werden in diesem Kapitel die Prozesse der Testanlage beschrieben, an der Netzwerkmitschnitte erzeugt wurden. Anschließend wird die Auswertung der PCAPS dargelegt. Im letzten Abschnitt wird die Erstellung der Regeln für *S7comm* aufgezeigt.

4.1 Prozesse der Testanlage

Die Testanlage simuliert eine Branium-Raffinerie⁴. Der Prozessbeschreibung der Testanlage nach ist Branium ein wichtiger Bestandteil in der Funkelektronik. Das fiktive Metall wird in kleinen Mengen im Antennen- und Funksubsystem von Mobiltelefonen, GPS TX/RX und

¹https://github.com/qut-infosec/2017QUT_S7comm

²Diese PCAPS befinden sich im Freigabeprozess.

³Wireshark wurde unter der GNU General Public License als freie quelloffene Software entwickelt.

⁴Beim Branium handelt es sich um ein fiktives Metall

tragbaren Computern wie Laptops und Tablets verarbeitet.

Es wird aus schwarzen Roherz in einem dreistufigen Prozess gewonnen. Die erste Stufe ist das Förderbandsystem. Dort wird das schwarze Roherz von den gelegentlichen weißen Erzverunreinigungen getrennt. Sobald eine ausreichende Menge an schwarzem Roherz geliefert wurde, wird das Erz gewaschen und mit chemischen Toxiclosin behandelt. Die Endstufe ist eine katalytische Reaktion in einem Rohrreaktor. Bei diesem Verfahren wird das Branium vom Roherz getrennt. [40]

Die Testanlage simuliert diese drei Prozesse. Die Details der drei Prozesse können über die Bedieneroberfläche eines HMI durch Anklicken der Schaltflächen 'Conveyor', 'Tank' und 'Reactor' aufgerufen werden. Jeder Prozess muss in einer bestimmten Reihenfolge abgeschlossen werden. Die simulierte Raffinerie kann nur einen Auftrag pro Instanz verarbeiten. Der Tankbildschirm ist erst zugänglich, wenn der Sortierprozess abgeschlossen ist. Der Prozess des Reaktortanks kann erst starten, wenn der Prozess des Wasch-Tank abgeschlossen ist. Dem Bediener wird anhand eines Ampelsystems vermittelt, in welchem Stadium sich der Gesamtprozess gerade befindet. Das rote Licht neben dem Reaktorprozess in Abbildung 4.1 zeigt an, dass der Prozess noch nicht abgeschlossen ist. Ein gelbes Licht zeigt an, dass der Prozess gerade läuft. Der Tankprozess in Abbildung 4.1 stellt dar, dass der Prozess gerade läuft und noch nicht abgeschlossen ist (rotes und gelbes Licht). Das grüne Licht neben dem Förderband zeigt an, dass dieser Prozess abgeschlossen ist. [40]

Mit dem Reset-Button kann das System zurückgesetzt werden und ein neuer Prozess gestartet werden. Wenn die Subprozesse bei Betätigung des Schalters noch nicht abgeschlossen waren, geht die aktuelle Chargeveredelung verloren. Mit dem Notausschalter wird ein Prozess sofort gestoppt. Dabei geht die aktuelle Charge ebenfalls verloren.

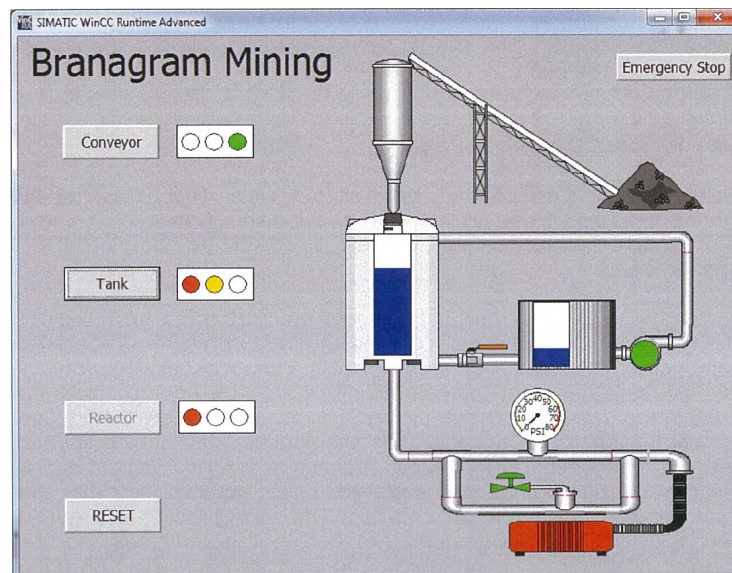


Abb. 4.1 Prozessübersichtsanzeige einer simulierten Branium-Raffinerie

Wie in dem Netzplan in Abbildung 4.2 zu sehen ist, besteht die Testanlage aus einem Netzwerk, in dem über einem Switch mehrere Clients, eine Master-PLC und drei Slave-PLCs miteinander verbunden sind.

Als Client traten zwei verschiedene HMIs, zwei EWS und ein Angreifer auf. Der Capture-Point befand sich vor dem Switch auf dem Primärbus.

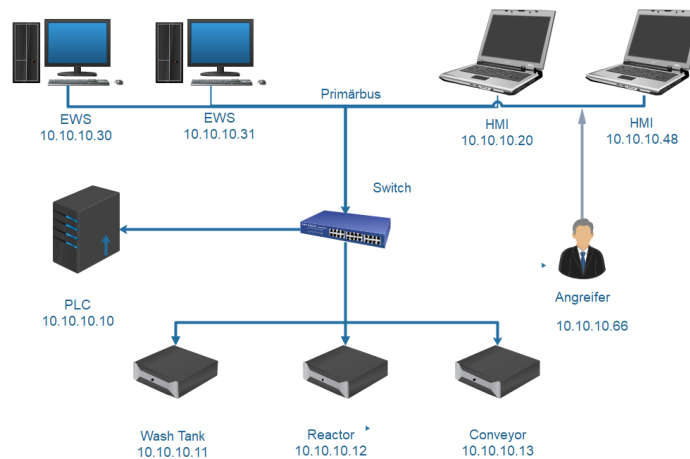


Abb. 4.2 Netzplan mit den Komponenten der Testanlage

4.2 Analyse der PCAPS

Um das Framework-Konzept protokollspezifisch anwenden zu können, wurden die erstellten Testdaten zum *S7comm* analysiert.

In Wireshark³ ist ein Plugin implementiert, dass die Inhalte der *S7comm*-Pakete dekodiert und für den Betrachter lesbar darstellt. Die Netzwerkdaten werden von Wireshark in verschiedenen Ansichten dargestellt: in einer Paketliste, in Paketdetails und in einer hexadezimalen Paketanzeige. Auf diese Weise kann für jeden Wert die Zugehörigkeit zum jeweiligen Paket und sein Offset nachvollzogen werden. Die PCAPS lassen sich nach jedem beliebigen Wert filtern.

In der Analyse wurde zunächst der gesamte Inhalt einiger Pakete betrachtet, um den Aufbau des *S7comm* kennenzulernen. Anschließend wurde nach Werten gesucht, die identifizierende Eigenschaften in Bezug auf das Protokoll selbst und auf kritische Befehle aufweisen. Diese Werte sind von Bedeutung, weil die später zu erstellenden Regeln darauf Treffer erzielen. Typischerweise zählen dazu Werte wie die Protokoll-ID oder Funktions-IDs, aber auch Wertebereiche.

Für die Analyse und auch die spätere Regelerstellung für das Protokoll *S7comm* standen zwei Testdatensätze in Form von Netzwerkmitschnitten zur Verfügung: '20161219132813_control_set' und '20161215163606_s7_process_attacks'. Die verwendeten Netzwerkschnitte sind auf dem

Github der QUT Infosec Gruppe¹ verfügbar.

Der Netzwerkmitschnitt '20161219132813_control_set' - im Weiteren auch 'Control-PCAP' oder 'Controlset' genannt - bildete das Verhalten der Testanlage im regulären Betrieb ab, ohne dass Wartungs- oder Aktualisierungs-Operationen erfolgten. Das HMI sendete Befehle an die Master-PLC. Die Master-PLC bestätigt die Anfragen und führt die Befehle aus.

Der Netzwerkmitschnitt '20161215163606_s7_process_attacks' - im Weiteren auch 'Angriffs-PCAP' genannt - zeigte, wie ein Angreifer sich Eintritt in das Netzwerk verschaffte und Befehle an die PLC sendete. Die Master-PLC bestätigte auch diese Befehle und führte sie aus, weil für den Angriff reguläre Operationen verwendet wurden. Die Anomalie des Verhaltens bestand in der Häufigkeit, in der die gleichen Operationen hintereinander ausgeführt wurden.

4.2.1 S7comm Kommunikation

Im Nachrichtenpaar der *S7comm* Kommunikation erfolgt die Anfrage (Job Request) prinzipiell immer von einem Client. Gerichtet werden die Anfragen an die Master-PLC. Der jeweilige Client kann für diese Kommunikation eine beliebigen Port nutzen. Die Master-PLC empfängt *S7comm*-Pakete immer auf dem Port 102. Entsprechend wird die Antwort (Ack Data) immer von der Master-PLC vom Port 102 an den Port des jeweiligen Clients gesendet.

Im Control-PCAP und in den PCAPS zu regulären oder fehlerhaften Tätigkeiten erfolgte eine Kommunikation wie in Abbildung 4.3 ausschließlich zwischen der Master-PLC (IP-Endung .10) und den Clients (IP-Endung .20). Tabelle 4.1 zeigt eine IP-Adresszuordnung zu den einzelnen Hosts.

Tab. 4.1 Den einzelnen Netzwerkkomponenten sind IP-Adressen zugeordnet.

Host	IP-Adresse
Master-PLC	10.10.10.10
Slave-PLC 'Wash Tank'	10.10.10.11
Slave-PLC 'Reactor'	10.10.10.12
Slave-PLC 'Conveyor'	10.10.10.13
HMI	10.10.10.20
AHMI	10.10.10.48
EWS 1	10.10.10.30
EWS 2	10.10.10.31
Angreifer	10.10.10.66

Im Angriffs-PCAP wurde zunächst das Netzwerk vom Angreifer erkundet. Nach erfolgreichem Eindringen in das Netzwerk wurde neben dem regulären Netzwerk-Traffic mehrfach über das

10.10.10.20	10.10.10.10	COTP	61 DT TPDU (0) [COTP fragment, 0 bytes]
10.10.10.20	10.10.10.10	S7COMM	241 ROSCTR:[Job] Function:[Read Var]
10.10.10.48	10.10.10.10	COTP	61 DT TPDU (0) [COTP fragment, 0 bytes]
10.10.10.10	10.10.10.20	S7COMM	198 ROSCTR:[Ack_Data] Function:[Read Var]
10.10.10.20	10.10.10.10	COTP	61 DT TPDU (0) [COTP fragment, 0 bytes]
10.10.10.10	10.10.10.20	S7COMM	158 ROSCTR:[Ack_Data] Function:[Read Var]
10.10.10.20	10.10.10.10	COTP	61 DT TPDU (0) [COTP fragment, 0 bytes]
10.10.10.48	10.10.10.10	S7COMM	97 ROSCTR:[Job] Function:[Read Var]
10.10.10.10	10.10.10.48	S7COMM	93 ROSCTR:[Ack_Data] Function:[Read Var]

Abb. 4.3 Darstellung einer Kommunikation zwischen HMI und Master-PLC in Wireshark

Interface 10.10.10.66 eine Kommunikation zur Master-PLC aufgebaut und Traffic erzeugt. In Abbildung 4.4 ist der Kommunikationsaufbau des Angreifers zu sehen.

10.10.10.66	10.10.10.10	TCP	74 40896 → 102 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM=1
Siemens_17:...	Broadcast	ARP	60 Who has 10.10.10.66? Tell 10.10.10.10
Broadcom_cb...	Siemens_17:...	ARP	42 10.10.10.66 is at 00:10:18:cb:8b:ec
Broadcom_cb...	Siemens_17:...	ARP	60 10.10.10.66 is at 00:10:18:cb:8c:13
Dell_e1:df:...	Siemens_17:...	ARP	60 10.10.10.66 is at f8:b1:56:e1:df:91
10.10.10.10	10.10.10.66	TCP	60 102 → 40896 [SYN, ACK] Seq=0 Ack=1 Win=4096 Len=0 MSS=1460
10.10.10.66	10.10.10.10	TCP	60 40896 → 102 [ACK] Seq=1 Ack=1 Win=29200 Len=0
10.10.10.66	10.10.10.10	COTP	76 CR TPDU src-ref: 0x0001 dst-ref: 0x0000
10.10.10.10	10.10.10.66	COTP	76 CC TPDU src-ref: 0x0007 dst-ref: 0x0001
10.10.10.66	10.10.10.10	TCP	60 40896 → 102 [ACK] Seq=23 Ack=23 Win=29200 Len=0
10.10.10.66	10.10.10.10	S7COMM	79 ROSCTR:[Job] Function:[Setup communication]
10.10.10.10	10.10.10.66	S7COMM	81 ROSCTR:[Ack_Data] Function:[Setup communication]

Abb. 4.4 Darstellung einer Kommunikation zwischen Angreifer und Master-PLC in Wireshark

4.2.2 S7comm Protokollstruktur

Für die Protokollspezifische Umsetzung des Frameworkes wurden der Zweck und die interne Struktur der Nachrichten 'Job Request' und 'Ack Data' untersucht. Die Nachrichtentypen sind sich sehr ähnlich und treten in der Regel zusammen auf.

Abbildung 4.5 zeigt ein *S7comm*-Paket. Es besteht aus den drei Hauptteilen Protokoll-Header, Parameter und Daten. Die ersten beiden Teile sind obligatorisch, der Teil Daten ist optional. Dieser kommt zum Tragen, wenn es Daten zu übertragen gibt.

Protokoll Header

Ein *S7comm*-Paket wird an der Protokoll-ID 0x32 (rot) erkannt. Der Nachrichtentyp mit dem Wert '1' (0x01 = Job Request) (blau) sagt aus, dass dieses Paket zu einer Anfrage von einem Client an die Master-PLC gehört. In Abbildung 4.5 sind diese Header-Informationen farblich markiert. Antwort-Pakete von der Master-PLC an den Client tragen an dieser Stelle den Wert '3' (0x03 = Ack Data). Eine Antwort ohne Datenübertragung wäre mit dem Wert '2' (0x02 = Ack)

```

  ▾ S7 Communication
    ▾ Header: (Job)
      Protocol Id: 0x32
      ROSCTR: Job (1)
      Redundancy Identification (Reserved): 0x0000
      Protocol Data Unit Reference: 1
      Parameter length: 14
      Data length: 5
    ▾ Parameter: (Write Var)
      Function: Write Var (0x05)
      Item count: 1
      ▾ Item [1]: (M 101.5 BIT 1)
        Variable specification: 0x12
        Length of following address specification: 10
        Syntax Id: S7ANY (0x10)
        Transport size: BIT (1)
        Length: 1
        DB number: 0
        Area: Flags (M) (0x83)
        ▸ Address: 0x00032d
      ▾ Data
        ▾ Item [1]: (Reserved)
          Return code: Reserved (0x00)
          Transport size: BIT (0x03)
          Length: 1
          Data: 01

```

Abb. 4.5 Darstellung eines S7comm-Paketes mit Header, Parameter und Daten in Wireshark

deklariert. Abbildung 4.5 zeigt die Header-Informationen in Wireshark.

Blockübertragungen, Schutzstufen-Abfragen durch Lesen von SZL-Listen und CPU-Funktionen werden dem Nachrichtentyp 'Userdata' mit dem Wert '7' (= 0x07) zugeordnet. Die übrigen Felder im Header sind für die Implementierung des Framework nicht von Bedeutung und werden an dieser Stelle nicht weiter betrachtet.

Parameter mit Funktionen und Adressierung

In den Nachrichtentypen 'Job' und 'Ack_Data' besteht der Parameter-Teil aus einem Funktionen-Teil und dem Adressen-Teil. Im Nachrichtentyp 'Userdata' besteht der Parameterteil ausschließlich aus einem Funktionen-Teil, der eine Funktionsgruppe und eine Unterfunktion umfasst. Außerdem enthält dieser Nachrichtentyp immer einen Daten-Teil.

Der 'Function'-Wert im Parameter-Header der Pakete beschreibt, um welchen Befehl es sich handelt. In den ausgewerteten PCAPS waren dies vorwiegend die Lese- oder Schreibvariablen mit

den Werte 0x04 und 0x05. Das sind Funktionen, die im regulären Betrieb häufig benutzt werden. Abbildung 4.5 zeigt, wie ein Schreibbefehl mit dem 'Function'-Wert 0x05 (orange) im Parameter-Header gekennzeichnet wird. Weitere geläufige Funktionswerte werden in Tabelle 4.2 dargestellt.

Tab. 4.2 zeigt die Parameter-Funktionen des S7comm

Wert	Befehl
0x00	CPU-Services
0xF0	Setup Communication
0x04	Read Variable
0x05	Write Variable
0x1A	Request Download
0x1B	Download Block
0x1C	Download Ended
0x1D	Start Upload
0x1E	Upload
0x1F	End Upload
0x28	PLC Control
0x29	PLC Stop

Operationen werden durch Angabe des Speicherbereichs der Variablen, ihrer Adresse und ihrer Größe oder ihres Typs adressiert. Das untersuchte Protokoll *S7comm* unterstützt die Abfrage mehrerer variabler Lese- und Schreibvorgänge in einer einzigen Nachricht mit unterschiedlichen Adressierungsmodi. In den PCAPS wurden die beiden Hauptmodi 'Standard' und 'DB-Type' benutzt. Die Regel ist der Standard-Adressiermodus und wird zur Abfrage beliebiger Variablen verwendet. Für jede adressierte Variable werden alle drei Parameter (Bereich, Adresse, Typ) angegeben.

DB-Type ist ein spezieller Modus, der entwickelt wurde, um Speicherbereich Datenblock-Bereichsvariablen (DB-Bereichsvariablen) anzusprechen. Er ist kompakter als die Adressierung eines beliebigen Typs. [41]

In den untersuchten PCAPS wurden im Standard-Adressiermodus zwei Speicherbereiche angewandt, die häufig Verwendung finden: Marker und Datenblock. Im Marker [M] befinden sich beliebige Markervariablen oder Markerregister. DB sind der häufigste Ort zur Datenspeicherung. Sie werden von verschiedenen Funktionen des Protokolls verwendet. Datenblöcke sind nummeriert. DB-Nummern sind Teil der Adresse.

Es gibt weitere Speicherbereiche, die an dieser Stelle nicht weiter betrachtet werden, da sie in den Testdaten nicht verwendet wurden.

Der Variablentyp bestimmt die Länge und die Art der Interpretation einer Variable. In den ausgewerteten PCAPS wurden ausschließlich die Variablentypen BIT, BYTE und REAL verwendet.

BIT ist ein einzelnes Bit, BYTE hat eine Länge von 8 Bit und REAL steht für eine vier Byte breite IEEE-Fließkommazahl.

Abbildung 4.6 zeigt eine Adressierung aus dem Controlset. Die Variable befindet sich im Speicherbereich Marker [M] (rot). Der Variablentyp ist BIT (grün). Hier wird auf das 1. Bit des 101. Bytes (orange) zugegriffen.

Die Adresse 0x000329 (blau) ist dem Automatic Mode des Waschtanks zugeordnet. In Tabelle 4.3 sind ein Teil der zulässigen Wertebereiche für den Waschtank gelistet.

```

v Item [1]: (M 101.1 BIT 1)
  Variable specification: 0x12
  Length of following address specification: 10
  Syntax Id: S7ANY (0x10)
  Transport size: BIT (1)
  Length: 1
  DB number: 0
  Area: Flags (M) (0x83)
v Address: 0x000329
  .... .000 0000 0011 0010 1... = Byte Address: 101
  .... .... .... .... .... .001 = Bit Address: 1

```

Abb. 4.6 Darstellung der Adressierung einer Variablen im Speicherbereich Merker [M] in Wireshark

Tab. 4.3 Zulässige Adressen für den Waschtank

Adresse	Bedeutung	Minimum (off)	Maximum (on)
0x000328	Manual Mode	00	01
0x000329	Automatic Mode	00	01
0x00032a	Off	00	01

Eine Adressierung im DB ist in Abbildung 4.7 zu sehen. Hier wird ein REAL gelesen, der im 0. Bit des 8. Bytes (blau) im 3. Datenblock (rot) beginnt.

Die DB-Typ-Adressierung wird für den Zugriff auf DB-Variablen verwendet und bietet eine Alternative, um mehrere Variablen in einem einzigen Element in einem kompakteren Format anzusprechen. Im Parameterheader sind die wesentlichen Informationen zum Parametertyp, der Funktionsgruppe und Unterfunktion enthalten. Die Information zum Blocktyp steht im Daten-Teil. Abbildung 4.8 bildet eine DB-Adressierung ab. Der Parameter Head 0x000112 (rot) ist der Spezifikationstyp. Parametertyp ist ein Request (blau). Die Anfrage richtet sich an die Funktionsgruppe 'Block Functions' (blau). Angefragt wird die Unterfunktion 'List blocks of typ' (gelb), in diesem Fall ein Functionblock (FB) (grün).

```

  Parameter: (Write Var)
    Function: Write Var (0x05)
    Item count: 2
  Item [1]: DB 3 DBX 8.0 REAL 2)
    Variable specification: 0x12
    Length of following address specification: 10
    Syntax Id: S7ANY (0x10)
    Transport size: REAL (8)
    Length: 2
    DB number: 3
    Area: Data blocks (DB) (0x84)
  Address: 0x000040
    .... .000 0000 0000 0100 0... = Byte Address: 8
    .... .... .... .... .... .000 = Bit Address: 0

```

Abb. 4.7 Darstellung der Adressierung einer Variablen im Speicherbereich Datenblock [DB] 3 in Wireshark

```

  Parameter: (Request) ->(Block functions) ->(List blocks of type)
    Parameter head: 0x000112
    Parameter length: 4
    Method (Request/Response): Undef (0x00)
    0100 .... = Type: Request (4)
    .... 0011 = Function group: Block functions (3)
    Subfunction: List blocks of type (2)
    Sequence number: 0
  Data: (FB)
    Return code: Success (0xff)
    Transport size: OCTET STRING (0x09)
    Length: 2
    Block type: 0E (FB)

```

Abb. 4.8 Darstellung der DB-Type Adressierung eines FB in Wireshark.

4.3 Regelerstellung S7comm

Um IDS-Regeln für das Protokoll *S7comm* erstellen zu können, war es zunächst erforderlich, die kritischen Befehle zu identifizieren, die für dieses Protokoll von Bedeutung sind.

Hierfür wurden die gleichen Kriterien berücksichtigt, wie im Abschnitt 3.1 zur Gesamtmenge der kritischen Operationen. Befehle, die einen Einfluss auf Programme und Prozesse nehmen können, der geeignet ist, Störungen auszulösen, gelten als kritisch. Aus verschiedenen Publikationen

zum *S7comm* [18, 19, 41, 42] wurden die Befehlssätze selektiert und mit den Befehlen aus Abschnitt 3.1 abgeglichen. Die auf diese Weise ermittelten kritischen Befehle (siehe Tabelle 4.4) stellen eine echte Teilmenge der kritischen Befehle aus Abschnitt 3.1 dar.

Tab. 4.4 Kritische Befehle S7comm

Gruppe	Kritische Gruppe	Kritische Operation
0	00_Generic	Generische Regeln und RFU
1	01_Download	start download all stop download all
2	02_Upload	start program upload stop program upload
5	05_Date-Time	set clock
7	07_Value	set valid value set multiple point values
8	08_Adress	set I/O-Adress modify I/O-Adress delete I/O-Adress
9	09_Authentication	success authentication
10	10_Mode	start PLC stop PLC
12	12_Communication	start communication end communication
13	13_Functions	Block Function (OB, DB, SDB, FC, SFC, FB, SFB) CPU Function (SZL) Programmer Command
14	14_Memory	memory reset ram to rom (copy memory) memory dump
15	15_Reset	reset factory setting

Auf der Grundlage der erstellten Netzwerkmitschnitte¹ und ihrer Auswertung wurden die Regeln erarbeitet. Eine Regel besteht aus einer Aktion, einem Header und verschiedenen Regeloptionen (siehe Listing 4.1). Die Aktion (blau) bestimmt, was passiert, wenn die Signatur übereinstimmt. Der Header (lila) definiert das Protokoll, die IP-Adressen, die Ports und die Richtung der Regel. Die Regeloptionen (rot) beschreiben die Besonderheiten der Regel [43].

```
action protocol source port -> destination port (message; content; offset;
depth; classtype; sid; rev;)
```

Listing 4.1 Regel-Muster

Beispielsweise löst die folgende Regel (Listing 4.2) einen Alarm aus, wenn ein nicht autorisierter Host eine Anfrage zum Verbindungsaufbau an die Master-PLC sendet:

```
# Alert on any connection request sent to a PLC on TCP/S7_PORT from an
  unauthorized host.
alert tcp-pkt !$S7_CLIENT any -> $S7_SERVER $S7_PORT (msg: "ALDI Setup
  Communication Request to Server attempt from non authorized Host"; flow:
  to_server, established; dsize: > 20; content: "|32 01|"; offset:7; depth:
  :2; content: "|F01"; offset:17; depth:1; classtype:bad-unknown; sid
  :2534321201; rev:2;)
```

Listing 4.2 S7comm-Regel zum Kommunikationsaufbau in Suricata

Die Aktionen werden in den Regeln immer mit 'alert' ausgeführt, auch wenn nur protokolliert werden soll. Suricata hat keine gesonderte Aktion zum Protokollieren implementiert. Es besteht die Möglichkeit, in der Datei 'thresholds.conf' anhand der SID oder auch basierend auf Quell- und Ziel-IP den Alarm zu unterdrücken. Für die Anwendung des Framework-Konzeptes auf *S7comm* wurden in der 'thresholds.conf' Alarmauslösungen anhand der SID unterdrückt.

In den Regeln wird auf das Protokoll TCP-PKT gefiltert. Transportprotokoll ist in den PCAPS TCP. In den IDS-Regeln kann sowohl auf TCP als auch auf TCP-PKT gefiltert werden. Beim Testen der Regeln in der Suricata-VM erzielten TCP und TCP-PKT die gleiche Anzahl von Treffern. TCP-PKT benötigte laut Suricata weniger Zeit als TCP für das Ergebnis. Überprüft wurde der Netzwerkverkehr zwischen den Clients von beliebigen Ports und der Master-PLC auf dem Port 102. Der Master-PLC ist die Variable \$S7_SERVER zugewiesen. Die Hosts können je nach Anfrage die Variablen \$S7_EWS, \$S7_HMI oder \$S7_CLIENT sein.

Die Regeloptionen beinhalten eine entsprechend angepasste Nachricht, eine Übertragungsrichtung zu oder vom Server, eine Mindestpaketgröße der zu übertragenden Daten, die Offsets der zu prüfenden Inhalte, einen Klassentyp und die SID.

Regeln wurden zu den in Tabelle 4.4 gelisteten kritischen Befehlen mit Ausnahme 08_Adress entwickelt. Diese Gruppe umfasst Befehle zum Schreiben, Verändern und Löschen von einzelnen Speicheradressen und begrenzten Speicheradressen-Bereichen. Für Speicherbereiche ist die Erstellung von IDS-Regeln nicht möglich. Um Speicherbereiche und Variablen zu überprüfen, müssten Listen mit Werten geschrieben werden, die in allgemeinen IDS-Regeln nicht erfasst werden können. Thresholdregeln wurden zur Minimierung der Log-Größen und für Schwellenwertüberschreitungen bei Schreiboperationen erstellt.

Die erstellten Regeln sind im Anhang B gelistet.

4.4 Anwendung der erstellten Regeln

Alle erstellten Regeln wurden in das Open Source IDS Suricata implementiert und auf ihre Treffer-Rate überprüft.

4.4.1 Einfache Regeln

Zu den einfachen Regeln gehört aus der Gruppe 12_Communication der kritischen Befehle der Befehl zum Kommunikationsaufbau. Jede Kommunikation zwischen Master-PLC und Host im ICS beginnt mit einem TCP-Handshake und einer *S7comm*-Anfrage zum Kommunikationsaufbau, wie sie beispielhaft in Abbildung 4.9 zu sehen ist. Das HMI mit der IP 10.10.10.20 schickt an die Master-PLC eine Anfrage mit gesetztem SYN-Flag. Die Master-PLC mit der IP antwortet mit dem Bestätigungssegment mit gesetztem SYN-Bit und ACK-Bit. 10.10.10.20 bestätigt schließlich den Empfang mit gesetztem ACK-Flag. Das S7-Protokoll ist in die Protokolle TPKT und ISO-COTP eingebettet, wodurch die PDU über TCP übertragen werden kann. In der *S7comm*-Funktion 'Setup Communication' werden Modalitäten zur Datenübertragung wie die Länge der Ack-Warteschlangen und die maximale Länge der PDU ausgehandelt. Erst wenn die Modalitäten des Verbindungsaufbaus zwischen den beteiligten Geräten ausgetauscht worden sind, kann eine weitere Kommunikation über *S7comm* erfolgen.

Source	Destination	Protocol	Length	Info
10.10.10.20	10.10.10.10	TCP	66	50408 → 102 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 W
10.10.10.10	10.10.10.20	TCP	60	102 → 50408 [SYN, ACK] Seq=0 Ack=1 Win=4096 Len=0
10.10.10.20	10.10.10.10	TCP	60	50408 → 102 [ACK] Seq=1 Ack=1 Win=64240 Len=0
10.10.10.20	10.10.10.10	COTP	76	CR TPDU src-ref: 0x0067 dst-ref: 0x0000
10.10.10.10	10.10.10.20	COTP	76	CC TPDU src-ref: 0x0006 dst-ref: 0x0067
10.10.10.20	10.10.10.10	S7COMM	79	ROSCTR:[Job] Function:[Setup communication]
10.10.10.10	10.10.10.20	S7COMM	81	ROSCTR:[Ack_Data] Function:[Setup communication]

Abb. 4.9 Darstellung eines Verbindungsaufbaus zwischen Host und Master-PLC in Wireshark

Zum Testen der Regeln für den Kommunikationsaufbau wurde ein Netzwerkmitschnitt eines Angreifers, der mehrmals erfolgreich eine Kommunikation zur Master-PLC aufgebaut hat, überprüft. In Wireshark konnten 12 Anfragen und Antworten zwischen den IPs 10.10.10.66 und 10.10.10.10 heraus gefiltert werden. Abbildung 4.10 zeigt die gefilterten Anfragen.

Wireshark bietet die Möglichkeit, in den Programmeinstellungen Suricata als Tool zu aktivieren. Wenn dies erfolgt ist, lässt sich der gesamte mitgeschnittene Netzwerkverkehr nach den Regeln, die in Suricata implementiert sind, filtern. Eine Alarmierung nach den einfachen Regeln erfolgt, wenn von bzw. zu einem nicht autorisierten Host eine Verbindungsanfrage oder Verbindungsbestätigung gesendet wird.

Abbildung 4.11 bestätigt, dass 12 Anfragen zu einem Kommunikationsaufbau erfolgt sind. Die Regel mit der SID 2534321201 erzielte die Treffer auf die Anfragen vom Client zur Master-PLC. Die SID 2534321202 erzielte Treffer auf die Antworten der Master-PLC zum Client.

Insgesamt wurden für die Regel mit der SID 2534321201 6704043 Inspektionen durchgeführt,

Destination	Source	Protocol	Length	Info
10.10.10.10	10.10.10.66	S7COMM	79	ROSCTR:[Job] Function:[Setup communication]
10.10.10.10	10.10.10.66	S7COMM	79	ROSCTR:[Job] Function:[Setup communication]
10.10.10.10	10.10.10.66	S7COMM	79	ROSCTR:[Job] Function:[Setup communication]
10.10.10.10	10.10.10.66	S7COMM	79	ROSCTR:[Job] Function:[Setup communication]
10.10.10.10	10.10.10.66	S7COMM	79	ROSCTR:[Job] Function:[Setup communication]
10.10.10.10	10.10.10.66	S7COMM	79	ROSCTR:[Job] Function:[Setup communication]
10.10.10.10	10.10.10.66	S7COMM	79	ROSCTR:[Job] Function:[Setup communication]
10.10.10.10	10.10.10.66	S7COMM	79	ROSCTR:[Job] Function:[Setup communication]
10.10.10.10	10.10.10.66	S7COMM	79	ROSCTR:[Job] Function:[Setup communication]
10.10.10.10	10.10.10.66	S7COMM	79	ROSCTR:[Job] Function:[Setup communication]
10.10.10.10	10.10.10.66	S7COMM	79	ROSCTR:[Job] Function:[Setup communication]
10.10.10.10	10.10.10.66	S7COMM	79	ROSCTR:[Job] Function:[Setup communication]

Abb. 4.10 Anfragen zum Verbindungsaufbau eines Angreifers (Source) in Wireshark

was einen Anteil von 48,12 % der gesamten Inspektionen entspricht. In 595 Fällen wurde eine Signatur geprüft. 51,88 % der Gesamtinspektionen fielen auf die Regel mit der SID 2534321202, was einer Anzahl von 7229144 entspricht. Für diese SID wurden in diesem PCAP 1208 Signaturen geprüft.

Rule	Gid	Rev	Ticks	%	Checks	Matches
2534321201	1	2	6704043	48.12	595	12
2534321202	1	2	7229144	51.88	1208	12

Abb. 4.11 Darstellung der erzielten Treffer zum Verbindungsaufbau eines Angreifers in Suricata

Abbildung 4.12 bildet einen Kommunikationsaufbau zwischen dem Angreifer und der Master-PLC detailliert ab. In der obersten Zeile steht das Testdatum. Darunter folgt die Regel, die auf die Anfrage vom Angreifer die Warnung erzeugt hat. Unten reiht sich die Regel an, die auf die Antwort vom Server einen Treffer erzielt hat. In Listing Anhang B.3 im Anhang finden sich die Regeln aus Abbildung 4.12 wieder.

Regeln zur Gruppe 09_Authentication aus werden ebenfalls unter den einfachen Regeln subsumiert.

In die Siemens S7-300 und in die Siemens S7-1200 sind Funktionen mit Authentifizierung implementiert. Die S7-300 fragt beim Download nach einem Passwort, wenn eines gesetzt wurde. In der S7-1200 können alle abzufragenden Werte durch den gleichen oder einen ähnlichen Mechanismus geschützt werden. Die hierzu erstellten PCAPS wurden an einer PLC der S7-300er Serie bei einem Download erzeugt.

Ein Alarm wird ausgelöst, wenn eine Authentifizierung von einem nicht autorisierten Host erfolgt. Die Regeln aus Listing Anhang B.2 wurden auf zwei unterschiedliche PCAPS angewendet. In


```
-----
Date: 30/5/2019 -- 22:25:21
-----
== Sid: 2534321201 ==
alert tcp-pkt !$S7_CLIENT any -> $S7_SERVER $S7_PORT (msg: "ALDI Setup
Communication Request to Server attempt from non authorized Host"; flow
:to_server,established; dsize:>20; content:"|32 01|"; offset:7; depth:2
; content:"|F0|"; offset:17; depth:1; classtype:bad-unknown; sid:253432
1201; rev:2;)
  Fast Pattern analysis:
    Fast pattern matcher: content
    Flags: Offset Depth
    Fast pattern set: no
    Fast pattern only set: no
    Fast pattern chop set: no
    Original content: 2\x01
    Final content: 2\x01

== Sid: 2534321202 ==
alert tcp-pkt $S7_SERVER $S7_PORT -> !$S7_CLIENT any (msg: "ALDI Setup
Communication Response from Server attempt to non authorized Host"; flo
w:to_client,established; dsize:>20; content:"|32 03|"; offset:7; depth:
2; content:"|F0|"; offset:19; depth:1; classtype:bad-unknown; sid:25343
21202; rev:2;)
  Fast Pattern analysis:
    Fast pattern matcher: content
    Flags: Offset Depth
    Fast pattern set: no
    Fast pattern only set: no
    Fast pattern chop set: no
    Original content: 2\x03
```

Abb. 4.12 Detaillierte Darstellung eines Request/ Response zum Verbindungsaufbau in Suricata

beiden Datensätzen wurde je eine Passwortanfrage von Suricata identifiziert.

4.4.2 Protokollregeln

Protokollregeln wurden zu den Gruppen 01_Download, 02_Upload, 05_Date-Time, 07_Value, 10_Mode, 13_Functions, 14_Memory und 15_Reset entwickelt.

Für Download-Operationen wurden verschiedene Szenarien an der Testanlage durchgeführt: *Download all*, *Download Software change* und *Download Hardware configuration*. Die Netzwerkmitschnitte unterschieden sich bezüglich der unterschiedlichen Download-Aktivitäten technisch nicht. In den PCAPS zeigten sie sich immer als Abfolge von den drei Function-Codes 'Request download' (0x1a), 'Download block' (0x1b) und 'Download ended' (0x1c), wobei 'Down-

load block' durchaus mehrfach in der Abfolge ausgeführt wurde. Das ist ein regulärer Vorgang. Während 'Request Download' und 'Download ended' für jeden abgeschlossenen Download nur einmal ausgeführt werden sollten, kann es durchaus sein, dass der Payload so groß ist, dass mehrere Blöcke für den Download benötigt werden. Abbildung 4.13 zeigt in Wireshark, wie die Befehle in der beschriebenen Reihenfolge gelistet werden.

10.10.10.10	10.10.10.30	S7COMM	104	ROSCTR:[Job]	Function:[Request download] -> Block:[SDB22]
10.10.10.30	10.10.10.10	S7COMM	74	ROSCTR:[Ack_Data]	Function:[Request download]
10.10.10.30	10.10.10.10	S7COMM	89	ROSCTR:[Job]	Function:[Download block] -> Block:[SDB22]
10.10.10.10	10.10.10.30	S7COMM	162	ROSCTR:[Ack_Data]	Function:[Download block]
10.10.10.30	10.10.10.10	S7COMM	89	ROSCTR:[Job]	Function:[Download ended] -> Block:[SDB22]
10.10.10.10	10.10.10.30	S7COMM	74	ROSCTR:[Ack_Data]	Function:[Download ended]

Abb. 4.13 Darstellung der Befehlsabfolge eines Downloads in Wireshark

Beim Download gibt es die Besonderheit, dass der Client nur für den 'Request download' die Anfrage stellt und vom Server die Antwort erhält. In Listing 4.3 sind die Regeln, auf die dieser Befehl Treffer erzielt, abgebildet. Für die Befehle 'Download block' und 'Download ended' stellt jeweils der Server die Anfrage und der Client antwortet.

```
# Message on any Connection Request Download sent to a PLC o TCP/$S7_PORT
from an authorized host (only log).
alert tcp-pkt $S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI Function '
Request Download' from authorized Host"; flow:to_server,established;
content:"|32 01|"; offset:7; depth:2; content:"|1A|"; offset:17; depth:1;
classtype: not-suspicious; sid:2534320101; rev:2;)

# Message on Response to execute a request download sent by a PLC on
TCP/$S7_PORT to an authorized host (only log).
5 alert tcp-pkt $S7_SERVER $S7_PORT -> $S7_EWS any (msg: "ALDI Command
Response 'Request Download' from Server to authorized Host"; flow:
to_client,established; content:"|32 03|"; offset:7; depth:2; content:"|1A
|"; offset:19; depth:1; classtype: not-suspicious; sid:2534320102; rev
:2;)
```

Listing 4.3 Regeln zum 'Request download' in S7comm

In den Regeln wird immer protokolliert, wenn eine Download-Aktivität getätigt wird. Eine Alarmierung erfolgt nur, wenn ein nicht autorisierter Host an der Aktion beteiligt ist.

Abbildung 4.14 zeigt, dass Treffer zu Regeln aus Listing Anhang B.4 erzielt wurden. Mit Suricata wurden zum 'Request download' 14 Anfragen von der EWS und 14 Antworten von der Master-PLC identifiziert. 'Download block' ergab 21 Anfragen von der, Master-PLC und 21 Antworten von EWS. 'Download ended' ergab jeweils 14 Treffer. Das Ergebnis stimmt mit den Testdaten überein. Im getesteten PCAP wurden mehrere Downloads verschiedener Größe mitgeschnitten.

Um Mitschnitte zu Upload-Operationen zu erzeugen, wurden eine Offline und eine Onlineversion mit Code-Unterschieden von einer Engineering Workstation abgeglichen. Der Versionsabgleich

Rule	Gid	Rev	Ticks	%	Checks	Matches
2534320106	1	2	1486542	2.80	39	21
2534320110	1	2	1196412	2.25	39	14
2534320105	1	2	1501606	2.83	84	21
2534320109	1	2	1372096	2.58	84	14
2534320101	1	2	14944060	28.14	2414	14
2534320102	1	2	32609253	61.40	5372	14

Abb. 4.14 Darstellung der erzielten Treffer zum Download in Suricata

löste eine Upload-Aktion aus. Ähnlich wie die Downloadfunktion der Gruppe 01_Download wurden die Updates als eine Abfolge von drei Function-Codes durchgeführt. Mit 'Start upload' (0x1d) wurde die Aktion jeweils angestoßen. Dann folgten ein oder mehrere Befehle 'Upload' (0x1e). Beendet wurden die Aktionen jeweils mit 'End upload' (0x1).

Alle drei Function-Codes erzielten mit den Regeln aus Listing Anhang B.5 Treffer in Suricata. Die Uploads werden immer protokolliert, um bei Störungen nachvollziehen zu können, welche Aktivitäten durchgeführt wurden. Eine Alarm wird ausgelöst, wenn ein nicht autorisierter Host einen Upload durchführt.

Die zugehörigen Befehle zum Einstellen der Zeit in Gruppe 05_Date-Time werden in *S7comm* über die Funktion Time Function und ihre Unterfunktionen zum Lesen und Setzen der Uhrzeit ausgeführt. Alarmiert wird, wenn ein nicht autorisierter Host eine Änderung der Uhrzeit anfragt. Autorisierte Modifikationen werden protokolliert. Auf diese Operationen werden auch durch die Regeln in Listing Anhang B.6 Treffer erzielt.

Die Befehle in der Gruppe 07_Value sind von der technischen Übertragung her einfache Schreibzugriffe. Die Regeln in Listing Anhang B.7 erzielten ihre Treffer auf den Funktionswert 0x05 im Parameter-Header. Dieser Wert wurde in Abbildung 4.5 optisch dargestellt.

Schreibzugriffe von autorisierten Hosts werden protokolliert, nicht autorisierte Schreibzugriffe lösen einen Alarm aus.

Befehle aus der Gruppe 10_Mode werden in *S7comm* als Request-/ Response-Nachrichten bearbeitet. Sie erzielen Treffer auf die Funktionswerte im Parameter. Die Regeln in Listing Anhang B.8 im Anhang protokollieren bzw. alarmieren bei unautorisiertem Zugriff jedes Starten und jedes Anhalten der PLC. Da Anlagen in ICS in der Regel hochverfügbar sein müssen, weisen Start- und Stop-Operationen insbesondere bei häufigeren Auftreten auf Anomalien hin.

Für die Operationen 'PLC Start' und 'PLC Stop' standen verschiedene PCAPS zur Verfügung. In allen Datensätzen wurden die kritischen Befehle vollständig durch Suricata identifiziert.

S7comm implementiert eine Reihe von Befehlen als Funktionen. Es existieren Funktionen zur Übertragung bzw. Ausführung von Datenblöcken (DB), Organisationsbausteinen OB),

Funktionen (FC)/ Systemfunktionen (SFC) und Funktionsbausteine (FB/ Systemfunktionsbausteine (SFB), die unter den Block Functions subsumiert werden. Für alle Funktionen existieren Unterfunktionen. Regeln können auf Funktionen, Unterfunktionen und die unterschiedlichen Datenblöcke in allen möglichen Kombinationen erstellt werden.

Organisationsbausteine bilden die Schnittstelle zwischen dem Betriebssystem und dem Anwenderprogramm. Mit Hilfe von Organisationsbausteinen (OB) können Programmteile z. B. beim Anlauf der CPU, in zyklischer oder auch zeitlich getakteter Ausführung und beim Auftreten von Fehlern oder Prozessalarmen, gezielt zur Ausführung gebracht werden.

Systemfunktionen (SFC) sind im Betriebssystem integrierte Funktionen. Funktionen (FC) sind Unterprogramme, die entweder von Organisationsbausteinen (OB) oder von anderen Funktionen (FC) oder Funktionsbausteinen (FB) aufgerufen werden. Funktionen (FC) besitzen im Gegensatz zu Funktionsbausteinen (FB) keinen Speicherbereich, der über den Bausteinaufruf hinaus gültig ist. Nach der Bearbeitung der Funktion gehen die lokalen Daten (z.B. Temperatur) verloren.

Funktionsbausteine (FB)/ Systemfunktionsbausteine (SFB) sind Baugruppen, die über einen Systemdatenbereich verfügen, auf den Sie von Ihrem Programm aus nur schreibend oder lesend zugreifen können.

Klassische Funktionen sind das 'Lesen von Schutzstufen' in den CPU-Functions, 'Löschen' im Programmer Command und 'Informieren über Datenblöcke' in den Blockfunctions.

Tabelle 4.5 zeigt die Funktionen und Unterfunktionen, die in den analysierten PCAPS mitgeschnitten wurden. Für die Gruppe 13_Funktionen wurden Regeln auf die Funktionen Block Functions, CPU-Functions und Programmer Command erstellt, die in Suricata Treffer erzielten. Diese Regeln in Listing Anhang B.9 sind Beispiele. Es konnte in der Literatur nicht recherchiert werden, welche Unterfunktionen tatsächlich kritische Befehle beinhalten. Einige Unterfunktionen konnten ihrer Verwendung in kritischen Operationen zugeordnet werden. Z. B. wurde beim memory reset die Funktion Programmer Command mit der Unterfunktion erase gestartet. Daraus wurde der Rückschluss gezogen, dass das ein Löschbefehl sein muss.

memory reset und ram to rom aus der Gruppe 14_Memory sind Funktionen in S7comm. Die Funktion Programmer Command führt diese Operationen mit unterschiedlichen Unterfunktionen aus. Im PCAP zum memory reset wird durch die Unterfunktion erase der Befehl 'Löschen' ausgeführt. Im PCAP zum ram to rom führt ausschließlich die Unterfunktion forces Befehle aus.

Treffer werden durch die Regeln 2534321401 – 2534321408 in Listing Anhang B.10 erzielt.

Für die Regelerstellung in Gruppe 15_Reset wurde ein Zurücksetzen auf Werkseinstellungen mitgeschnitten. Das PCAP gleicht technisch dem Mitschnitt vom ram to rom aus der Gruppe 14_Memory. Die Funktion *Programmer Command* führt diese Operation mit der Unterfunktion forces durch. Eine neue Regel wurde nicht erstellt. Die Regeln 2534321405 - 2534321408 aus Listing Anhang B.10 erzielen Treffer.

Tab. 4.5 Userdata Functions S7comm

Funktion	Unterfunktion
Programmer Command	erase forces read diag data
Block Functions	list blocks list blocks of type get block info
CPU functions	read SZL message service
Security	PLC password
Time functions	read clock set clock

4.4.3 Thresholdregeln

Thresholdregeln werden verwendet, um die Alarmfrequenz einer Regel zu steuern. Es gibt 3 Modi: die Begrenzung des Schwellenwertes (Typ 'threshold'), des Grenzwertes (Typ 'limit') sowie eine Kombination aus beiden Werten (Typ 'both'). Der Typ 'threshold' kann verwendet werden, um einen Mindestschwellenwert für eine Regel festzulegen, bevor sie Warnungen erzeugt. Der Typ 'limit' kann verwendet werden, um die Größe der Log-Dateien auf ein Minimum zu senken. Schwellenwerte können sowohl pro Regel als auch global konfiguriert werden. [43]

Es wurden Thresholdregeln für die einzelnen Regeln und eine globale Thresholdregel erstellt. Für die Erstellung der Eventfilter für einzelne Regeln stand die Minimierung der Log-Dateien im Vordergrund. Die erstellte globale Thresholdregel unterscheidet zwischen normalem und abnormalem Verhalten bei Anfragen mit regulären Werten und Operationen und warnt, wenn die Anfragen für Schreibzugriffe einen bestimmten Wert überschreiten. Ein ungewöhnlich hohe Anzahl von Anfragen kann ein Indiz für einen Angriff oder einen Fehler im Produktionsablauf sein.

```
threshold gen_id <gid>, sig_id <sid>, type <threshold|limit|both>, \
  track <by_src|by_dst>, count <N>, seconds <T>
```

Abb. 4.15 Vorlage der Suricata-Dokumentation zum Erstellen der Eventfilter [24]

Die Schwellenwerte für die einzelnen Regeln werden über Eventfilter in der Konfigurationsdatei 'thresholds.conf' festgelegt. Abbildung 4.15 zeigt die Vorlage, nach der die Eventfilter erstellt

wurden.

Für die getesteten Regeln wurde eine Begrenzung von 3 Anfragen innerhalb von 5 Sekunden an die Ziel-IP festgelegt. Der Eventfilter (Listing B.14) für das Schreiben eines Wertes lautet:

```
threshold gen_id 1, sig_id 2534320701, type limit, track by_dst, count 3,
seconds 5
```

Listing 4.4 Eventfilter

Die globale Thresholdregel (siehe auch Listing Anhang B.12) warnt, wenn eine ungewöhnlich hohe Rate von Anfragen für Schreiboperationen auftritt. Ein Alarm wird ausgelöst, wenn Anfragen, die an eine Master-PLC auf TCP Port 102 gesendet werden, einen definierten Schwellenwert überschreiten.

Der Schwellenwert wurde für die Anwendung des Konzeptes für *S7comm* auf 200 Anfragen innerhalb von 1 Minute festgelegt. Für die Berechnung dieses Wertes wurden die Anfragen für Schreiboperationen aus den PCAPS der QUT Infosec Gruppe¹ gezählt. Diese lagen im Maximum bei ca. 180 Anfragen pro Minute. Dieser Wert wurde mit dem Faktor '1, 10' multipliziert und als Schwellenwert festgelegt. Im Vergleich lagen die Anfragen für Schreiboperationen des Angreifers bei ungefähr 12.000 Anfragen in 1 Minute.

Abbildung 4.16 stellt dar, dass der Angreifer für Leseoperationen signifikant weniger Anfragen tätigte als der durchschnittliche reguläre Client. Anfragen für Leseoperationen lagen in den ausgewerteten PCAPS im Mittel bei ca. 250 Anfragen pro Minute. Die Anfragen des Angreifers lagen dagegen bei 5-6 Anfragen pro Minute.

Der globale Schwellenwert muss in der Produktiv-Umgebung jeweils an die individuelle Nutzung der Anlage angepasst werden. Wenn der Schwellenwert festgelegt wurde, kann er gegebenenfalls aus der Regel herausgelöst werden. Bei etwaigen Updates entfällt dann eine erneute Anpassung.

```
# Alert on Connection Request that is sent to a PLC on TCP/$TS_PORT if it
# passes the defined threshold: 200 within 60 seconds
alert tcp-pkt any any -> $S7_SERVER $S7_PORT (msg:"ALDI Connection Request
to Server attempt above threshold"; content:"|32 01|"; offset:7; depth:2;
content:"|05|"; offset:17; depth:1; threshold: type threshold, track
by_dst, count 200, seconds 60; classtype:bad-unknown; sid:2534321601; rev
:2;)
```

Listing 4.5 Threshold Regel

4.4.4 Exploitregeln

Exploitregeln konnten nicht für das *S7comm*-Protokoll erstellt werden. Es lagen keine PCAPS vor, die einen bekannten Angriff auf die Testanlage mitgeschnitten haben. Es war auch nicht Ziel

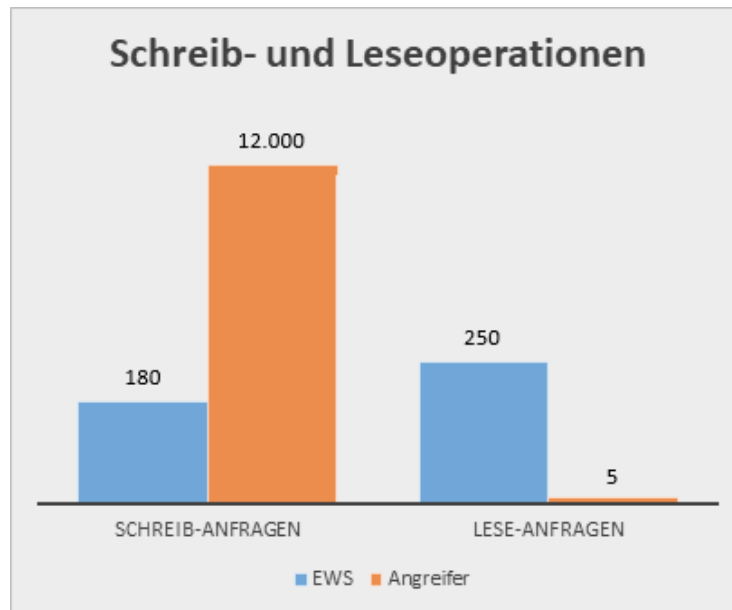


Abb. 4.16 Darstellung der Schreib- und Leseoperationen EWS vs. Angreifer

der Arbeit, spezielle Exploitregeln zu entwickeln. Der Augenmerk lag auf der Entwicklung des Konzeptes für ein Framework. Aus diesem Grund wurden auch keine speziellen PCAPS erzeugt.

4.4.5 Adressbereich Regeln

Adressregeln konnten für *S7comm* nicht erstellt werden. Mit IDS-Regeln lassen sich direkt keine Treffer auf die Adressbereiche bestimmter Werte erzielen. In Tabelle 4.3 sind ein Teil der zulässigen Wertebereiche für den 'Tank' gelistet. Solche Wertebereiche sind auch für den 'Reactor' und den 'Conveyor' festgelegt worden. Jeder dieser einzelnen Werte müsste gesondert geprüft werden. Dafür müssten Skripte geschrieben werden, die in Konfigurationsdateien implementiert werden. Der Teil war nicht Aufgabe dieser Arbeit.

4.4.6 Allgemeine Regeln

Diese Regeln warnen bei jedem Paket, das innerhalb des Netzwerks über TCP/ Port 102 von bzw. an eine PLC gesendet wird. Diese Regeln sind nicht für den Einsatz in einer Produktiv-Umgebung vorgesehen. Die Allgemeinen Regeln können in Listing Anhang B.13 eingesehen werden.

5 Diskussion

Für das Protokoll *S7comm* wurde das Konzept des Frameworks angewendet. Für die Anwendung stand eine Testanlage, die eine Raffinerie simuliert, zur Verfügung. Zu den einzelnen Kategorien der kritischen Befehle wurden PCAPS als Testdatensätze in dieser Testanlage erstellt und analysiert. Anhand dieser Analysen wurden nach dem Konzept des Frameworks Regeln erstellt.

Bei der Entwicklung und dem Testen der Regeln wurde festgestellt, dass die durch das Framework erstellten Regelsätze wie in Abschnitt 4.3 für verschiedene Gruppen der kritischen Befehle Treffer erzielten. Das Erstellen allgemeiner Regelsätze für die jeweiligen Protokolle lässt sich zumindest für den Teil der einfachen Regeln und für Teile des Regelsatzes für Protokollregeln realisieren.

In den Protokollregeln gab es Besonderheiten mit der Gruppe 13_Functions. *S7comm* implementiert eine Reihe von Befehlen als Funktionen. Dazu gehören beispielsweise die Zeitsynchronisation (=Time Functions) und die Passwortabfrage (=Security). Es existieren auch Funktionen zur Übertragung bzw. Ausführung von Datenblöcken (DB), Organisationsbausteinen (OB), Funktionen (FC)/ Systemfunktionen (SFC) und Funktionsbausteinen (FB)/ Systemfunktionsbausteinen (SFB), die unter den Block Functions subsumiert werden. Für alle Funktionen existieren Unterfunktionen. Sowohl auf Funktionen, Unterfunktionen als auch auf die unterschiedlichen Datenblöcke können in allen möglichen Kombinationen IDS-Regeln erstellt werden, aufgrund denen in den Test-Datensätzen Treffer erzielt werden. Nicht alle Unterfunktionen und Datenblöcke sind kritisch. Es kann auch sein, dass ein eigentlich kritischer Datenblock mit der Ausführung der entsprechenden Unterfunktion 'unkritisch' wird. Das hat zur Folge, dass unterschiedliche, individuelle Regeln für einzelne Befehle dieser Gruppe entwickelt werden müssen. Eine individuelle Regelerstellung widerspricht jedoch dem Konzept, ein allgemeingültiges Framework zu erstellen.

Für Wertebereiche und Variablen ist die Erstellung von IDS-Regeln nicht möglich. Um Wertebereiche und Variablen zu überprüfen, müssten Listen mit Werten geschrieben werden, die in allgemeinen IDS-Regeln nicht erfasst werden können. Das müsste über das Einbinden von Scripten in den Konfigurations-Dateien umgesetzt werden.

Der Nachteil zur Vorgehensweise mit Scripten ist, dass diese Vorgehensweise die Performance beeinträchtigen kann. Derartige Beeinträchtigungen könnten die Anwendung in einer Produktivumgebung in Zweifel ziehen. In weiteren Forschungen sollte untersucht werden, inwieweit tatsächlich die Performance beeinträchtigt wird. Für PCAP-Auswertung und Forensische Analysen wären Scripte eine Bereicherung.

Das Konzept des Frameworks bietet die Möglichkeit, allgemeine Regeln zu erstellen, die für

viele Operationen zutreffend sind. Das hat die Anwendung für das *S7comm* Protokoll gezeigt. Auch wenn sich nicht alle kritischen Szenarien über generierte IDS-Regeln abbilden lassen, bietet diese Arbeit einen Mehrwert, weil sich ein großer Teil der kritischen Befehle über dieses Framework durch Regeln absichern lässt.

6 Fazit und Ausblick

Die allgemeine Cyber-Sicherheitslage ist angespannt. Dies gilt auch für industrielle Anlagen, deren Betreiber sich mit der stetig steigenden Vernetzung mit IT-Systemen auseinandersetzen müssen.

IDS bieten eine Möglichkeit, laufende Angriffe zu erkennen, bei denen der Angreifer sich bereits im Netz befindet. Bestehende Arbeiten konzentrieren sich auf die Anwendung von IDS und der Erkennung von Angriffen anhand der Angriffsvektoren auf der Basis verschiedener Modelle.

Diese Arbeit konzentriert sich auf die Darstellung von Grundlagen zur Erstellung von IDS-Regeln. Damit kann der Anwender seine Regeln selbst schnell erzeugen, was zu einer erhöhten Akzeptanz für die Anwendung von IDS im Netzwerk sorgen soll.

Im Grundlagenteil wurden zunächst der generelle Aufbau von ICS, die Unterschiede von einem ICS zur herkömmlichen IT und die *S7comm*-Kommunikation beschrieben. Dies dient zum generellen Verständnis, um spezifischen Schwachstellen in ICS durch die genutzten Protokolle zu verstehen bzw. nachvollziehen zu können. Im weiteren Verlauf dieser Arbeit wurde der Mehrwert durch den Einsatz eines IDS in einem ICS veranschaulicht.

Im Kapitel 3 wurde die Erstellung des Konzeptes eines allgemeinen Frameworks zur teilautomatisierten Erstellung von prozessbasierten IDS-Regeln in ICS beschrieben. Hierzu wurde die Methodik zur Identifizierung und Klassifizierung der kritischen Befehle und Operationen aufgezeigt und eine Vorgehensweise zum Erstellen von Regelsätzen formuliert.

Für das Protokoll *S7comm* wurde das Konzept angewendet. Dafür wurden für verschiedene Szenarien Test-PCAPS in einer Testanlage, die eine Raffinerie simuliert, erzeugt und analysiert. Anhand dieser Analysen wurden Regeln nach dem Konzept des Frameworks erstellt. Die Erprobung der Regeln wurde in einer virtuellen Suricata-Maschine durchgeführt.

Die Anwendung mit *S7comm* zeigt, dass sich das Konzept in Teilen umsetzen lässt. Abschnitt 4.4.1 und 4.4.2 dokumentieren, dass sich für die Teile der einfachen Regeln und der Protokoll-Regeln anhand des Frameworks IDS-Regeln erzeugen lassen, die auf den Testdaten von verschiedenen Klassen kritischer Befehle Treffer erzielen. In der Gruppe 13_Function gibt es die Besonderheit, dass unterschiedliche Funktionen verschiedene Unterfunktionen subsumieren. Die Unterfunktionen stellen zum Teil kritische Operationen dar. Alle Unterfunktionen der Funktion Block Functions sind auf verschiedene Blöcke anwendbar, die in den BlockFunctions selbst abschließend gelistet sind. Auf den Funktionen, Unterfunktionen und auf die unterschiedlichen Datenblöcke können in allen möglichen Kombinationen IDS-Regeln erstellt werden. Nicht alle Unterfunktionen und Datenblöcke sind kritisch. In bestimmten Kombinationen kann es jedoch möglich sein, dass eigentlich kritische Datenblöcke im Zusam-

menhang mit der Ausführung unkritischer Unterfunktionen als unkritisch eingeordnet werden müssen. Das hat zur Folge, dass unterschiedliche, individuelle Regeln für einzelne Befehle dieser Gruppe entwickelt werden müssen. Für andere kritische Befehle lassen sich keine IDS-Regeln erzeugen, weil es z. B. nicht möglich ist, mit den Regeln allein Wertebereiche abzufragen.

Ausblick

Mit dem entwickelten Konzept ist es möglich, allgemeine Regeln zu erstellen, die für einen großen Teil der kritischen Operationen zutreffend sind. Es können jedoch nicht alle kritischen Szenarien über generelle IDS-Regeln abgebildet werden. In weiteren Forschungsarbeiten sollte untersucht werden, inwieweit mit anderen Mitteln und Script-Sprachen fehlende kritische Szenarien aufgefangen werden können und mit welchen Ressourcen eine Umsetzung möglich ist.

In dieser Arbeit wird das Framework für ein Protokoll angewendet. Das Konzept des Frameworks und die Vorgehensweise für das Protokoll dienen als Grundlage für die teilautomatisierte Implementierung des Frameworks. Sie bilden aber auch die Grundlage für die Aufnahme weiterer Protokolle.

Im Weiteren gilt es zu prüfen, wie sich die nach dem Konzept erstellten Regeln in der Praxis verhalten. Hierfür müssen möglicherweise Anpassungen in den Konfigurationsdateien vorgenommen werden.

Abschließend lässt sich feststellen, dass durch das Konzept des Frameworks eine Lösung aufgezeigt wird, die einen großen Teil kritischer Befehle über IDS-Regeln absichert.

7 Literaturverzeichnis

- [1] „*Attacken auf deutsche Industrie verursachten 43 Milliarden Euro Schaden*“ vom 13.09.2018. <https://www.bitkom.org/Presse/Presseinformation/Attacken-auf-deutsche-Industrie-verursachten-43-Milliarden-Euro-Schaden.html>, 2018. – Accessed: 03.12.2018
- [2] KREMP, Mathias: „*Spektakuläre Virus-Analyse: Stuxnet sollte Uran-Anreicherung stören*“ vom 16.11.2010. <http://www.spiegel.de/netzwelt/gadgets/spektakulaere-virus-analyse-stuxnet-sollte-irans-urananreicherung-stoeren-a-729329.html>, 2010. – Accessed: 03.12.2018
- [3] ANTON CHEREPANOV, ESET: „*Industroyer: Biggest threat to industrial control systems since Stuxnet*“ vom 12.06.2017. https://www.welivesecurity.com/wp-content/uploads/2017/06/Win32_Industroyer.pdf, 2017. – Accessed: 03.12.2018
- [4] ICS-CERT: „*MAR-17-352-01 HatMan – Safety System Targeted Malware (Update A)*“ vom 10.04.2018. [https://www.us-cert.gov/sites/default/files/documents/MAR-17-352-01HatMan-SafetySystemTargetedMalware\(UpdateA\)_S508C.PDF](https://www.us-cert.gov/sites/default/files/documents/MAR-17-352-01HatMan-SafetySystemTargetedMalware(UpdateA)_S508C.PDF), 2018. – Accessed: 08.12.2018
- [5] COLBERT, Alexander Edward J. M.; K. Edward J. M.; Kott: „*Cyber-security of SCADA and Other Industrial Control Systems*“. Springer-Verlag, 31.08.2016
- [6] TEAM, Suricata D.: „*UP KRITIS - Öffentlich-Private Partnerschaft zum Schutz Kritischer Infrastrukturen*“ vom Februar 2014. https://www.kritis.bund.de/SharedDocs/Downloads/Kritis/DE/UP_KRITIS_Fortschreibungsdokument.pdf, 2014. – Accessed: 20.05.2019
- [7] EIGNER, Oliver ; KREIMEL, Philipp ; TAVOLATO, Paul: Identifying S7comm Protocol Data Injection Attacks in Cyber-Physical Systems. In: *5th International Symposium for ICS & SCADA Cyber Security Research 2018 (ICS-CSR 2018)* (2018), S. 51–56
- [8] „*BSI ICS-Security-Kompendium vom 25.11.2013*“. https://www.bsi.bund.de/SharedDocs/Downloads/DE/BSI/ICS/ICS-Security_kompendium_pdf, 2013. – Accessed: 03.12.2018
- [9] YÜKSEL, Ömer ; HARTOG, Jerry den ; ETALLE, Sandro: Reading between the fields: practical, effective intrusion detection for industrial control systems. In: *ACM Symposium on Applied Computing* (2016), S. 2063–2070

- [10] TÜV-NORD: „*IT-und-Funktionale-Sicherheit-im-Fokus-der-Industrie-4.0*“ vom 03.11.2015. <https://www.tuev-nord-group.com/de/newsroom/aktuelle-pressemeldungen/details/article/it-und-funktionale-sicherheit-im-fokus-der-industrie-40/>, 2015. – Accessed: 08.05.2019
- [11] ISI-PROJEKTGRUPPE: „*Internet-Sicherheit: Einführung, Grundlagen, Vorgehensweise*“ von 2011. <https://www.bsi.bund.de/SharedDocs/Downloads/DE/BSI/Internetsicherheit/ISi-E.pdf>, 2011. – Accessed: 26.05.2019
- [12] WILLIAMSON, Graham: „*OT, ICS, SCADA – What’s the difference?*“ vom 07.07.2015. <https://www.kuppingercole.com/blog/williamson/ot-ics-scada-whats-the-difference>, 2015. – Accessed: 26.05.2019
- [13] CLINT BODUNGEN, Aaron Shbeeb Kyle Wilhoit Stephen H. Bryan Singer S. Bryan Singer: *Hacking Exposed - Industrial Control Systems*. 1. Auflage. McGraw-Hill Education Ltd, 2016
- [14] OBREGON, Luciana: „*Secure Architecture for Industrial Control Systems*“ vom 23.09.2015. <https://www.sans.org/reading-room/whitepapers/ICS/secure-architecture-industrial-control-systems-3632>, 2015. – Accessed: 31.03.2019
- [15] CHAN, Zhanwei: „*Increasing resiliency by segmenting the OT network based on the Purdue model*“ vom 22.03.2019. <https://technical.nttsecurity.com/post/102fh58/increasing-resiliency-by-segmenting-the-ot-network-based-on-the-purdue-model>, 2019. – Accessed: 09.05.2019
- [16] STOUFFER, Keith ; FALCO, Joe ; SCARFONE, Karen: „Guide to industrial control system security“ von Mai 2015. In: *Technical Report SP 800-82* (2011)
- [17] TOM, S. ; CHRISTIANSEN, D. ; BERRRETT, D.: Recommended practice for patch management of control systems. In: *Department of Homeland Security (DHS)* (2008)
- [18] SIEMENS, Fa.: „*What properties, advantages and special features does the S7 protocol offer?*“ vom 24.09.2007. <https://support.industry.siemens.com/cs/document/26483647/what-properties-advantages-and-special-features-does-the-s7-protocol-offer>, 2007. – Accessed: 12.02.2019
- [19] NARDELLA, Davide: „*Step7 Open Source Ethernet Communication Suite*“ vom 03.09.2013. <http://snap7.sourceforge.net/>, 2013. – Accessed: 12.02.2019
- [20] MIRU, Gyorgy: „*The Siemens S7 Communication - Part 2 Job Requests and Ack Data*“ vom 30.01.2016. <http://gmiru.com/article/s7comm/>, 2016. – Accessed: 28.01.2019
- [21] KAPPES, Martin: *Netzwerk- und Datensicherheit: Eine praktische Einführung*. 2. Auflage. Springer Verlag, 2013

- [22] DAVIDOFF, Sherri ; HAM, Jonathan: *Network Forensics: Tracking Hackers Through Cyberspace*. 1. Auflage. Prentice Hall, 2012
- [23] VERBA, Jared ; MILVICH, Michael: Idaho national laboratory supervisory control and data acquisition intrusion detection system (SCADA IDS). In: *2008 IEEE Conference on Technologies for Homeland Security* (2008), S. 469–473
- [24] TEAM, Suricata D.: „*Suricata User Guide*“ vom 2016. <https://suricata.readthedocs.io/en/suricata-4.1.4/>, 2016. – Accessed: 17.05.2019
- [25] GENGE, Béla ; RUSU, Dorin A. ; HALLER, Piroška: A connection pattern-based approach to detect network traffic anomalies in critical infrastructures. In: *EuroSec '14 Proceedings of the Seventh European Workshop on System Security* (2014), S. 1–6
- [26] GHAEINI, Hamid R. ; TIPPENHAUER ; OLE, Nils: HAMIDS - Hierarchical Monitoring Intrusion Detection. In: *CPS '17 Proceedings of the 2017 Workshop on Cyber-Physical Systems Security and Privacy* (2016), S. 103–111
- [27] JARDINE, William ; FREY, Sylvain ; GREEN, Benjamin ; RASHID, Awais: SENAMI: Selective Non-Invasive Active Monitoring for ICS Intrusion Detection. In: *CPS-SPC '16: Proceedings of the 2nd ACM Workshop on Cyber-Physical Systems Security and Privacy* (2016), S. 23–34
- [28] MARKMAN, Chen ; WOOL, Avishai ; CARDENAS, Alvaro: A New Burst-DFA model for SCADA Anomaly Detection. In: *CPS '17 Proceedings of the 2017 Workshop on Cyber-Physical Systems Security and Privacy* (2017), S. 1–12
- [29] MARKMAN, Chen ; WOOL, Avishai ; CARDENAS, Alvaro: Temporal Phase Shifts in SCADA Networks. In: *CPS '17 Proceedings of the 2017 Workshop on Cyber-Physical Systems Security and Privacy* (2017), S. 84–89
- [30] KLEINMANN, Amit ; WOOL, Avishai: ACCURATE MODELING OF THE SIEMENS S7SCADA PROTOCOL FOR INTRUSION DETECTION AND DIGITAL FORENSICS. In: *ResearchGate* (2014)
- [31] MORRIS, Thomas ; GAO, Wei: Industrial Control System Traffic Data Sets for Intrusion Detection Research. In: *ICCIP 2014: Critical Infrastructure Protection VIII* (2014), S. 65–78
- [32] SCHMIDT, Thomas: „*Recognizing Anomalies in Protocols of Safety Networks: Schneider Electric's TriStation (RAPSN SETS)*“ vom 21.06.2018. https://www.bsi.bund.de/DE/Themen/Industrie_KRITIS/ICS/Tools/RAPSN_SETS/RAPSN_SETS_node.html, 2018. – Accessed: 07.03.2019
- [33] AG, Siemens: „*Kommunikation mit SIMATIC*“ vom Oktober 1999. http://www.kleissler-online.de/Siemens/s7komm_d.pdf, 1999. – Accessed: 03.01.2019

- [34] BAUER, Camille: „*Modbus Grundlagen*“ vom 03.08.2006. <https://www.camillebauer.com/src/download/ModbusGrundlagen.pdf>, 2006. – Accessed: 07.01.2019
- [35] GROUP, Ethercat T.: „*EtherCAT – Der Ethernet-Feldbus*“ vom 21.09.2003. https://www.ethercat.org/pdf/ethercat_d.pdf, 09 2003. – Accessed: 09.01.2019
- [36] ELECTRONICS, Saia-Burgess: „*26-860_DE_Handbuch_ProfibusDP_02*“ vom 24.01.2014. [https://www.sbc-support.com/index.php?id=1460&tx_srcproducts_srcproducts\[file\]=26-860_DE_Handbuch_ProfibusDP_02.pdf](https://www.sbc-support.com/index.php?id=1460&tx_srcproducts_srcproducts[file]=26-860_DE_Handbuch_ProfibusDP_02.pdf), 2014. – Accessed: 15.01.2019
- [37] MDUDEK-ICS: „*TRISIS-TRITON-HATMAN*“ vom 27.11.2018. <https://github.com/MDudek-ICS/TRISIS-TRITON-HATMAN>, 2018. – Accessed: 24.02.2019
- [38] CANADA, Government: „*Public Safety Canada Industrial Control Systems Security Events: 2019 ICS Security Symposium*“ vom 15.05.2019. <https://www.publicsafety.gc.ca/cnt/ntnl-scrnt/cbr-scrnt/ndstrl-cntrl-sstms/vnts-en.aspxgnd>, 2019. – Accessed: 12.09.2019
- [39] SCHMIDT, Thomas: „*Public Safety Canada Industrial Control Systems Security Events: 2019 ICS Security Symposium*“ vom 25.05.2019. https://nems-sngu.hre-ehr.gc.ca/cigateway/_layouts/15/WopiFrame.aspx?sourcedoc=/cigateway/community_documents_ics/PEI%20-%202019-05-27-30/ICS_Symposium_Charlottetown_2019_English_Presentations_-_Tues_May_28.PDF&action=default, 2019. – Accessed: 12.09.2019
- [40] ED: *BRANAGRAM MINING*. 2012. – Documentation for Branium Refinery
- [41] MIRU, Gyorgy: „*The Siemens S7 Communication - Part 2 Job Requests and Ack Data*“ vom 10.06.2017. <http://gmiru.com/article/s7comm-part2/>, 2017. – Accessed: 31.01.2019
- [42] AG, Siemens: „*SIMATIC NET S7-Programmierschnittstelle*“ vom November 2002. https://cache.industry.siemens.com/dl/files/203/13649203/att_38729/v1/mn_s7api_d.pdf, 2002. – Accessed: 24.01.2019
- [43] REVISION, OISF: „*Suricata User Guide*“ vom November 2016. <https://suricata.readthedocs.io/en/suricata-4.1.4/rules/intro.html>, 2016. – Accessed: 29.05.2019

A Liste der PCAPS

A.1 Liste der PCAPS für den Entwurf des Frameworks (2017QUT_S7Comm dataset)

1. 20161219132813_control_set
 - describes a regular operating procedure without incidents
2. 20161215163606_s7_process_attacks
 - describes an operational sequence that is manipulated by an attacker

A.2 Liste der erstellten PCAPS für die Anwendung der Regeln

1. 20190416-1823_HUB_AHMI+HMI+Set_value_from_HMI.pcapng
 - 2 cycles run by AHMI
 - HMI connected to the network
 - set pressure values via HMI
 - 2 cycles: ca. 30'
2. 20190417-1007_HUB_AHMI+HMI_20190417000725.pcapng
 - 2 cycles run by AHMI
 - HMI connected to the network
 - 2 cycles: ca. 30'
3. 20190417-1047_*_AHMI+HMI_20190417004700.pcapng
 - 2 cycles run by AHMI
 - HMI connected to the network
 - 2 cycles: ca. 30'
4. 20190417-1517_*_HMI_20190417051717.pcapng
 - 1 cycles run manually by Operator through HMI

- AHMI not connected to the network
 - 1 cycles: ca. 16'
5. 20190417-2*_HUB_AHMI+EWS-I+Read_tag_table_201904171*.pcapng
- 1 cycles by AHMI
 - EWS I connected to the network and monitoring the Default tag table starting in the first phase until after the cycle ended
 - 1 cycles: ca. 16'
6. 20190417-2323_*_AHMI+EWS-I+Read_tag_table_20190417132312.pcapng
- 1 cycles by AHMI
 - EWS I connected to the network and monitoring the Default tag table starting at the en of the first phase until after the cycle ended
 - 1 cycles: ca. 16'
7. 20190418-1603_*_EWS-I+Stop_CPU_20190418060327.pcapng
- Stop the CPU from TIA portal without being online
8. 20190418-1624_*_EWS-I+Start_CPU_20190418062416.pcapng
- Start the CPU from TIA portal without being online
9. 20190418-1628_*_EWS-I+Online+Stop_CPU+Offline_20190418062847.pcapng
- Go online
 - stop the CPU from TIA portal
 - go offline
10. 20190418-1637_*_EWS-I+Online+Start_CPU+Offline_20190418063704.pcapng
- Go online
 - start the CPU from TIA portal
 - go offline
11. 20190418-1710_*_AHMI+EWS-I+Process_Running+Stop_CPU_20190418071056.pcapng
- Stop the CPU from TIA portal without being online while AHMI is running a process
12. 20190418-1715_*_AHMI+EWS-I+Process_Running_Inconsistent_State+Start_CPU_2019041807153.pcapng
- Start the CPU from TIA portal without being online while AHMI is connected
 - process is in an inconsistent state
13. 20190419-0024_*_EWS-I+Start_CPU+Already_Started_20190418142416.pcapng

- Start the CPU from TIA portal without being online when it's already running
14. 20190419-0026_*_EWS-I+Stop_CPU+Already_Stopped_20190418142619.pcapng
- Stop the CPU from TIA portal without being online when it's already stopped
15. 20190419-0209_*_EWS-I+Comparing_online_with_offline+no_difference_20190418160923.pcapng
- Compare offline with online version from EWS I without any differences
16. 20190419-0214_*_EWS-I+Comparing_online_with_offline+differences_in_Code_20190418161413.p
- Compare offline with online version from EWS I with differences in code (Main [OB1]: 2, 6, 16, 17, 18, 21, 22, 25, 26)
17. 20190419-0241_*_EWS-I+Going_online_20190418164136.pcapng
- Going online from EWS I
18. 20190419-0244_*_EWS-I+Going_offline_20190418164408.pcapng
- Going offline from EWS I
19. 20190429-2352_*_AHMI+EWS-I+Download_all+consistent_20190429135217.pcapng
- Download all from EWS I while AHMI present; no changes to the program made
20. 20190429-2359_*_EWS-I+Download_Software_all_blocks_20190429135947.pcapng
- Download Software (all blocks) from EWS I
21. 20190430-0004_*_EWS-I+Download_Software+changes_20190429140414.pcapng
- Download Software from EWS I
 - changes made to the program
22. 20190430-0007_*_EWS-I+Download_Hardware_configuration_20190429140708.pcapng
- Download Hardware configuration from EWS I
 - no changes made
23. 20190430-1905_*_EWS-I+Download_all+Password_20190430090522.pcapng
- Download all from EWS I
 - setting password "Whitehat"
24. 20190430-2335_*_EWS-I+Download_Software+Password_20190430133548.pcapng
- Download Software from EWS I
 - registering with password "Whitehat"
25. 20190430-2344_*_EWS-I+Download_All+No_Password_20190430134427.pcapng

- Download All from EWS I
 - without password
26. 20190501-0013_*_EWS-I+Stop+Copy_Ram_to_Rom+Start_20190430141311.pcapng
- Stop CPU
 - Copy values from RAM to ROM
 - Start CPU
27. 20190501-0027_*_EWS-I+Firmware_update_20190430142735.pcapng
- Firmware update to version 3.2.7
28. 20190501-0135_*_AHMI_Attacker+DoS_Communication_Setup_20190430153523.pcapng
- AHMI connected
 - Attacker connects 12 times successful
29. 20190501-0141_*_AHMI_Attacker+DoS_Communication_Setup_20190430154111.pcapng
- AHMI connected
 - Attacker connects 12 times successful with idle time of 2s between attempts
30. 20190501-0145_*_Attacker+DoS_Communication_Setup_20190430154510.pcapng
- Attacker connects 12 times successful with idle time of 2s between attempts

B Regeln für das Protokoll s7comm

```
# Suricata rules for anomaly detection in the S7comm protocol
#
# The rules consist of four parts of rules:
5 # Part 1 – simple rules:
# It captures packets sent from an unauthenticated host to the master PLC.
# Part 2 – Protocol rules:
# Captures packets that contain execution commands and that execute an alert
#   if the packets were not sent to or from an authenticated host. If the
#   packets were sent to or from an authenticated host, they are logged.
# Part 3 – Threshold rules:
10 # Captures packets with commands that are sent at a rate highly different
#   from the average.
# Part 4 – Address range rules:
# Captures packets that send values outside the assigned address range for
#   the data type.
# Part 5 – Generic rules:
# These rules alert on any TCP packet that is sent to the
15 # Controller for testing purposes. It is not for using in production
# The filter part contains a unique filter for each rule of these parts to
#   limit the number of warning/log events.
```

```
# Variablen
# $S7_Master = [10.10.10.10]
20 # $S7_Slave = [10.10.10.11, 10.10.10.12, 10.10.10.13]
# $S7_EWS = [10.10.10.30, 10.10.10.31]
# $S7_HMI = [10.10.10.20, 10.10.10.48]
# $S7_CLIENT = [10.10.10.20, 10.10.10.30, 10.10.10.31, 10.10.10.48]
# $S7_SERVER = '$S7_Master'
25 # $S7_PORT = Listening Communication Controller Port 102
```

Listing B.1 Variablen mit IP-Zuordnung

===== Simple Rules =====

```

#===== Group 9 =====
# Messages/ Alerts on commands about Authenticate-Operations sent to
  $S7_SERVER.
## => File: 20190430-1905_Profishark_EWS-I+Download_all+
      Password_20190430090522.pcapng
## => File: 20190501-1018_Profishark_EWS-I+
      Download_all_resetting_password_to_unprotected+
      typing_password_20190501001824.pcapng
30 # Security Function (Subfunction PLC Password)
# Alert on any connection request Security Function (PLC Password) sent to a
  PLC on TCP/$S7_PORT from an unauthorized host.
alert tcp-pkt !$S7_CLIENT any -> $S7_SERVER $S7_PORT (msg: "ALDI PLC
  Password Request to Server attempt from non authorized Host"; flow:
  to_server,established; content:"|32 07|"; offset:7; depth:2; content:"|00
  01 12|"; offset:17; depth:3; content:"|45 01|"; offset:22; depth:2;
  classtype:bad-unknown; sid:2534320901 rev:2;)

35 # Alert on any connection response sent from a PLC on TCP/$S7_PORT to an
  unauthorized host.
alert tcp-pkt $S7_SERVER $S7_PORT -> !$S7_CLIENT any (msg: "ALDI Password
  Response from Server attempt to non authorized Host"; flow:to_client,
  established; content:"|32 07|"; offset:7; depth:2; content:"|00 01 12|";
  offset:17; depth:3; content:"|45 01|"; offset:22; depth:2; classtype:
  bad-unknown; sid:2534320902; rev:2;)

```

Listing B.2 Security Function Rules

```

#===== Group 12 =====
# Messages/ Alerts on commands about Setup Communication-Operations sent to
  $S7_SERVER.
## => File: 20190501-0135_Profishark_AHMI_Attacker+
      DoS_Communication_Setup_20190430153523.pcapng
40 ## => File: 20190501-0141_Profishark_AHMI_Attacker+
      DoS_Communication_Setup_20190430154111.pcapng
## => File: 20190501-0145_Profishark_Attacker+
      DoS_Communication_Setup_20190430154510.pcapng

# Alert on any connection request sent to a PLC on TCP/$S7_PORT from an
  unauthorized host.
alert tcp-pkt !$S7_CLIENT any -> $S7_SERVER $S7_PORT (msg: "ALDI Setup
  Communication Request to Server attempt from non authorized Host"; flow:
  to_server,established; dsize:>20; content:"|32 01|"; offset:7; depth:2;
  content:"|F0|"; offset:17; depth:1; classtype:bad-unknown; sid
  :2534321201; rev:2;)

45 # Alert on any connection response sent from a PLC on TCP/$S7_PORT to an
  unauthorized host.

```

```

alert tcp-pkt $S7_SERVER $S7_PORT -> !$S7_CLIENT any (msg: "ALDI Setup
Communication Response from Server attempt to non authorized Host"; flow:
to_client,established; dsize:>20; content:"|32 03|"; offset:7; depth:2;
content:"|F0|"; offset:19; depth:1; classtype:bad-unknown; sid
:2534321202; rev:2;)

```

Listing B.3 Setup Communication Rules

==== Protocol Rules ====

```

#===== Group 1 =====
50 # Messages/ Alerts on commands about Download-Operations sent to $S7_SERVER.
## => File: 20190429-2352_Profishark_AHMI+EWS-I+Download_all+
consistent_20190429135217.pcapng
## => File: 20190429-2359_Profishark_EWS-I+
Download_Software_all_blocks_20190429135947.pcapng
## => File: 20190430-0004_Profishark_EWS-I+Download_Software+
changes_20190429140414.pcapng
## => File: 20190430-0007_Profishark_EWS-I+
Download_Hardware_configuration_20190429140708.pcapng
55 ## => File: 20190430-1905_Profishark_EWS-I+Download_all+
Password_20190430090522.pcapng
## => File: 20190430-2335_Profishark_EWS-I+Download_Software+
Password_20190430133548.pcapng
## => File: 20190430-2344_Profishark_EWS-I+Download_All+
No_Password_20190430134427.pcapng
## => File: 20190501-1010_HUB_EWS-I+
Download_all_setting_password_20190501001004v
## => File: 20190501-1018_Profishark_EWS-I+
Download_all_resetting_password_to_unprotected+
typing_password_20190501001824.pcapng
60 # Message on any Connection Request Download sent to a PLC on TCP/ $S7_PORT
from an authorized host (only log).
alert tcp-pkt $S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI Function '
Request Download' from authorized Host"; flow:to_server,established;
content:"|32 01|"; offset:7; depth:2; content:"|1A|"; offset:17; depth:1;
classtype: not-suspicious; sid:2534320101; rev:2;)

# Message on Response to execute a request download sent by a PLC on TCP/
$S7_PORT to an authorized host (only log).
65 alert tcp-pkt $S7_SERVER $S7_PORT -> $S7_EWS any (msg: "ALDI Command
Response 'Request Download' from Server to authorized Host"; flow:
to_client,established; content:"|32 03|"; offset:7; depth:2; content:"|1A
|"; offset:19; depth:1; classtype: not-suspicious; sid:2534320102; rev
:2;)

# Alert on any Connection Request Download sent to a PLC on TCP/ $S7_PORT
from an unauthorized host.
alert tcp-pkt !$S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI Function '

```

```

Request Download' from unauthorized Host"; flow:to_server,established;
content:"|32 01|"; offset:7; depth:2; content:"|1A|"; offset:17; depth:1;
classtype:bad-unknown; sid:2534320103; rev:2;)

70 # Alert on Response to execute a request download sent by a PLC on TCP/
    $S7_PORT to an unauthorized host.
    alert tcp-pkt $S7_SERVER $S7_PORT -> !$S7_EWS any (msg: "ALDI Command
        Response 'Request Download' from Server to unauthorized Host"; flow:
        to_client,established; content:"|32 03|"; offset:7; depth:2; content:"|1A
        |"; offset:19; depth:1; classtype:bad-unknown; sid:2534320104; rev:2;)

# Message on any Connection Request Download sent to a PLC on TCP/ $S7_PORT
    from an authorized host (only log).
75 alert tcp-pkt $S7_SERVER $S7_PORT -> $S7_EWS any (msg: "ALDI Function '
    Download Block' from authorized Host"; flow:to_client,established;
    content:"|32 01|"; offset:7; depth:2; content:"|1B|"; offset:17; depth:1;
    classtype: not-suspicious; sid:2534320105; rev:2;)

# Message on Response to execute a request download sent by a PLC on TCP/
    $S7_PORT to an authorized host (only log).
    alert tcp-pkt $S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI Command
        Response 'Download Block' from Server to authorized Host"; flow:to_server
        ,established; content:"|32 03|"; offset:7; depth:2; content:"|1B|";
        offset:19; depth:1; classtype: not-suspicious; sid:2534320106; rev:2;)

80 # Alert on any Connection Request Download sent to a PLC on TCP/ $S7_PORT
    from an unauthorized host.
    alert tcp-pkt $S7_SERVER $S7_PORT -> !$S7_EWS any (msg: "ALDI Function '
        Download Block' from unauthorized Host"; flow:to_client,established;
        content:"|32 01|"; offset:7; depth:2; content:"|1B|"; offset:17; depth:1;
        classtype:bad-unknown; sid:2534320107; rev:2;)

# Alert on Response to execute a request download sent by a PLC on TCP/
    $S7_PORT to an unauthorized host.
    alert tcp-pkt !$S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI Command
        Response 'Download Block' from Server to unauthorized Host"; flow:
        to_server,established; content:"|32 03|"; offset:7; depth:2; content:"|1B
        |"; offset:19; depth:1; classtype:bad-unknown; sid:2534320108; rev:2;)

85 # Message on any Connection Request Download sent to a PLC on TCP/ $S7_PORT
    from an authorized host (only log).
    alert tcp-pkt $S7_SERVER $S7_PORT -> $S7_EWS any (msg: "ALDI Function '
        Download Ended' from authorized Host"; flow:to_client,established;
        content:"|32 01|"; offset:7; depth:2; content:"|1C|"; offset:17; depth:1;
        classtype: not-suspicious; sid:2534320109; rev:2;)

90 # Message on Response to execute a request download sent by a PLC on TCP/
    $S7_PORT to an authorized host (only log).
    alert tcp-pkt $S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI Command

```

```

Response 'Download Ended' from Server to authorized Host"; flow:to_server
,established; content:"|32 03|"; offset:7; depth:2; content:"|1C|";
offset:19; depth:1; classtype: not-suspicious; sid:2534320110; rev:2;)

# Alert on any Connection Request Download sent to a PLC on TCP/ $S7_PORT
from an unauthorized host.
alert tcp-pkt $S7_SERVER $S7_PORT -> !$S7_EWS any (msg: "ALDI Function '
Download Ended' from unauthorized Host"; flow:to_client,established;
content:"|32 01|"; offset:7; depth:2; content:"|1C|"; offset:17; depth:1;
classtype:bad-unknown; sid:2534320111; rev:2;)

95 # Alert on Response to execute a request download sent by a PLC on TCP/
$S7_PORT to an unauthorized host.
alert tcp-pkt !$S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI Command
Response 'Download Ended' from Server to unauthorized Host"; flow:
to_server,established; content:"|32 03|"; offset:7; depth:2; content:"|1C
|"; offset:19; depth:1; classtype:bad-unknown; sid:2534320112; rev:2;)

```

Listing B.4 Download Rules

```

#===== Group 2 =====
100 #Messages/ Alerts on commands about Upload-Operations sent to $S7_SERVER.
## => File: 20190419-0214_HUB_EWS-I+Comparing_online_with_offline+
differences_in_Code_20190418161413.pcap

# Start Upload
# Message on any Connection Request a Start Upload, which is sent to a PLC
on TCP/ $S7_PORT from an authorized host (only log).
105 alert tcp-pkt $S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI Function 'Start
Upload' from authorized Host"; flow:to_server,established; content:"|32
01|"; offset:7; depth:2; content:"|1D|"; offset:17; depth:1; classtype:
not-suspicious; sid:2534320201; rev:2;)

# Message on Response to execute a request Start Upload that a PLC sends to
an authorized host on TCP/ $S7_PORT (only log).
alert tcp-pkt $S7_SERVER $S7_PORT -> $S7_EWS any (msg: "ALDI Command
Response 'Start Upload' from Server to authorized Host"; flow:to_client,
established; content:"|32 03|"; offset:7; depth:2; content:"|1D|"; offset
:19; depth:1; classtype: not-suspicious; sid:2534320202; rev:2;)

110 # Alert on any Connection Request a Start Upload sent to a PLC on TCP/
$S7_PORT from an unauthorized host.
alert tcp-pkt !$S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI Function '
Start Upload' from unauthorized Host"; flow:to_server,established;
content:"|32 01|"; offset:7; depth:2; content:"|1D|"; offset:17; depth:1;
classtype:bad-unknown; sid:2534320203; rev:2;)

# Alert on Response to execute a Start Upload sent by a PLC on TCP/ $S7_PORT
to an unauthorized host.
alert tcp-pkt $S7_SERVER $S7_PORT -> !$S7_EWS any (msg: "ALDI Command
Response 'Start Upload' from Server to unauthorized Host"; flow:to_client

```

```

, established; content:"I32 03I"; offset:7; depth:2; content:"I1DI";
offset:19; depth:1; classtype:bad-unknown; sid:2534320204; rev:2;)
115

# Upload
# Message on any Connection Request a Start Upload, which is sent to a PLC
on TCP/ $S7_PORT from an authorized host (only log).
alert tcp-pkt $S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI Function '
Upload' from authorized Host"; flow:to_server, established; content:"I32
01I"; offset:7; depth:2; content:"I1EI"; offset:17; depth:1; classtype:
not-suspicious; sid:2534320205; rev:2;)
120

# Message on Response to execute a request Start Upload that a PLC sends to
an authorized host on TCP/ $S7_PORT (only log).
alert tcp-pkt $S7_SERVER $S7_PORT -> $S7_EWS any (msg: "ALDI Command
Response 'Upload' from Server to authorized Host"; flow:to_client,
established; content:"I32 03I"; offset:7; depth:2; content:"I1EI"; offset
:19; depth:1; classtype: not-suspicious; sid:2534320206; rev:2;)

# Alert on any Connection Request a Start Upload sent to a PLC on TCP/
$S7_PORT from an unauthorized host.
125 alert tcp-pkt !$S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI Function '
Upload' from unauthorized Host"; flow:to_server, established; content:"I32
01I"; offset:7; depth:2; content:"I1EI"; offset:17; depth:1; classtype:
bad-unknown; sid:2534320207; rev:2;)

# Alert on Response to execute a Start Upload sent by a PLC on TCP/ $S7_PORT
to an unauthorized host.
alert tcp-pkt $S7_SERVER $S7_PORT -> !$S7_EWS any (msg: "ALDI Command
Response 'Upload' from Server to unauthorized Host"; flow:to_client,
established; content:"I32 03I"; offset:7; depth:2; content:"I1EI"; offset
:19; depth:1; classtype:bad-unknown; sid:2534320208; rev:2;)
130

# End Upload
# Message on any Connection Request a Start Upload, which is sent to a PLC
on TCP/ $S7_PORT from an authorized host (only log).
alert tcp-pkt $S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI Function 'End
Upload' from authorized Host"; flow:to_server, established; content:"I32
01I"; offset:7; depth:2; content:"I1FI"; offset:17; depth:1; classtype:
not-suspicious; sid:2534320209; rev:2;)

135 # Message on Response to execute a request Start Upload that a PLC sends to
an authorized host on TCP/ $S7_PORT (only log).
alert tcp-pkt $S7_SERVER $S7_PORT -> $S7_EWS any (msg: "ALDI Command
Response 'End Upload' from Server to authorized Host"; flow:to_client,
established; content:"I32 03I"; offset:7; depth:2; content:"I1FI"; offset
:19; depth:1; classtype: not-suspicious; sid:2534320210; rev:2;)

# Alert on any Connection Request a Start Upload sent to a PLC on TCP/
$S7_PORT from an unauthorized host.

```

```

alert tcp-pkt !$S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI Function 'End
Upload' from unauthorized Host"; flow:to_server,established; content:"|32
01|"; offset:7; depth:2; content:"|1F|"; offset:17; depth:1; classtype:
bad-unknown; sid:2534320211; rev:2;)
140 # Alert on Response to execute a Start Upload sent by a PLC on TCP/ $S7_PORT
to an unauthorized host.
alert tcp-pkt $S7_SERVER $S7_PORT -> !$S7_EWS any (msg: "ALDI Command
Response 'End Upload' from Server to unauthorized Host"; flow:to_client,
established; content:"|32 03|"; offset:7; depth:2; content:"|1F|"; offset
:19;depth:1; classtype:bad-unknown; sid:2534320212; rev:2;)

```

Listing B.5 Upload Rules

```

#==== Group 5 =====
#Messages/ Alerts on commands about Time Functions sent to $S7_SERVER.
145 ## => File: s7comm_reading_setting_plc_time.pcap
# Time functions
# Message on any connection request Time Functions sent to a PLC on
TCP/$S7_PORT from an authorized host (only log).
alert tcp-pkt $S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI S7 list Time
Function 'set clock' detected from authorized Host"; flow:to_server,
established; content:"|32 07|"; offset:7; depth:2; content:"|00 01 12|";
offset:17; depth:3; content:"|47 02|"; offset:22; depth:2; classtype:
not-suspicious; sid:2534320501; rev:2;)

150 # Message on Response to execute a request Time Functions sent from a PLC
on TCP/$S7_PORT to an authorized host (only log).
alert tcp-pkt $S7_SERVER $S7_PORT -> $S7_EWS any (msg: "ALDI S7 list Time
Function Response 'set clock' from server to authorized Host"; flow:
to_client,established; content:"|32 07|"; offset:7; depth:2; content:"|00
01 12|"; offset:17; depth:3; content:"|87 02|"; offset:22; depth:2;
classtype: not-suspicious; sid:2534320502; rev:2;)

# Alert on any connection request Time Functions sent to a PLC on
TCP/$S7_PORT from an unauthorized host.
alert tcp-pkt !$S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI S7 list Time
Function 'set clock' detected from unauthorized Host"; flow:to_server,
established; content:"|32 07|"; offset:7; depth:2; content:"|00 01 12|";
offset:17; depth:3; content:"|47 02|"; offset:22; depth:2; classtype:
bad-unknown; sid:2534320503; rev:2;)

155 # Alert on Response to execute a request Time Functions sent from a PLC on
TCP/$S7_PORT to an unauthorized host.
alert tcp-pkt $S7_SERVER $S7_PORT -> !$S7_EWS any (msg: "ALDI S7 list Time
Function Response 'set clock' from server to authorized Host"; flow:
to_client,established; content:"|32 07|"; offset:7; depth:2; content:"|00
01 12|"; offset:17; depth:3; content:"|87 02|"; offset:22; depth:2;
classtype:bad-unknown; sid:2534320504; rev:2;)

```

Listing B.6 Time Function Rules


```

#===== Group 7 =====
# Messages/ Alerts on commands about general Write-Operations sent to
  $S7_SERVER.
160 ## => File:20190416-1823_HUB_AHMI+HMI+Set_value_from_HMI.pcapng
  ## => File: 20190417-1047_HUB_AHMI+HMI_20190417004700.pcapng
  ## => File: 20190417-1517_HUB_HMI_20190417051717.pcapng
  ## => File: 20190417-2150_HUB_AHMI+EWS-I+Read_tag_table_20190417115017.
    pcapng
  ## => File: 20190417-2212_HUB_AHMI+EWS-I+Read_tag_table_20190417121233.
    pcapng
165 ## => File: 20190417-2323_Profishark_AHMI+EWS-I+
    Read_tag_table_20190417132312.pcapng

# Message on any connection request a write command sent to a PLC on TCP/
  $S7_PORT from an authorized host (only log).
alert tcp-pkt $S7_CLIENT any -> $S7_SERVER $S7_PORT (msg: "ALDI Command '
  Write Var' from authorized Host"; flow:to_server,established; dsize:>20;
  content:"I32 01I"; offset:7; depth:2; content:"I05I"; offset:17; depth:1;
  classtype: not-suspicious; sid:2534320701; rev:2;)

170 # Message on Response to execute a write command sent by a PLC on TCP/
  $S7_PORT from an unauthorized host (only log).
alert tcp-pkt $S7_SERVER $S7_PORT -> $S7_CLIENT any (msg: "ALDI Command
  Response 'Write Var' from Server to authorized Host"; flow:to_client,
  established; dsize:>20; content:"I32 03I"; offset:7; depth:2; content:"
  I05I"; offset:19; depth:1; content:"IFFI"; offset:21; depth:1; classtype:
  not-suspicious; sid:2534320702; rev:2;)

# Alert on any connection request a write command sent to a PLC on TCP/
  $S7_PORT from an unauthorized host.
alert tcp-pkt !$S7_CLIENT any -> $S7_SERVER $S7_PORT (msg: "ALDI Command '
  Write Var' from unauthorized Host"; flow:to_server,established; dsize
  :>20; content:"I32 01I"; offset:7; depth:2; content:"I05I"; offset:17;
  depth:1; classtype:bad-unknown; sid:2534320703; rev:2;)
175 # Alert on Response to execute a write command sent by a PLC on TCP/
  $S7_PORT from an unauthorized host.
alert tcp-pkt $S7_SERVER $S7_PORT -> !$S7_CLIENT any (msg: "ALDI Command
  Response 'Write Var' from Server to unauthorized Host"; flow:to_client,
  established; dsize:>20; content:"I32 03I"; offset:7; depth:2; content:"
  I05I"; offset:19; depth:1; content:"IFFI"; offset:21; depth:1; classtype:
  bad-unknown; sid:2534320704; rev:2;)

```

Listing B.7 Value Rules

```

#===== Group 10 =====
Messages/ Alerts on commands about Working-Operations sent to \ $S7_SERVER.
180 ## => File: 20190418-1603_Profishark_EWS-I+Stop_CPU_20190418060327.pcapng
  ## => File: 20190418-1624_Profishark_EWS-I+Start_CPU_20190418062416.pcapng
  ## => File: 20190418-1628_Profishark_EWS-I+Online+Stop_CPU+
    Offline_20190418062847.pcapng

```

```

## => File: 20190418-1637_Profishark_EWS-I+Online+Start_CPU+
Offline_20190418063704.pcapng
## => File: 20190418-1710_Profishark_AHMI+EWS-I+Process_Running+
Stop_CPU_20190418071056.pcapng
185 ## => File: 20190418-1715_Profishark_AHMI+EWS-I+
Process_Running_Inconsistent_State+Start_CPU_20190418071534.pcapng
## => File: 20190419-0024_Profishark_EWS-I+Start_CPU+
Already_Started_20190418142416.pcapng
## => File: 20190419-0026_Profishark_EWS-I+Stop_CPU+
Already_Stopped_20190418142619.pcapng
## => File: 20190501-0013_Profishark_EWS-I+Stop+Copy_Ram_to_Rom+
Start_20190430141311.pcapng

190 # Message on request of a PLC start sent to a PLC on TCP/ $S7_PORT from an
authorized host (only log).
alert tcp-pkt $S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI Function 'PLC
Start' from authorized Host"; flow:to_server,established; content:"|32
01|"; offset:7; depth:2; content:"|28|"; offset:17; depth:1; classtype:
not-suspicious; sid:2534321001; rev:2;)

# Message on Response a PLC start sent by a PLC on TCP/ $S7_PORT to an
authorized host (only log).
alert tcp-pkt $S7_SERVER $S7_PORT -> $S7_EWS any (msg: "ALDI Function
Response 'PLC Start' from Server to authorized Host"; flow:to_client,
established; content:"|32 03|"; offset:7; depth:2; content:"|28|"; offset
:19; depth:1; classtype: not-suspicious; sid:2534321002; rev:2;)

195 # Alert on request of a PLC start sent to a PLC on TCP/ $S7_PORT from an
unauthorized host.
alert tcp-pkt !$S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI Function 'PLC
Start' from unauthorized Host"; flow:to_server,established; content:"|32
01|"; offset:7; depth:2; content:"|28|"; offset:17; depth:1; classtype:
bad-unknown; sid:2534321003; rev:2;)

# Alert on Response to execute a PLC start sent by a PLC on TCP/ $S7_PORT to
an unauthorized host.
200 alert tcp-pkt $S7_SERVER $S7_PORT -> !$S7_EWS any (msg: "ALDI Function
Response 'PLC Start' from Server to unauthorized Host"; flow:to_client,
established; content:"|32 03|"; offset:7; depth:2; content:"|28|"; offset
:19; depth:1; classtype:bad-unknown; sid:2534321004; rev:2;)

# Message on request of a PLC stop sent to a PLC on TCP/ $S7_PORT from an
authorized host (only log).
alert tcp-pkt $S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI Command 'PLC
Stop' from authorized Host"; flow:to_server,established; content:"|32 01|
"; offset:7; depth:2; content:"|29|"; offset:17; depth:1; classtype:
not-suspicious; sid:2534321005; rev:2;)

205 # Message on Response a PLC stop sent by a PLC on TCP/ $S7_PORT to an
authorized host (only log).

```

```

alert tcp-pkt $S7_SERVER $S7_PORT -> $S7_EWS any (msg: "ALDI Command
Response 'PLC Stop' from Server to authorized Host"; flow:to_client,
established; content:"|32 03|"; offset:7; depth:2; content:"|29|"; offset
:19; depth:1; classtype: not-suspicious; sid:2534321006; rev:2;)

# Alert on request of a PLC stop sent to a PLC on TCP/ $S7_PORT from an
unauthorized host.
210 alert tcp-pkt !$S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI Command 'PLC
Stop' from unauthorized Host"; flow:to_server,established; content:"|32
01|"; offset:7; depth:2; content:"|29|"; offset:17; depth:1; classtype:
bad-unknown; sid:2534321007; rev:2;)

# Alert on Response to execute a PLC stop sent by a PLC on TCP/ $S7_PORT to
an unauthorized host.
alert tcp-pkt $S7_SERVER $S7_PORT -> !$S7_EWS any (msg: "ALDI Command
Response 'PLC Stop' from Server to unauthorized Host"; flow:to_client,
established; content:"|32 03|"; offset:7; depth:2; content:"|29|"; offset
:19; depth:1; classtype:bad-unknown; sid:2534321008; rev:2;)

```

Listing B.8 Mode Rules

```

#==== Group 13 =====
215 #Messages/ Alerts on commands about Functions sent to $S7_SERVER
## => File: 20190417-2212_HUB_AHMI+EWS-I+Read_tag_table_20190417121233.
pcapng
## => File: 20190418-1628_Profishark_EWS-I+Online+Stop_CPU+
Offline_20190418062847.pcapng
## => File: 20190418-1637_Profishark_EWS-I+Online+Start_CPU+
Offline_20190418063704.pcapng
## => File: 20190419-0209_Profishark_EWS-I+Comparing_online_with_offline+
no_difference_20190418160923.pcapng
220 ## => File: 20190419-0214_Profishark_EWS-I+Comparing_online_with_offline+
differences_in_Code_20190418161413.pcapng
## => File: 20190419-0241_Profishark_EWS-I+Going_online_20190418164136.
pcapng
## => File: 20190429-2352_Profishark_AHMI+EWS-I+Download_all+
consistent_20190429135217.pcapng
## => File: 20190429-2359_Profishark_EWS-I+
Download_Software_all_blocks_20190429135947.pcapng
## => File: 20190430-1905_Profishark_EWS-I+Download_all+
Password_20190430090522.pcapng
225 ## => File: 20190501-1018_Profishark_EWS-I+
Download_all_resetting_password_to_unprotected+
typing_password_20190501001824.pcapng

# Block Functions (OB/ DB/ SDB/ FC/ SFC/ FB/ SFB)
# Message on any connection request Block Functions sent to a PLC on
TCP/$S7_PORT from an authorized host (only log).
alert tcp-pkt $S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI S7 list Block
Function command detected from authorized Host"; flow:to_server,
established; content:"|32 07|"; offset:7; depth:2; content:"|00 01 12|";

```

```

offset:17; depth:3; content:"|43 02|"; offset:22; depth:2; content:"|30
45|"; offset:29; depth:2; classtype: not-suspicious; sid:2534321301; rev
:2;)
230 # Message on Response to execute a request Block Functions sent from a PLC
on TCP/$S7_PORT to an authorized host (only log).
alert tcp-pkt $S7_SERVER $S7_PORT -> $S7_EWS any (msg: "ALDI S7 list Block
Function command Response from Server to authorized Host"; flow:to_client
, established; content:"|32 07|"; offset:7; depth:2; content:"|00 01 12|";
offset:17; depth:3; content:"|83|"; offset:22; depth:1; classtype:
not-suspicious; sid:2534321302; rev:2;)

# Alert on any connection request Block Functions sent to a PLC on
TCP/$S7_PORT from an unauthorized host.
235 alert tcp-pkt !$S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI S7 list Block
Function command detected from unauthorized Host"; flow:to_server,
established; content:"|32 07|"; offset:7; depth:2; content:"|00 01 12|";
offset:17; depth:3; content:"|43 02|"; offset:22; depth:2; content:"|30
45|"; offset:29; depth:2; classtype:bad-unknown; sid:2534321303; rev:2;)

# Alert on Response to execute a request Block Functions sent from a PLC on
TCP/$S7_PORT to an unauthorized host.
alert tcp-pkt $S7_SERVER $S7_PORT -> !$S7_EWS any (msg: "ALDI S7 list Block
Function command Response from Server to authorized Host"; flow:to_client
, established; content:"|32 07|"; offset:7; depth:2; content:"|00 01 12|";
offset:17; depth:3; content:"|83|"; offset:22; depth:1; classtype:
bad-unknown; sid:2534321304; rev:2;)

240 # CPU Functions (SZL)
# Message on any connection request CPU Functions (SZL) sent to a PLC on
TCP/$S7_PORT from an authorized host (only log).
alert tcp-pkt $S7_CLIENT any -> $S7_SERVER $S7_PORT (msg: "ALDI S7 list CPU
Functions (SZL) command detected from authorized Host"; flow:to_server,
established; content:"|32 07|"; offset:7; depth:2; content:"|00 01 12|";
offset:17; depth:3; content:"|44|"; offset:22; depth:1; classtype:
not-suspicious; sid:2534321305; rev:2;)

245 # Message on Response to execute a request CPU Functions sent from a PLC
on TCP/$S7_PORT to an authorized host (only log).
alert tcp-pkt $S7_SERVER $S7_PORT -> $S7_CLIENT any (msg: "ALDI S7 list CPU
Functions (SZL) command Response from Server to authorized Host"; flow:
to_client, established; content:"|32 07|"; offset:7; depth:2; content:"|00
01 12|"; offset:17; depth:3; content:"|84|"; offset:22; depth:1;
classtype: not-suspicious; sid:2534321306; rev:2;)

# Alert on any connection request CPU Functions sent to a PLC on
TCP/$S7_PORT from an unauthorized host.
alert tcp-pkt !$S7_CLIENT any -> $S7_SERVER $S7_PORT (msg: "ALDI S7 list CPU
Functions (SZL) command detected from unauthorized Host"; flow:to_server
, established; content:"|32 07|"; offset:7; depth:2; content:"|00 01 12|";

```

```

    offset:17; depth:3; content:"|44|"; offset:22; depth:1; classtype:
    bad-unknown; sid:2534321307; rev:2;)
250
# Alert on Response to execute a request CPU Functions sent from a PLC on
TCP/$S7_PORT to an unauthorized host.
alert tcp-pkt $S7_SERVER $S7_PORT -> !$S7_CLIENT any (msg: "ALDI S7 list CPU
Functions (SZL) command Response from Server to authorized Host"; flow:
to_client,established; content:"|32 07|"; offset:7; depth:2; content:"|00
01 12|"; offset:17; depth:3; content:"|84|"; offset:22; depth:1;
classtype:bad-unknown; sid:2534321308; rev:2;)

255 # Programmer Commands
# Message on any connection request CPU Functions (SZL) sent to a PLC on
TCP/$S7_PORT from an authorized host (only log).
alert tcp-pkt $S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI S7 list
Programmer Commands detected from authorized Host"; flow:to_server,
established; content:"|32 07|"; offset:7; depth:2; content:"|00 01 12|";
offset:17; depth:3; content:"|41|"; offset:22; depth:1; classtype:
not-suspicious; sid:2534321309; rev:2;)

# Message on Response to execute a request CPU Functions sent from a PLC
on TCP/$S7_PORT to an authorized host (only log).
260 alert tcp-pkt $S7_SERVER $S7_PORT -> $S7_EWS any (msg: "ALDI S7 list
Programmer Commands Response from Server to authorized Host"; flow:
to_client,established; content:"|32 07|"; offset:7; depth:2; content:"|00
01 12|"; offset:17; depth:3; content:"|81|"; offset:22; depth:1;
classtype: not-suspicious; sid:2534321310; rev:2;)

# Alert on any connection request CPU Functions sent to a PLC on
TCP/$S7_PORT from an unauthorized host.
alert tcp-pkt !$S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI S7 list
Programmer Commands detected from unauthorized Host"; flow:to_server,
established; content:"|32 07|"; offset:7; depth:2; content:"|00 01 12|";
offset:17; depth:3; content:"|41|"; offset:22; depth:1; classtype:
bad-unknown; sid:2534321311; rev:2;)

265 # Alert on Response to execute a request CPU Functions sent from a PLC on
TCP/$S7_PORT to an unauthorized host.
alert tcp-pkt $S7_SERVER $S7_PORT -> !$S7_EWS any (msg: "ALDI S7 list
Programmer Commands Response from Server to authorized Host"; flow:
to_client,established; content:"|32 07|"; offset:7; depth:2; content:"|00
01 12|"; offset:17; depth:3; content:"|81|"; offset:22; depth:1;
classtype:bad-unknown; sid:2534321312; rev:2;)

```

Listing B.9 Function Rules

```

#==== Group 14 ====
# Messages/ Alerts on commands about memory reset or copy
270 ## => File:20190501-0013_Profishark_EWS-I+Stop+Copy_Ram_to_Rom+
Start_20190430141311

```

```

## => File: 20190501-1122_Profishark_EWS_I+Memory_Reset_20190501012217

# Programmer Commands
# Message on any connection request Programmer Commands (erase) sent to a
  PLC on TCP/$S7_PORT from an authorized host (only log).
275 alert tcp-pkt $S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI S7 list
  Programmer Command 'erase' detected from authorized Host"; flow:to_server
  ,established; content:"|32 07|"; offset:7; depth:2; content:"|00 01 12|";
  offset:17; depth:3; content:"|41 0C|"; offset:22; depth:2; classtype:
  not-suspicious; sid:2534321401; rev:2;)

# Message on Response to execute a request Programmer Commands (erase) sent
  from a PLC on TCP/$S7_PORT to an authorized host (only log).
alert tcp-pkt $S7_SERVER $S7_PORT -> $S7_EWS any (msg: "ALDI S7 list
  Programmer Command 'erase' Response from Server to authorized Host"; flow
  :to_client,established; content:"|32 07|"; offset:7; depth:2; content:"
  |00 01 12|"; offset:17; depth:3; content:"|81 0C|"; offset:22; depth:2;
  classtype: not-suspicious; sid:2534321402; rev:2;)

280 # Alert on any connection request Programmer Commands (erase) sent to a PLC
  on TCP/$S7_PORT from an unauthorized host.
alert tcp-pkt !$S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI S7 list
  Programmer Command 'erase' detected from unauthorized Host"; flow:
  to_server,established; content:"|32 07|"; offset:7; depth:2; content:"|00
  01 12|"; offset:17; depth:3; content:"|41 0C|"; offset:22; depth:2;
  classtype:bad-unknown; sid:2534321403; rev:2;)

# Alert on Response to execute a request Programmer Commands (erase) sent
  from a PLC on TCP/$S7_PORT to an unauthorized host.
alert tcp-pkt $S7_SERVER $S7_PORT -> !$S7_EWS any (msg: "ALDI S7 list
  Programmer Command Response 'erase' from Server to authorized Host"; flow
  :to_client,established; content:"|32 07|"; offset:7; depth:2; content:"
  |00 01 12|"; offset:17; depth:3; content:"|81 0C|"; offset:22; depth:2;
  classtype:bad-unknown; sid:2534321404; rev:2;)
285

# Message on any connection request Programmer Commands (forces) sent to a
  PLC on TCP/$S7_PORT from an authorized host (only log).
alert tcp-pkt $S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI S7 list
  Programmer Command 'forces' detected from authorized Host"; flow:
  to_server,established; content:"|32 07|"; offset:7; depth:2; content:"|00
  01 12|"; offset:17; depth:3; content:"|41 10|"; offset:22; depth:2;
  classtype: not-suspicious; sid:2534321405; rev:2;)

290 # Message on Response to execute a request Programmer Commands (forces)
  sent from a PLC on TCP/$S7_PORT to an authorized host (only log).
alert tcp-pkt $S7_SERVER $S7_PORT -> $S7_EWS any (msg: "ALDI S7 list
  Programmer Command Response 'forces' from Server to authorized Host";
  flow:to_client,established; content:"|32 07|"; offset:7; depth:2; content
  :"|00 01 12|"; offset:17; depth:3; content:"|81 10|"; offset:22; depth:2;
  classtype: not-suspicious; sid:2534321406; rev:2;)

```



```

# Alert on any connection request Programmer Commands (forces) sent to a PLC
  on TCP/$S7_PORT from an unauthorized host.
alert tcp-pkt !$S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI S7 list
  Programmer Command 'forces' detected from unauthorized Host"; flow:
  to_server, established; content:"|32 07|"; offset:7; depth:2; content:"|00
  01 12|"; offset:17; depth:3; content:"|41 10|"; offset:22; depth:2;
  classtype:bad-unknown; sid:2534321407; rev:2;)
295
# Alert on Response to execute a request Programmer Commands (forces) sent
  from a PLC on TCP/$S7_PORT to an unauthorized host.
alert tcp-pkt $S7_SERVER $S7_PORT -> !$S7_EWS any (msg: "ALDI S7 list
  Programmer Command Response 'forces' from Server to authorized Host";
  flow:to_client, established; content:"|32 07|"; offset:7; depth:2; content
  :"|00 01 12|"; offset:17; depth:3; content:"|81 10|"; offset:22; depth:2;
  classtype:bad-unknown; sid:2534321408; rev:2;)

```

Listing B.10 Memory Rules

```

#==== Group 15 =====
# Messages/ Alerts on commands reset factory setting. It is the same as
  Group 14_Memory
300 ## => File: 20190501-1130_Profishark_EWS_I+
  Factory_Reset_failed_20190501013037.pcapng

# Programmer Commands
# Message on any connection request Programmer Commands (forces) sent to a
  PLC on TCP/$S7_PORT from an authorized host (only log).
alert tcp-pkt $S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI S7 list
  Programmer Command 'forces' detected from authorized Host"; flow:
  to_server, established; content:"|32 07|"; offset:7; depth:2; content:"|00
  01 12|"; offset:17; depth:3; content:"|41 10|"; offset:22; depth:2;
  classtype: not-suspicious; sid:2534321405; rev:2;)
305
# Message on Response to execute a request Programmer Commands (forces)
  sent from a PLC on TCP/$S7_PORT to an authorized host (only log).
alert tcp-pkt $S7_SERVER $S7_PORT -> $S7_EWS any (msg: "ALDI S7 list
  Programmer Command Response 'forces' from Server to authorized Host";
  flow:to_client, established; content:"|32 07|"; offset:7; depth:2; content
  :"|00 01 12|"; offset:17; depth:3; content:"|81 10|"; offset:22; depth:2;
  classtype: not-suspicious; sid:2534321406; rev:2;)

# Alert on any connection request Programmer Commands (forces) sent to a PLC
  on TCP/$S7_PORT from an unauthorized host.
310 alert tcp-pkt !$S7_EWS any -> $S7_SERVER $S7_PORT (msg: "ALDI S7 list
  Programmer Command 'forces' detected from unauthorized Host"; flow:
  to_server, established; content:"|32 07|"; offset:7; depth:2; content:"|00
  01 12|"; offset:17; depth:3; content:"|41 10|"; offset:22; depth:2;
  classtype:bad-unknown; sid:2534321407; rev:2;)

# Alert on Response to execute a request Programmer Commands (forces) sent

```

```

from a PLC on TCP/$S7_PORT to an unauthorized host.
alert tcp-pkt $S7_SERVER $S7_PORT -> !$S7_EWS any (msg: "ALDI S7 list
Programmer Command Response 'forces' from Server to authorized Host";
flow:to_client,established; content:"|32 07|"; offset:7; depth:2; content
:"|00 01 12|"; offset:17; depth:3; content:"|81 10|"; offset:22; depth:2;
classtype:bad-unknown; sid:2534321408; rev:2;)

```

Listing B.11 Reset Rules

```

#==== Threshold Rules (Advanced) ====
315 # Alert on any unusual high rate of certain commands
# Note: Adjust the threshold values to your usage

# Alert on Connection Request that is sent to a PLC on TCP/$S7_PORT if it
passes the defined threshold: 200 within 60 seconds
alert tcp-pkt any any -> $S7_SERVER $S7_PORT (msg:"ALDI Connection Request
to Server attempt above threshold"; content:"|32 01|"; offset:7; depth:2;
content:"|051|"; offset:17; depth:1; threshold: type threshold, track
by_dst, count 200, seconds 60; classtype:bad-unknown; sid:2534321601; rev
:2;)

```

Listing B.12 Threshold Rules

```

320 #===== Generic Rules =====
#Alert on any packet that is sent within the network on TCP/$S7_PORT
alert tcp-pkt any any <> any $S7_PORT (msg: "ALDI Generic possible S7Comm
Traffic detected"; classtype:bad-unknown; sid:2534320001; rev:1;)

#Alert on any S7Comm packet that is sent to a PLC on TCP/$S7_PORT
325 alert tcp-pkt any any -> $S7_SERVER $S7_PORT (msg: "ALDI Generic S7Comm
Traffic to PLC detected"; content:"|32|"; offset:7; depth:1; classtype:
bad-unknown; sid:2534320002; rev:1;)

#Alert on any S7Comm packet that is sent from a PLC on TCP/$S7_PORT
alert tcp-pkt $S7_SERVER $S7_PORT -> any any (msg: "ALDI Generic S7Comm
Traffic from PLC detected"; content:"|32|"; offset:7; depth:1; classtype:
bad-unknown; sid:2534320003; rev:1;)

```

Listing B.13 Generic Rules

```

#==== Filter ====
# These event filters are used since the original threshold directive is
deprecated.
# For Simple rules:

5 threshold gen_id 1, sig_id 2534320901, type limit, track by_dst, count 3,
seconds 5
threshold gen_id 1, sig_id 2534320902, type limit, track by_dst, count 3,
seconds 5
threshold gen_id 1, sig_id 2534321201, type limit, track by_dst, count 3,
seconds 5

```



```
threshold gen_id 1, sig_id 2534321202, type limit, track by_dst, count 3,
seconds 5
```

```
10 # For Protocol Rules
```

```
threshold gen_id 1, sig_id 2534320101, type limit, track by_dst, count 3,
seconds 5
```

```
threshold gen_id 1, sig_id 2534320102, type limit, track by_dst, count 3,
seconds 5
```

```
threshold gen_id 1, sig_id 2534320103, type limit, track by_dst, count 3,
seconds 5
```

```
15 threshold gen_id 1, sig_id 2534320104, type limit, track by_dst, count 3,
seconds 5
```

```
threshold gen_id 1, sig_id 2534320105, type limit, track by_dst, count 3,
seconds 5
```

```
threshold gen_id 1, sig_id 2534320106, type limit, track by_dst, count 3,
seconds 5
```

```
threshold gen_id 1, sig_id 2534320107, type limit, track by_dst, count 3,
seconds 5
```

```
threshold gen_id 1, sig_id 2534320108, type limit, track by_dst, count 3,
seconds 5
```

```
20 threshold gen_id 1, sig_id 2534320109, type limit, track by_dst, count 3,
seconds 5
```

```
threshold gen_id 1, sig_id 2534320110, type limit, track by_dst, count 3,
seconds 5
```

```
threshold gen_id 1, sig_id 2534320111, type limit, track by_dst, count 3,
seconds 5
```

```
threshold gen_id 1, sig_id 2534320112, type limit, track by_dst, count 3,
seconds 5
```

```
25 threshold gen_id 1, sig_id 2534320201, type limit, track by_dst, count 3,
seconds 5
```

```
threshold gen_id 1, sig_id 2534320202, type limit, track by_dst, count 3,
seconds 5
```

```
threshold gen_id 1, sig_id 2534320203, type limit, track by_dst, count 3,
seconds 5
```

```
threshold gen_id 1, sig_id 2534320204, type limit, track by_dst, count 3,
seconds 5
```

```
threshold gen_id 1, sig_id 2534320205, type limit, track by_dst, count 3,
seconds 5
```

```
30 threshold gen_id 1, sig_id 2534320206, type limit, track by_dst, count 3,
seconds 5
```

```
threshold gen_id 1, sig_id 2534320207, type limit, track by_dst, count 3,
seconds 5
```

```
threshold gen_id 1, sig_id 2534320208, type limit, track by_dst, count 3,
seconds 5
```

```
threshold gen_id 1, sig_id 2534320209, type limit, track by_dst, count 3,
seconds 5
```

```
threshold gen_id 1, sig_id 2534320210, type limit, track by_dst, count 3,
seconds 5
```

```
35 threshold gen_id 1, sig_id 2534320211, type limit, track by_dst, count 3,
```

```

seconds 5
threshold gen_id 1, sig_id 2534320212, type limit, track by_dst, count 3,
seconds 5

threshold gen_id 1, sig_id 2534320501, type limit, track by_dst, count 3,
seconds 5
threshold gen_id 1, sig_id 2534320502, type limit, track by_dst, count 3,
seconds 5
40 threshold gen_id 1, sig_id 2534320503, type limit, track by_dst, count 3,
seconds 5
threshold gen_id 1, sig_id 2534320504, type limit, track by_dst, count 3,
seconds 5

threshold gen_id 1, sig_id 2534320701, type limit, track by_dst, count 3,
seconds 5
threshold gen_id 1, sig_id 2534320702, type limit, track by_dst, count 3,
seconds 5
45 threshold gen_id 1, sig_id 2534320703, type limit, track by_dst, count 3,
seconds 5
threshold gen_id 1, sig_id 2534320704, type limit, track by_dst, count 3,
seconds 5

threshold gen_id 1, sig_id 2534321001, type limit, track by_dst, count 3,
seconds 5
threshold gen_id 1, sig_id 2534321002, type limit, track by_dst, count 3,
seconds 5
50 threshold gen_id 1, sig_id 2534321003, type limit, track by_dst, count 3,
seconds 5
threshold gen_id 1, sig_id 2534321004, type limit, track by_dst, count 3,
seconds 5
threshold gen_id 1, sig_id 2534321005, type limit, track by_dst, count 3,
seconds 5
threshold gen_id 1, sig_id 2534321006, type limit, track by_dst, count 3,
seconds 5
threshold gen_id 1, sig_id 2534321007, type limit, track by_dst, count 3,
seconds 5
55 threshold gen_id 1, sig_id 2534321008, type limit, track by_dst, count 3,
seconds 5

threshold gen_id 1, sig_id 2534321301, type limit, track by_dst, count 3,
seconds 5
threshold gen_id 1, sig_id 2534321302, type limit, track by_dst, count 3,
seconds 5
threshold gen_id 1, sig_id 2534321303, type limit, track by_dst, count 3,
seconds 5
60 threshold gen_id 1, sig_id 2534321304, type limit, track by_dst, count 3,
seconds 5
threshold gen_id 1, sig_id 2534321305, type limit, track by_dst, count 3,
seconds 5
threshold gen_id 1, sig_id 2534321306, type limit, track by_dst, count 3,
seconds 5

```

```
threshold gen_id 1, sig_id 2534321307, type limit, track by_dst, count 3,  
seconds 5  
threshold gen_id 1, sig_id 2534321308, type limit, track by_dst, count 3,  
seconds 5  
65 threshold gen_id 1, sig_id 2534321309, type limit, track by_dst, count 3,  
seconds 5  
threshold gen_id 1, sig_id 2534321310, type limit, track by_dst, count 3,  
seconds 5  
threshold gen_id 1, sig_id 2534321311, type limit, track by_dst, count 3,  
seconds 5  
threshold gen_id 1, sig_id 2534321312, type limit, track by_dst, count 3,  
seconds 5  
  
70 threshold gen_id 1, sig_id 2534321401, type limit, track by_dst, count 3,  
seconds 5  
threshold gen_id 1, sig_id 2534321402, type limit, track by_dst, count 3,  
seconds 5  
threshold gen_id 1, sig_id 2534321403, type limit, track by_dst, count 3,  
seconds 5  
threshold gen_id 1, sig_id 2534321404, type limit, track by_dst, count 3,  
seconds 5  
threshold gen_id 1, sig_id 2534321405, type limit, track by_dst, count 3,  
seconds 5  
75 threshold gen_id 1, sig_id 2534321406, type limit, track by_dst, count 3,  
seconds 5  
threshold gen_id 1, sig_id 2534321407, type limit, track by_dst, count 3,  
seconds 5  
threshold gen_id 1, sig_id 2534321408, type limit, track by_dst, count 3,  
seconds 5
```

Listing B.14 Eventfilter